HOSPITAL DE ESPECIALIDADES

VERAPAZ · COBÁN, ALTA VERAPAZ

MADIÁN

SISTEMA MADIÁN · DOCUMENTACIÓN

Manual técnico

Arquitectura, instalación, despliegue, seguridad y mantenimiento del sistema de gestión de expediente clínico y administrativo del hospital.

React 18 · Node.js 22 · PostgreSQL 16 · Prisma · Docker · GitHub Actions

Versión 1.0 · 26 de octubre de 2026

Universidad Mariano Gálvez de Guatemala · Campus Cobán

Contenido

1. Alcance 5
2. Guía de instalación y configuración del sistema 5
    2.1 Requisitos previos 5
    2.2 Clonar el repositorio 6
    2.3 Configurar las variables de entorno 6
    2.4 Instalación con Docker Compose (recomendada) 8
    2.5 Instalación local sin Docker 9
    2.6 HTTPS en desarrollo y escaneo desde el teléfono 10
    2.7 Verificar la instalación 10
    2.8 Procedimiento de actualización 11
    2.9 Infraestructura del servidor 11
3. Arquitectura y diagramas 13
    3.1 Arquitectura general 13
    3.2 Diagrama de despliegue 14
    3.3 Estructura de carpetas 15
    3.4 Capas del backend y flujo de una petición 17
    3.5 Relación entre módulos 18
    3.6 Flujo de autenticación 19
    3.7 Flujo de acceso al diagnóstico con token temporal 20
    3.8 Flujo de escaneo por teléfono 22
    3.9 Control de acceso por rol 23
    3.10 Especificaciones técnicas 24
4. Modelo de datos 25
    4.1 Motor y convenciones 25
    4.2 Diagrama entidad-relación 25
    4.3 Diagramas de clases 25
    4.4 Entidades 29
    4.5 Relaciones principales 30
    4.6 Restricciones de integridad relevantes 31
    4.7 Ciclos de vida 31
    4.8 Borrado lógico y retención 33
    4.9 Historial de migraciones 33
5. Documentación técnica del backend 34
    5.1 Infraestructura principal 34
    5.2 Middlewares 36
    5.3 Utilidades 37
    5.4 Servicios 39
    5.5 Autenticación y tokens 40
    5.6 Usuarios 42
    5.7 Pacientes 44
    5.8 Documentos del paciente 47
    5.9 Lectura avanzada de expedientes 48
    5.10 Expediente clínico y diagnóstico 49
    5.11 Escaneo móvil 50
6. Documentación técnica del frontend 51
    6.1 Punto de entrada y enrutamiento 51
    6.2 Contexto de autenticación 52
    6.3 Servicio de API 53
    6.4 Hooks 53
    6.5 Utilidades 53
    6.6 Componentes reutilizables 55
    6.7 Páginas 56

6.8 Construcción y servido 57
7. Referencia de la API REST 57
7.1 Autenticación y seguridad (/api/auth) 57
7.2 Usuarios (/api/usuarios) 57
7.3 Pacientes (/api/pacientes) 58
7.4 Expediente clínico (/api/expedientes) 58
7.5 Tratamiento, recetas y referidos 59
7.6 Facturación y gastos (/api/facturacion) 59
7.7 Farmacia (/api/farmacia) 60
7.8 Bitácora de visitas (/api/bitacora) 60
7.9 Reportes (/api/reportes) 60
7.10 Escaneo móvil (/api/escaneo-movil) 61
7.11 Catálogo CIE-10 y sistema 61
7.12 Contrato de errores 61
7.13 Transacciones 62
8. Seguridad 62
8.1 Autenticación 62
8.2 Contraseñas 62
8.3 Autorización por rol 63
8.4 Tokens de acceso temporal 63
8.5 Cifrado 63
8.6 Manejo de archivos 64
8.7 Auditoría 64
8.8 Límites de intentos 64
8.9 Red, CORS y TLS 65
9. Pruebas 65
9.1 Matriz de pruebas 65
9.2 Prueba de integración 68
9.3 Pruebas E2E (Playwright) 68
10. Integración continua 69
10.1 Evidencia de la integración continua 70
11. Operación y mantenimiento 72
11.1 Monitoreo 72
11.2 Respaldos 73
11.3 Scripts de mantenimiento 74
11.4 Rotación de llaves 74
11.5 Notificaciones, lectura avanzada y escaneo móvil 75
12. Guía para desarrolladores 75
12.1 Agregar un endpoint 75
12.2 Agregar un modelo o cambiar el esquema 75
12.3 Agregar un rol 75
12.4 Convenciones observadas 76
13. Limitaciones y riesgos 76
13.1 Riesgos de seguridad 76
13.2 Deuda técnica 78
14. Glosario 79

El documento describe el sistema **Madián**, desarrollado para el Hospital de Especialidades Verapaz, tal como está implementado en el código (commit `3f0bf85`, rama `main`, 26 de octubre de 2026). Está dirigido a administradores de sistemas, desarrolladores y personal de soporte técnico que deban instalar, mantener o ampliar la aplicación. Las rutas, roles, variables y valores por defecto citados se verificaron contra el código fuente.

El manual sigue este orden: guía de instalación, arquitectura con sus diagramas, modelo de datos, documentación técnica del backend y del frontend módulo por módulo, referencia de la API, seguridad, pruebas, integración continua, operación y limitaciones conocidas.

La infraestructura en producción (servidor, dominio, HTTPS, respaldos) no puede capturarse desde el código, por lo que se documenta con diagramas explicativos de elaboración propia. El contenido funcional de cada pantalla se explica en el manual de usuario, no en este documento.

1. Alcance

El sistema es una aplicación web para la gestión del expediente clínico y administrativo de un hospital. Cubre:

- Registro y admisión de pacientes, con historial de ingresos múltiples.
- Expediente clínico con diagnósticos cifrados y control de acceso por token temporal.
- Documentos escaneados y archivos adjuntos del paciente, con escaneo desde el teléfono.
- Tratamientos intrahospitalarios, recetas médicas y clientes referidos.
- Facturación del hospital y de farmacia, gastos del hospital y reportes financieros.
- Inventario y ventas de farmacia.
- Gestión de usuarios, roles y auditoría.

Fuera del alcance de este documento: el uso de cada pantalla (manual de usuario) y la especificación de requerimientos (`docs/Req_y_Estructura.docx`).

2. Guía de instalación y configuración del sistema

2.1 Requisitos previos

Componente	Versión	Para qué se necesita
Docker Engine con Compose v2	actual	Instalación recomendada (todo el sistema en contenedores).
Node.js	22	Desarrollo local sin Docker. Es la versión de CI y de los contenedores.
PostgreSQL	16	Desarrollo local sin Docker. Con Docker se levanta solo.
Git	actual	Obtener el código.
`pg_dump`	16 o superior	Solo si se usará `npm run backup` desde el equipo (ver 11.2).

El proyecto no incluye mediciones de consumo de memoria ni de procesador. El dimensionamiento del servidor lo define el hospital al contratar el VPS (ver el análisis de costos del informe final).

2.2 Clonar el repositorio

```
git clone https://github.com/MaryG0107/hospital-verapaz-sistema.git
cd hospital-verapaz-sistema
```

El repositorio contiene en su raíz las carpetas `backend/` y `frontend/` y un único `docker-compose.yml`, de modo que ambas partes se versionan y se despliegan juntas.

2.3 Configurar las variables de entorno

Hay dos archivos de configuración distintos. El de la raíz alimenta a Docker Compose; el de `backend/` solo se usa al ejecutar el backend fuera de Docker.

Variables del backend

Se leen desde `backend/.env` (desarrollo) o desde el entorno del contenedor (Docker, inyectadas por `docker-compose.yml`). La validación crítica ocurre en `src/config/env.js` al arrancar: si falta una variable obligatoria o es inválida, el proceso termina con código 1 y muestra el detalle.

Variable	Obligatoria	Por defecto	Descripción
`DATABASE_URL`	Sí	—	Cadena de conexión PostgreSQL (`postgresql://usuario:clave@host:5432/hospital_verapaz`).
`JWT_SECRET`	Sí	—	Secreto de firma de los JWT. Mínimo 16 caracteres.
`ENCRYPTION_KEY`	Sí	—	Llave AES-256 en hexadecimal: exactamente 64 caracteres hex (32 bytes).
`JWT_EXPIRES_IN`	No	`8h`	Duración de la sesión (formato de `jsonwebtoken`).
`TEMP_TOKEN_EXPIRES_MIN`	No	`15`	Vigencia de los tokens de acceso temporal, en minutos.
`PASSWORD_RESET_EXPIRES_MIN`	No	`30`	Vigencia del enlace de recuperación de contraseña.
`PASSWORD_RESET_EXPOSE_TOKEN`	No	`false`	Si es `true`, `forgot-password` devuelve el token en la respuesta. Solo para laboratorio.
`NOTIFICACION_PROVEEDOR`	No	`consola`	Adaptador de notificación. Hoy solo existe `consola`; otro valor lanza error.
`APP_URL`	No	`http://localhost:5173`	URL pública del frontend; arma el enlace de recuperación.
`PORT`	No	`4000`	Puerto del backend.
`UPLOAD_MAX_MB_ESTUDIOS`	No	`5`	Tamaño máximo por archivo de estudios del diagnóstico.

Variable	Obligatoria	Por defecto	Descripción
DOCUMENT_MAX_MB	No	20	Tamaño máximo por archivo en documentos del paciente y adjuntos.
DOCUMENT_MAX_PAGES	No	30	Páginas máximas de un documento subido a un paciente existente.
UPLOAD_TIPOS_PERMITIDOS	No	application/pdf,image/png, image/jpeg	Lista de tipos MIME permitidos.
HOSPITAL_TZ, HOSPITAL_UTC_OFFSET	No	America/Guatemala, -06:00	Zona horaria con la que los reportes agrupan por día, semana y mes.
SEED_ADMIN_EMAIL	No	admin@hospitalverapaz.gt	Correo del administrador que crea el seed.
SEED_ADMIN_PASSWORD	No	CambiarEsta123	Contraseña del administrador que crea el seed. Debe cambiarse.
ANTHROPIC_API_KEY	No	—	Activa la lectura avanzada de fichas manuscritas. Sin ella, la función está apagada.
ANTHROPIC_MODEL	No	claude-opus-5	Modelo de visión de la lectura avanzada.

UPLOAD_MAX_MB_ANEXOS aparece en .env.example pero el código no la usa: la función de anexos se fusionó con la de documentos el 23/09/2026.

Variables de Docker Compose (archivo .env en la raíz)

Variable	Por defecto	Uso
POSTGRES_PASSWORD	obligatoria	Clave del usuario verapaz de PostgreSQL.
JWT_SECRET	obligatoria	Se pasa al backend.
ENCRYPTION_KEY	obligatoria	Se pasa al backend.
FRONTEND_PORT	8080	Puerto publicado del frontend.
APP_URL	http://localhost:8080	Enlaces de recuperación de contraseña.
RUN_MIGRATIONS	true	Aplica prisma migrate deploy al arrancar el backend.
RUN_SEED	true	Ejecuta prisma/seed.js al arrancar el backend (ver 13.2, D1).
SEED_ADMIN_EMAIL, SEED_ADMIN_PASSWORD	valores de .env.example	Administrador inicial.

El resto de variables del backend se pueden añadir en la sección environment del servicio backend de docker-compose.yml.

Generar y custodiar la llave de cifrado

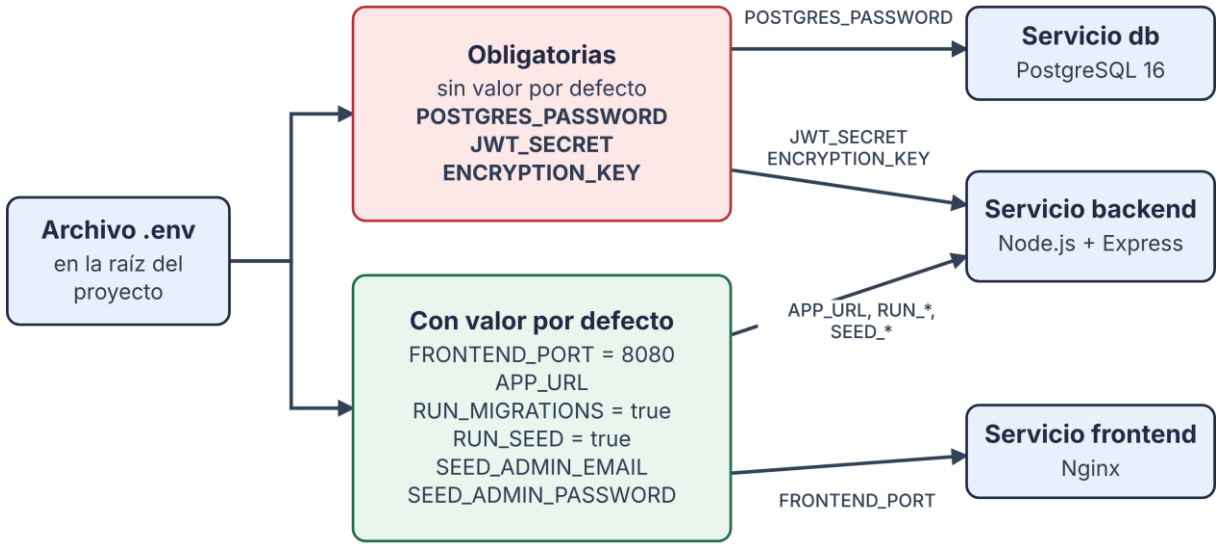
env.js exige que ENCRYPTION_KEY cumpla /^[0-9a-fA-F]{64}$/. Para generar una:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

La llave protege el contenido de cada diagnóstico (`Diagnostico.textoCifrado`), los datos clínicos de egreso (`EgresoClinico.textoCifrado`) y todos los binarios de `uploads/`. **Si la llave se pierde o cambia, esos datos no se pueden recuperar.** El proyecto no tiene mecanismo de recuperación ni de rotación. Debe guardarse en custodia separada del respaldo de la base de datos.

Figura 1

Variables del archivo .env y servicios que las consumen. Se muestran solo los nombres, nunca los valores.



Nota. Elaboración propia.

2.4 Instalación con Docker Compose (recomendada)

```
# 1. Crear la configuración
cp .env.example .env
# Completar: POSTGRES_PASSWORD, JWT_SECRET, ENCRYPTION_KEY, SEED_ADMIN_PASSWORD

# 2. Construir y levantar
docker compose up -d --build

# 3. Verificar
docker compose ps
curl -fsS http://localhost:8080/api/health     # {"status":"ok","db":"ok"}
```

El archivo `docker-compose.yml` define tres servicios y tres volúmenes:

Servicio	Imagen	Puerto publicado	Volúmenes	Notas
`db`	`postgres:16-alpine`	ninguno	`pgdata`	Healthcheck con `pg_isready`. Solo es alcanzable desde la red interna (`db:5432`).

Servicio	Imagen	Puerto publicado	Volúmenes	Notas
`backend`	build de `backend/`	ninguno	`uploads`, `backups`	Usuario no root `verapaz`. Healthcheck contra `/api/health`. Espera a que `db` esté sano.
`frontend`	build de `frontend/`	`${FRONTEND_PORT:-8080}` → 80	ninguno	Nginx con la SPA compilada y proxy de `/api/` al backend.

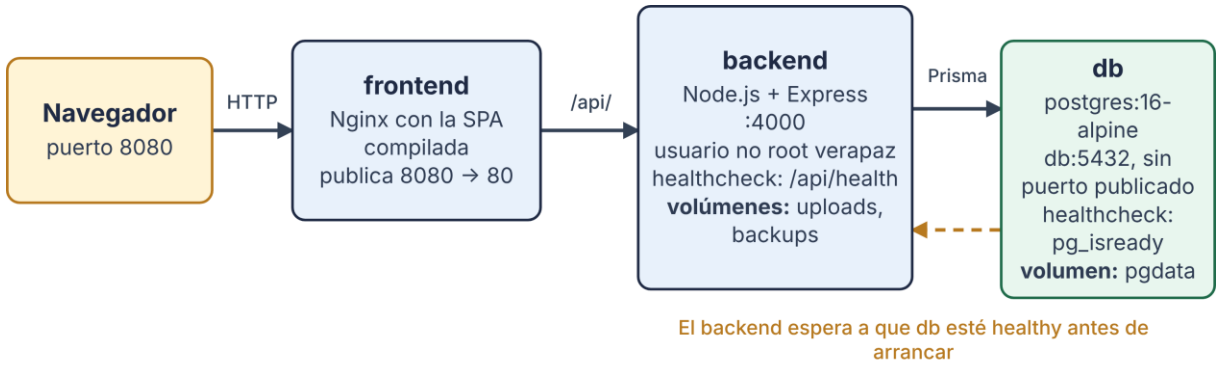
Al arrancar, el contenedor del backend ejecuta en orden `prisma migrate deploy` (si `RUN_MIGRATIONS=true`), `node prisma/seed.js` (si `RUN_SEED=true`) y `node src/server.js`. Terminada la instalación inicial conviene poner `RUN_SEED=false`; `RUN_MIGRATIONS=true` puede mantenerse porque es idempotente.

Operaciones frecuentes:

```
docker compose logs -f backend          # seguir los logs del backend
docker compose exec backend sh          # shell dentro del contenedor
docker compose restart backend          # reinicio
docker compose down                     # detener (conserva los volúmenes)
docker compose down -v                  # borra los volúmenes: base de datos y archivos
```

Figura 2

Servicios de Docker Compose y su comunicación: navegador, frontend (Nginx), backend y base de datos.



Nota. Elaboración propia.

2.5 Instalación local sin Docker

Requiere Node 22 y una instancia de PostgreSQL 16 accesible.

```
# Base de datos (una vez)
createdb hospital_verapaz

# Backend
cd backend
cp .env.example .env            # completar DATABASE_URL, JWT_SECRET, ENCRYPTION_KEY
npm ci                          # el postinstall ejecuta prisma generate
npx prisma migrate deploy       # aplica las migraciones existentes
npm run prisma:seed             # usuarios de prueba, inventario y catálogos
npm run dev                     # watch sobre src/ y schema.prisma, puerto 4000
```

```
# Frontend (otra terminal)
cd frontend
npm ci
npm run dev                     # Vite en http://localhost:5173, proxy /api hacia localhost:4000
```

Notas:

- `npm run prisma:migrate` ejecuta `prisma migrate dev`, que puede crear migraciones y resetear datos. No debe usarse con datos reales; para aplicar migraciones use `npx prisma migrate deploy`.
- `npm run start` ejecuta `node src/server.js` sin modo watch.
- Para crear una migración nueva: `npx prisma migrate dev --name <nombre_descriptivo>` y revisar el SQL generado antes de versionarlo.

2.6 HTTPS en desarrollo y escaneo desde el teléfono

La cámara del navegador (`getUserMedia`) y `crypto.randomUUID()` exigen contexto seguro: HTTPS o `localhost`. Para probar el escaneo desde un teléfono en la misma red hay dos caminos:

- **Certificado autofirmado de desarrollo.** Coloque `dev-key.pem` y `dev-cert.pem` en `frontend/.cert/` (carpeta ignorada por git). `vite.config.js` los detecta y activa HTTPS. Ver `frontend/.cert/README.md`.
- **Túnel HTTPS con ngrok.** Ejecute `ngrok http 8080` (Docker) o `ngrok http 5173` (Vite) y abra el sistema desde la URL del túnel. El código QR usa `window.location.origin`, así que apunta a esa URL. Vite tiene `allowedHosts: true` para aceptar el dominio del túnel.

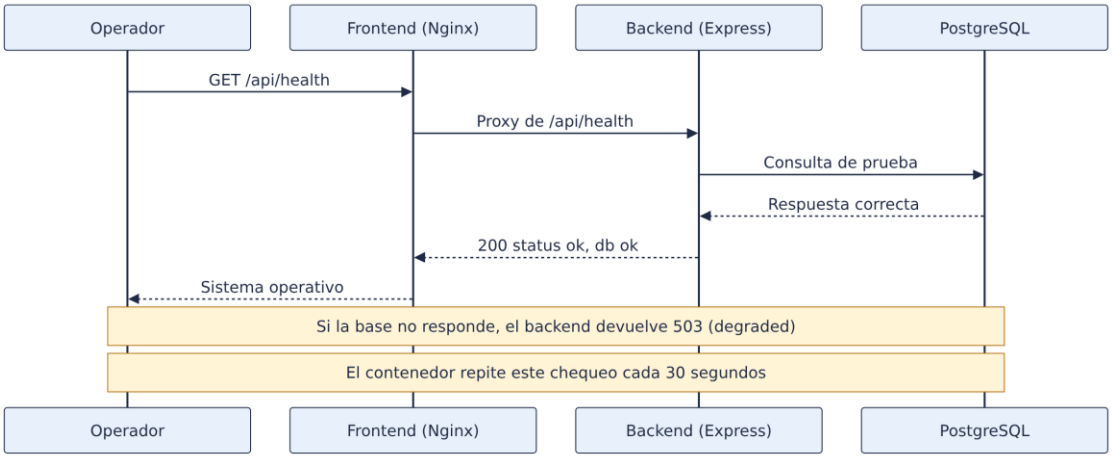
El authtoken de ngrok es una credencial: no debe versionarse ni compartirse.

2.7 Verificar la instalación

1. `docker compose ps` muestra los tres servicios en estado *Up*; `db` y `backend` en *healthy*.
2. `GET /api/health` responde `200` con `{"status":"ok","db":"ok"}`. Si la base de datos no contesta, responde `503` con `{"status":"degraded","db":"error"}`.
3. En el log del backend aparece `Servidor backend Hospital Verapaz escuchando en puerto 4000` y no hay errores de Prisma.
4. La pantalla `/login` carga y acepta las credenciales del administrador definidas en `SEED_ADMIN_EMAIL` y `SEED_ADMIN_PASSWORD`.
5. La tabla `_prisma_migrations` contiene las 24 migraciones aplicadas.

Figura 3

Verificación del estado del sistema mediante el endpoint /api/health.



Nota. Elaboración propia.

2.8 Procedimiento de actualización

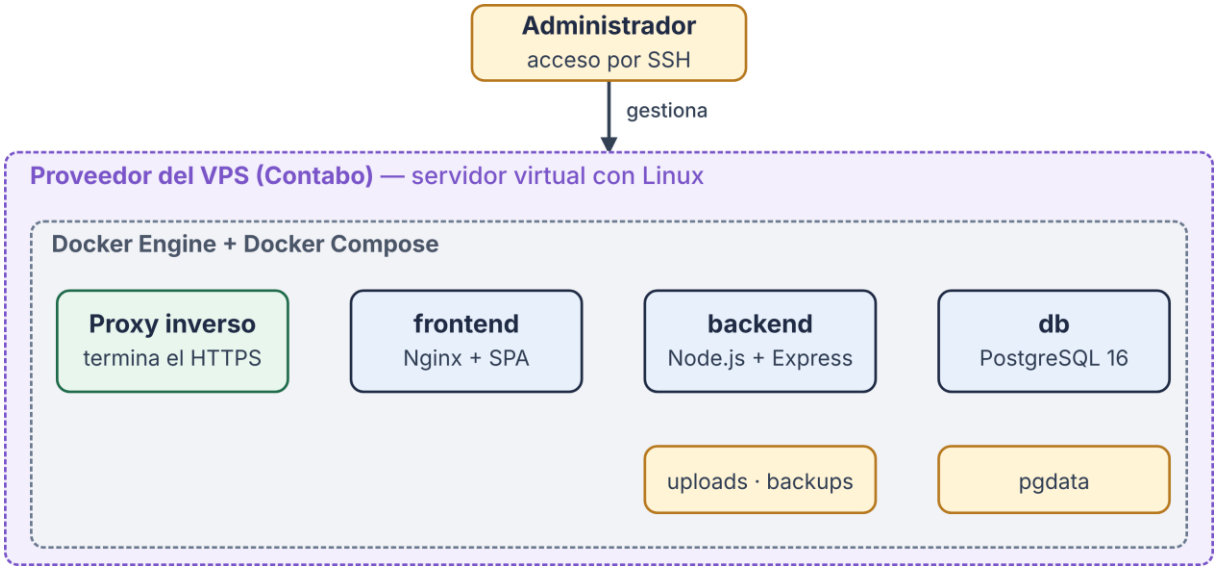
1. Respaldar la base de datos y el volumen `uploads/` (sección 11.2).
2. Obtener el código de la versión objetivo (`git pull` o un artefacto nuevo).
3. Ejecutar `docker compose up -d --build`. Con `RUN_MIGRATIONS=true`, las migraciones pendientes se aplican al arrancar el backend.
4. Verificar `/api/health` y, si el entorno lo permite, ejecutar la prueba de integración (sección 9.2).
5. Revisar `docker compose logs backend` buscando errores de Prisma. El `CMD` del `Dockerfile` encadena los pasos con `;` y no con `&&`, de modo que si `migrate deploy` falla el servidor arranca igualmente con el esquema anterior (ver D14). Si una migración falló, restaure el respaldo antes de reintentar.

2.9 Infraestructura del servidor

El sistema se aloja en un servidor virtual (VPS) con Docker Compose. El HTTPS lo termina un proxy inverso colocado delante del contenedor `frontend`, como describe el diagrama de despliegue de la sección 3.2. Los archivos de configuración de ese proxy y los datos del contrato del VPS no forman parte del repositorio, por eso esta sección se explica mediante diagramas.

Figura 4

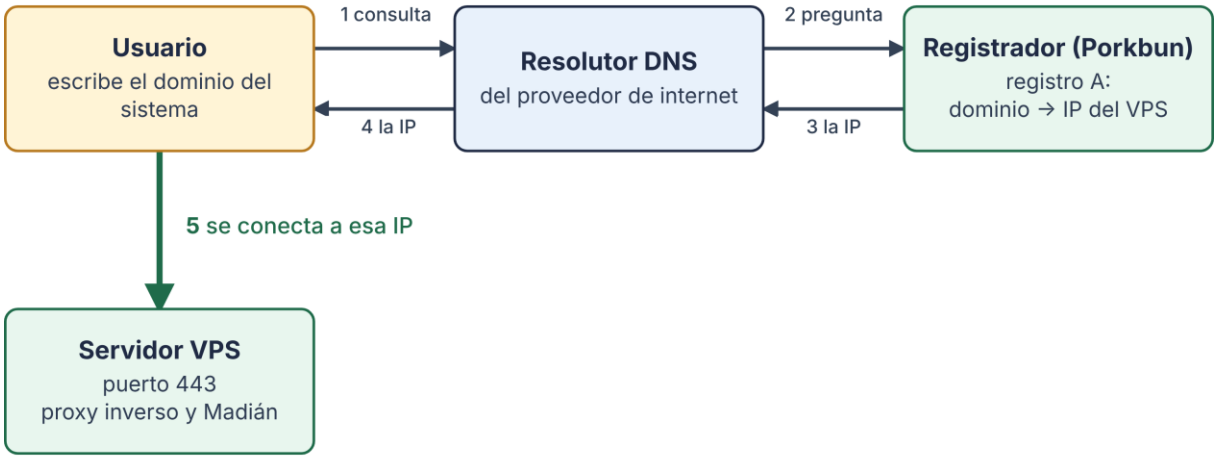
Distribución del sistema dentro del servidor virtual (VPS) con Docker Engine y Docker Compose.



Nota. Elaboración propia.

Figura 5

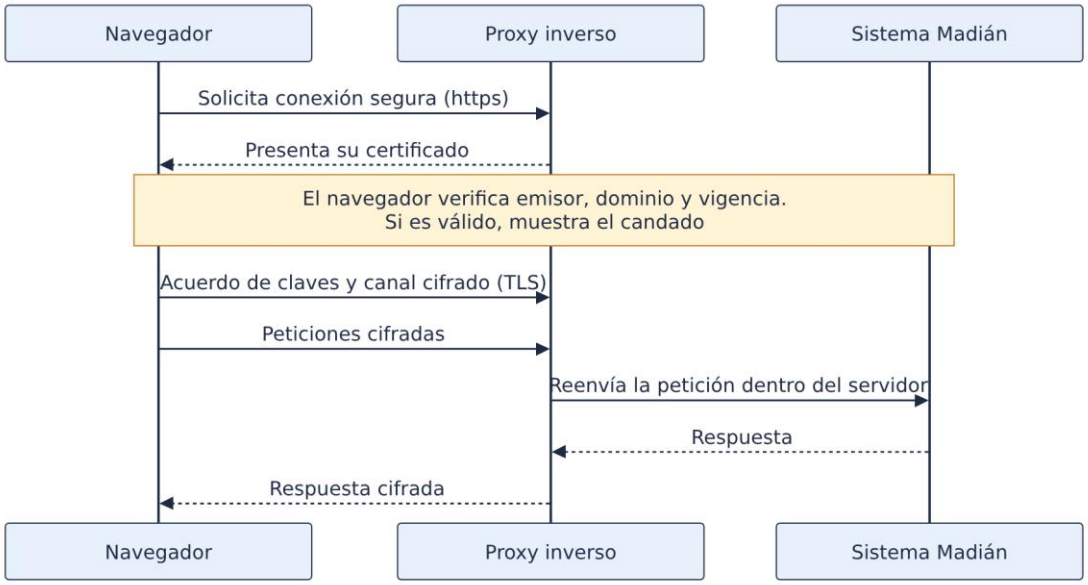
Resolución del nombre de dominio hacia la IP del servidor mediante el registro A.



Nota. Elaboración propia.

Figura 6

Establecimiento de la conexión HTTPS entre el navegador y el proxy inverso.



Nota. Elaboración propia.

3. Arquitectura y diagramas

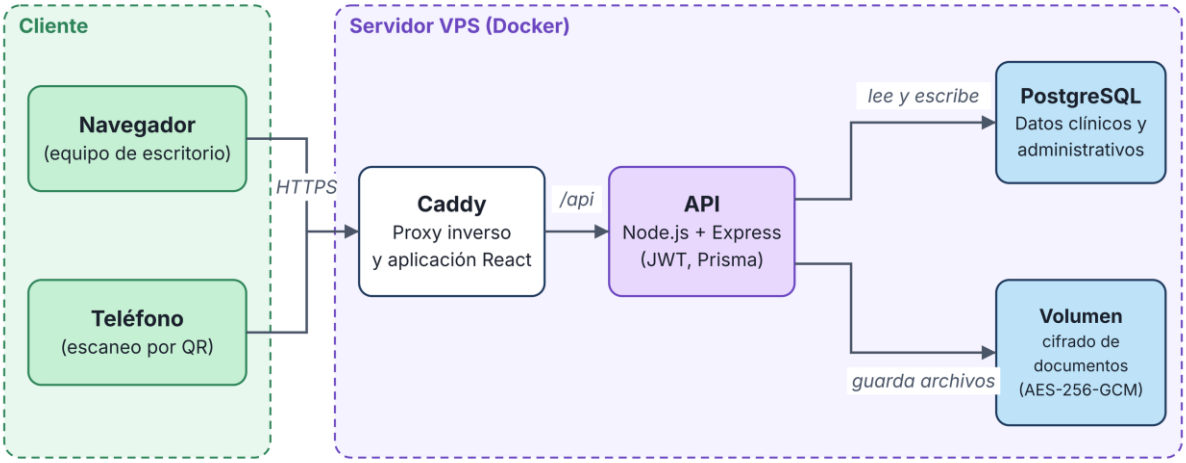
3.1 Arquitectura general

El sistema es una aplicación web de tres capas con una interfaz de página única, una API REST y una base de datos relacional. La API almacena los archivos clínicos cifrados en un volumen del servidor, no en la base de datos.

Aspecto	Clasificación
Patrón general	Arquitectura en capas
Comunicación cliente-servidor	API REST sobre HTTP, con JSON y multipart/form-data
Gestión de sesiones	Sin estado en el servidor: JWT enviado como `Authorization: Bearer`
Autorización	Por rol, en el backend, y adicionalmente por token temporal para el diagnóstico
Almacenamiento de archivos	Volumen local `uploads/`, con archivos cifrados (AES-256-GCM)
ORM	Prisma (abstracción de la base de datos y migraciones)
Escaneo desde teléfono	Sesiones efímeras en memoria del backend

Figura 7

Arquitectura del sistema Madián



Nota. Elaboración propia.

Componentes

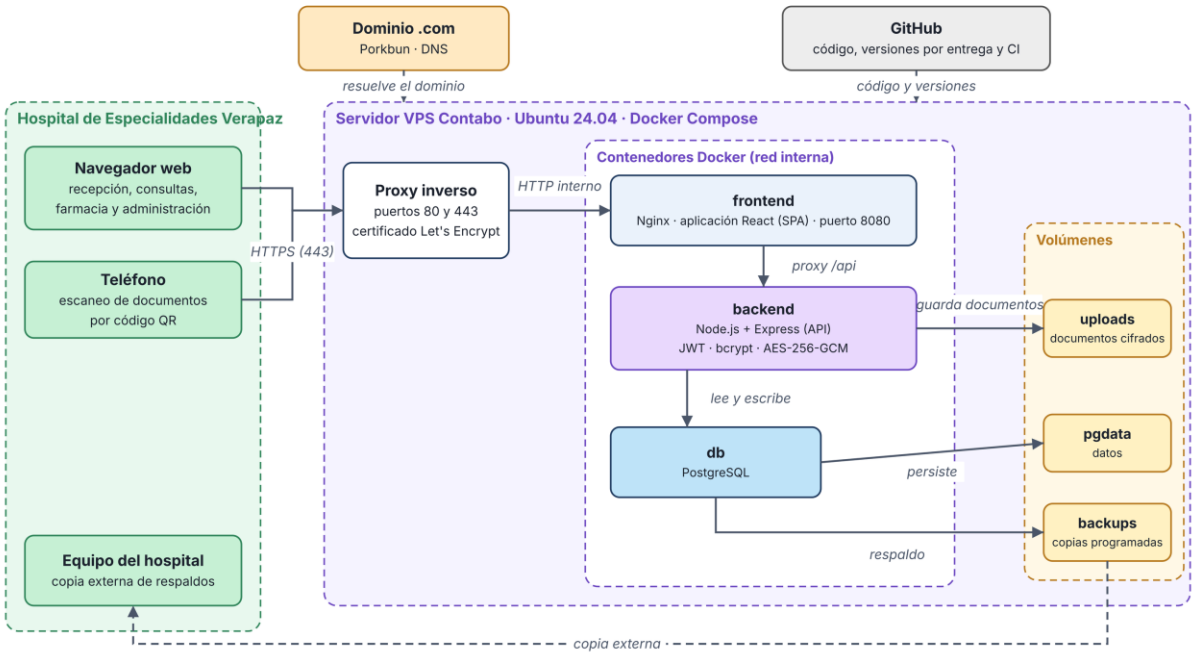
Componente	Tecnología	Responsabilidad	Ubicación
Interfaz (SPA)	React 18.3, Vite 5.4, react-router-dom 6.26, Tailwind 3	Pantallas, validación de formularios, OCR y escaneo en el navegador	`frontend/`
Servidor web	Nginx (`nginx:alpine`)	Sirve la SPA compilada, proxy de `/api/`, corrige el MIME de `.mjs`	`frontend/nginx.conf`
API	Node 22, Express 4.19, express-async-errors	Lógica de negocio, autenticación, autorización y cifrado	`backend/src/`
ORM	Prisma 5 (`@prisma/client`)	Acceso a datos y migraciones	`backend/prisma/`
Base de datos	PostgreSQL 16	Persistencia	servicio `db`
Archivos	Volumen `uploads`	Binarios cifrados (sobres AES-GCM)	`backend/uploads/`
Escaneo móvil	`Map` en memoria con TTL de 10 min	Recibir el PDF desde el teléfono	`escaneoMovil.controller.js`
Lectura avanzada (opcional)	`@anthropic-ai/sdk`	Leer letra manuscrita de la ficha escaneada	`lecturaVision.service.js`

3.2 Diagrama de despliegue

En producción, los navegadores y los teléfonos del hospital se conectan por HTTPS al dominio del sistema. Un proxy inverso con certificado de Let's Encrypt termina la conexión segura y reenvía el tráfico al contenedor `frontend`, que a su vez reenvía `/api/` al contenedor `backend`. La base de datos no se expone fuera de la red interna de Docker. Los volúmenes `pgdata`, `uploads` y `backups` persisten los datos entre reinicios y actualizaciones.

Figura 8

Diagrama de despliegue del sistema en el servidor VPS



Nota. Elaboración propia.

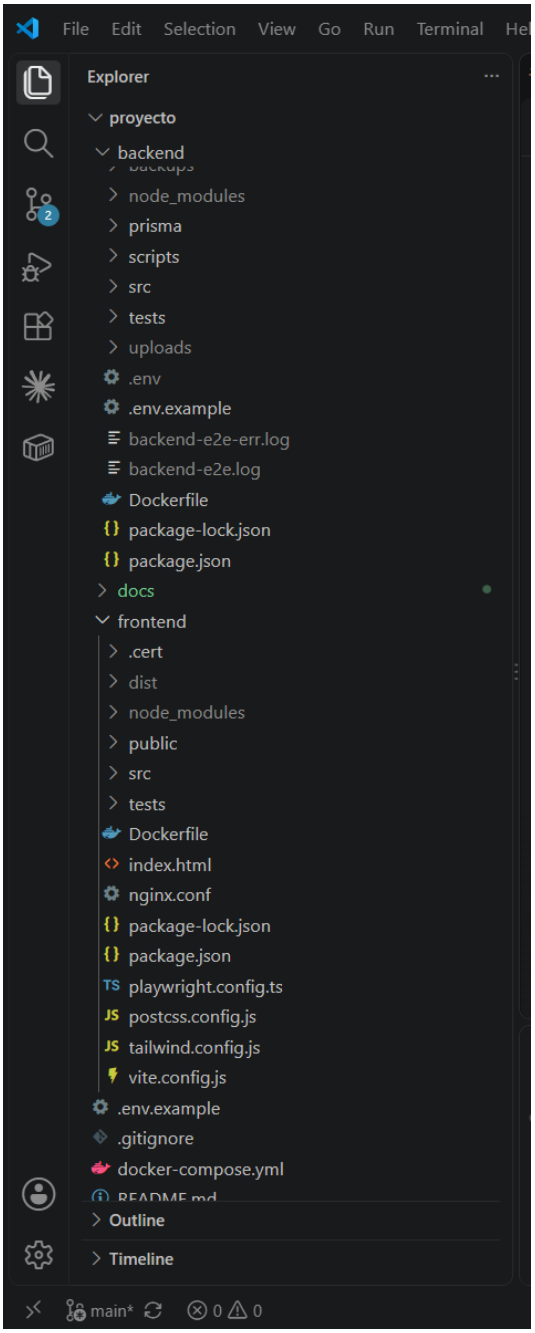
Solo el frontend publica un puerto hacia el host (`${FRONTEND_PORT:-8080}`). El backend escucha en el 4000 y PostgreSQL en el 5432, ambos solo dentro de la red de Compose.

3.3 Estructura de carpetas

La figura siguiente muestra el árbol de carpetas del proyecto en el editor de código, con `backend`, `frontend` y `docs` bajo la carpeta `proyecto`.

Figura 9

Estructura de carpetas del proyecto en el editor de código



Nota. Captura del editor de código con la carpeta del proyecto.

Raíz del repositorio

```
hospital-verapaz-sistema/
├── backend/                API, Prisma, scripts y pruebas
├── frontend/               SPA React, Nginx y pruebas
├── docs/                   documentación del proyecto (incluye este manual)
├── .github/workflows/ci.yml pipeline de CI/CD
├── docker-compose.yml      orquestación de db, backend y frontend
└── .env.example            variables para Docker Compose
```

Backend

```
backend/
├── prisma/
│   ├── schema.prisma            modelo de datos (fuente de verdad)
│   ├── migrations/              24 migraciones SQL versionadas
│   ├── seed.js                  usuarios de prueba, inventario y catálogos
│   └── data/cie10.js            catálogo CIE-10 (subconjunto curado)
├── scripts/
│   ├── backup.js                respaldo con pg_dump
│   ├── migrar-egreso-cifrado.js        migración histórica (una vez)
│   ├── migrar-anexos-a-documentos.js   migración histórica (una vez)
│   └── recifrar-archivos.js             recifrado de archivos en claro (una vez)
├── src/
│   ├── app.js                   Express: montaje de rutas, health, manejador de errores
│   ├── server.js                listen en PORT (4000 por defecto)
│   ├── config/                  env.js (validación), prisma.js (cliente único)
│   ├── routes/                  un router por módulo (13 archivos)
│   ├── controllers/             un controlador por módulo (17 archivos)
│   ├── middlewares/             auth.middleware.js, rate-limit.middleware.js
│   ├── services/                facturación, actividad, notificación, lectura visión
│   └── utils/                   crypto, archivos, roles, paginación, periodos
├── tests/
│   ├── unit/                    pruebas unitarias (node:test)
│   └── integracion.smoke.mjs    prueba de integración contra un stack levantado
├── Dockerfile
└── package.json
```

Frontend

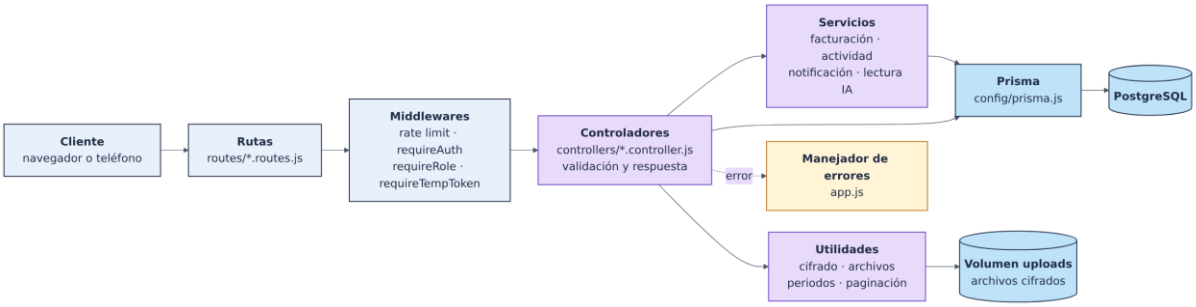
```
frontend/
├── src/
│   ├── main.jsx                 punto de entrada: Router y AuthProvider
│   ├── App.jsx                  rutas y protección por módulo
│   ├── pages/                   una página por módulo (15 archivos)
│   ├── components/              Layout, formularios, escaneo, impresión
│   ├── context/AuthContext.jsx  estado de sesión
│   ├── services/api.js          cliente HTTP único
│   ├── hooks/                   useFetch, usePaginatedFetch
│   ├── utils/                   validadores, roles, fechas, OCR, escaneo, PDF
│   ├── styles/                  index.css (Tailwind) y tokens.js (colores)
│   └── assets/                  logos de Madián y del hospital
├── tests/                       pruebas unitarias (.test.mjs) y E2E (.spec.ts)
├── nginx.conf
├── vite.config.js
├── playwright.config.ts
└── Dockerfile
```

3.4 Capas del backend y flujo de una petición

El backend separa responsabilidades en capas. Las rutas declaran qué rol puede llamar a cada endpoint; los middlewares verifican la identidad y los permisos; los controladores validan la entrada y arman la respuesta HTTP; los servicios y las utilidades concentran la lógica reutilizable (facturación, cifrado, archivos); Prisma accede a PostgreSQL.

Figura 10

Capas del backend y recorrido de una petición



Nota. Elaboración propia.

El recorrido de una petición autenticada es el siguiente:

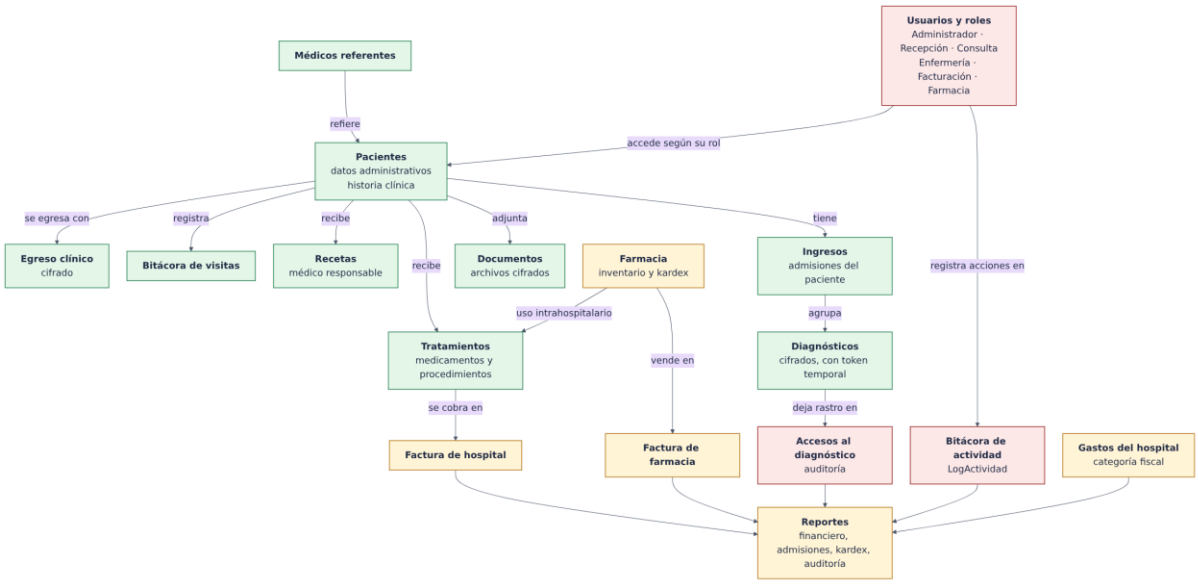
1. El navegador envía `Authorization: Bearer <JWT>` a `/api/...`. Nginx reenvía la petición a `backend:4000`.
2. `requireAuth` verifica la firma del JWT con `JWT_SECRET`, vuelve a leer el usuario y sus roles actuales desde la base de datos y deja `req.user = { id, nombre, roles }`. También actualiza `Usuario.ultimaActividad` sin bloquear la respuesta.
3. `requireRole(...)` compara los roles del usuario con los permitidos por la ruta.
4. Para el expediente, `requireTempToken` valida el encabezado `x-temp-token` (ver 3.7).
5. El controlador ejecuta la operación. Las operaciones de varios pasos usan `prisma.$transaction`.
6. La acción se registra en `LogActividad` con `registrarActividad`; si ese registro falla, la operación principal no se interrumpe.
7. Un error no controlado llega al manejador central de `app.js` y se devuelve como `{ "error": "mensaje" }`.

3.5 Relación entre módulos

El sistema se organiza alrededor del paciente. Los usuarios y sus roles determinan a qué módulos se accede; cada acción relevante deja constancia en la bitácora de actividad. De la admisión del paciente cuelgan sus ingresos, documentos, tratamientos y recetas. Los tratamientos pendientes se cobran en la factura del hospital, y esta junto con las ventas de farmacia y los gastos alimenta los reportes.

Figura 11

Relación entre los módulos del sistema



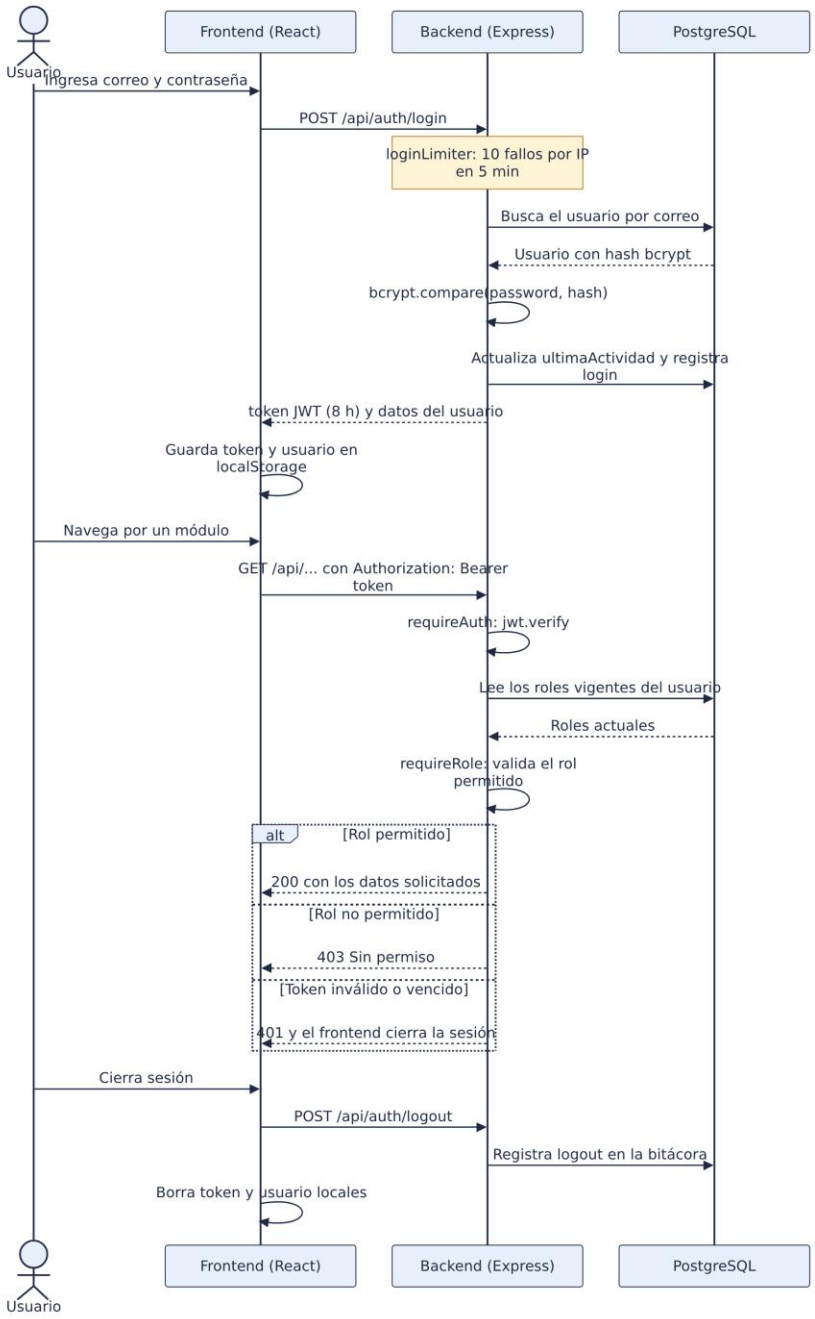
Nota. Elaboración propia.

3.6 Flujo de autenticación

El inicio de sesión valida las credenciales con bcrypt y entrega un JWT de 8 horas. A partir de ahí, cada petición lleva el token; el backend lo verifica, consulta los roles vigentes en la base y decide si el rol puede usar la ruta. Si el token venció o dejó de ser válido, el backend responde 401 y el frontend cierra la sesión de forma automática.

Figura 12

Flujo de autenticación y autorización por rol



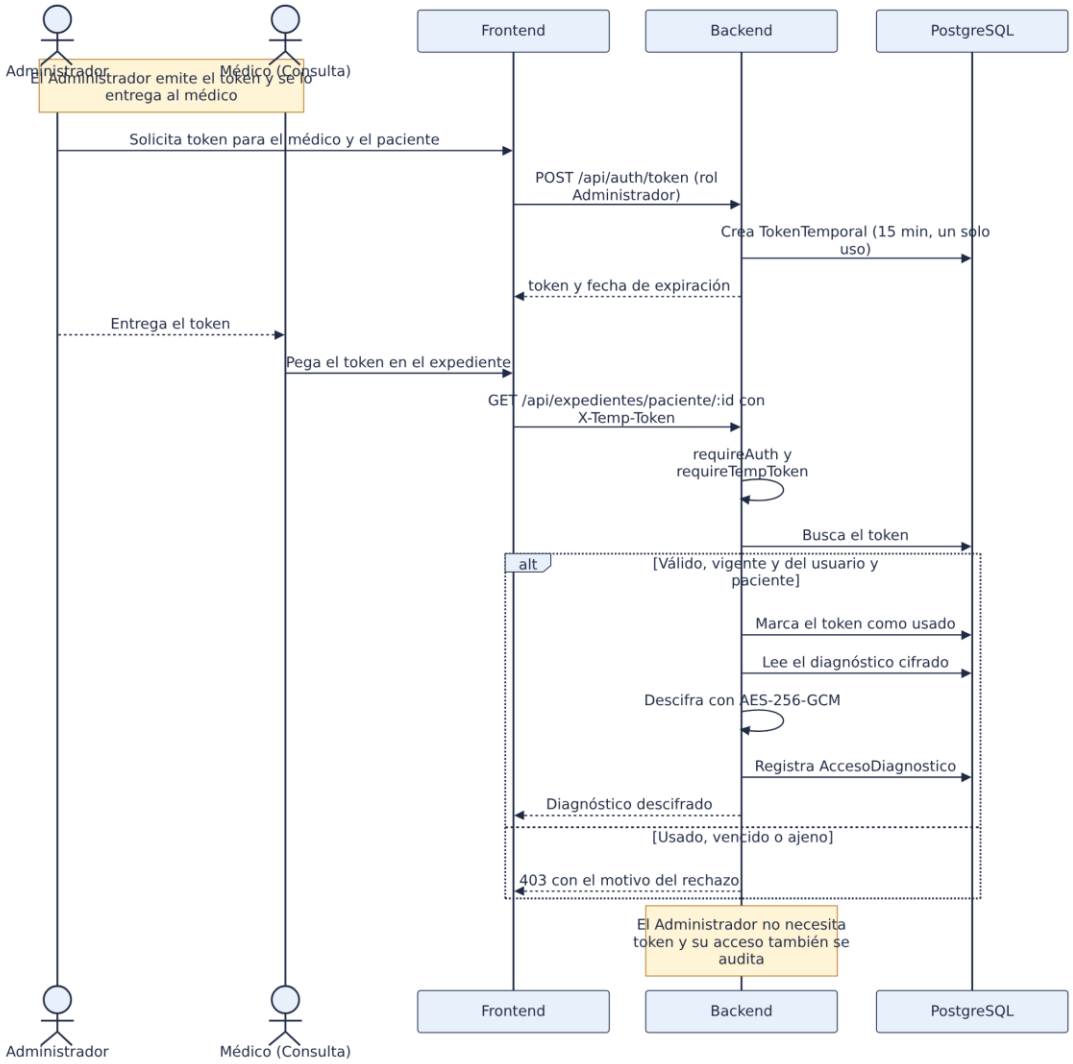
Nota. Elaboración propia.

3.7 Flujo de acceso al diagnóstico con token temporal

El contenido de un diagnóstico es confidencial. El Administrador tiene acceso permanente. Cualquier otro rol necesita un token temporal emitido por el Administrador, de 15 minutos y de un solo uso, que se envía en el encabezado `x-temp-token`. Cada lectura o registro deja un renglón en `AccesoDiagnostico` para auditoría.

Figura 13

Flujo de acceso al diagnóstico con token temporal

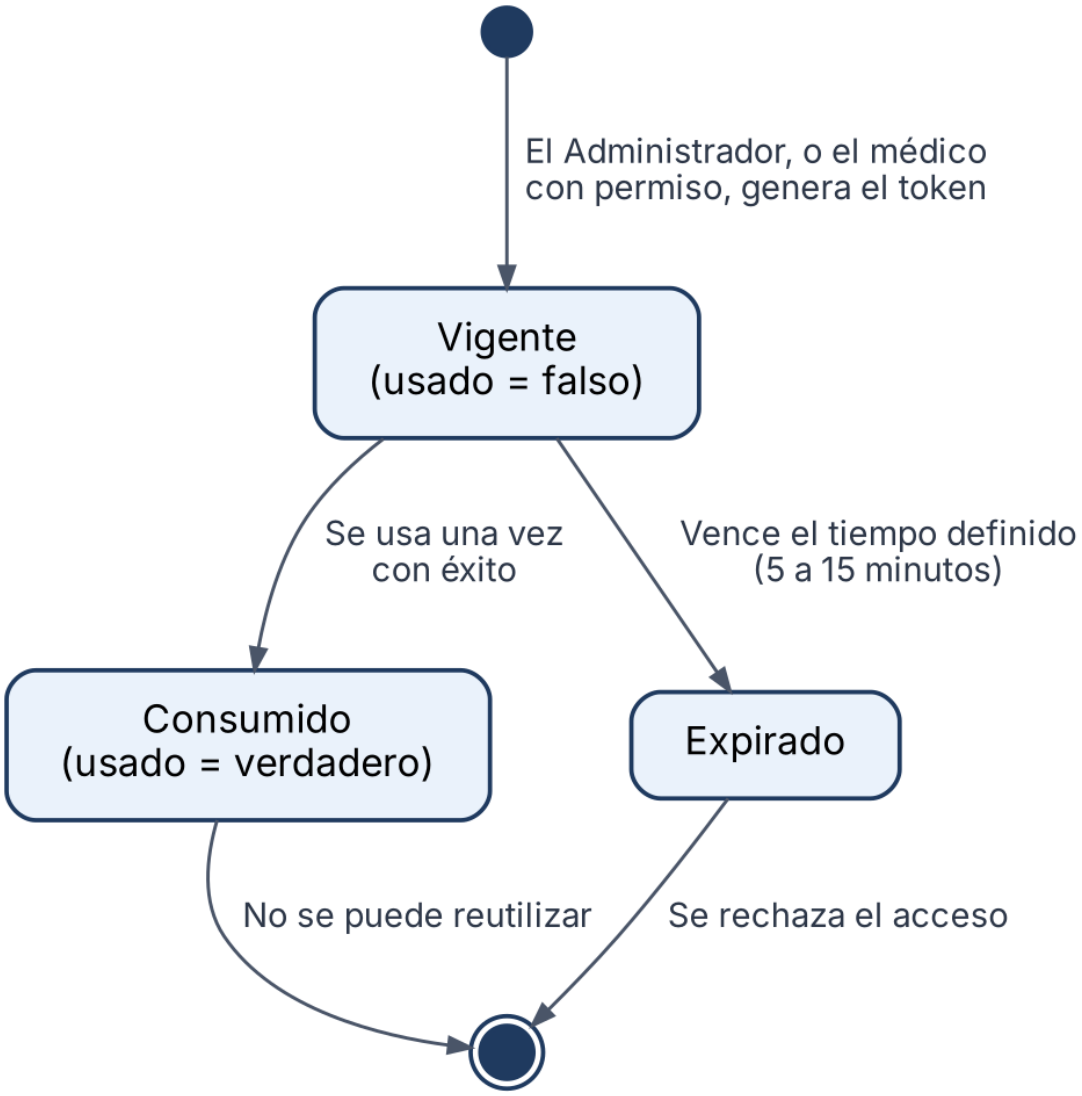


Nota. Elaboración propia.

El ciclo de vida del token se resume en el siguiente diagrama de estados.

Figura 14

Estados del token temporal de acceso



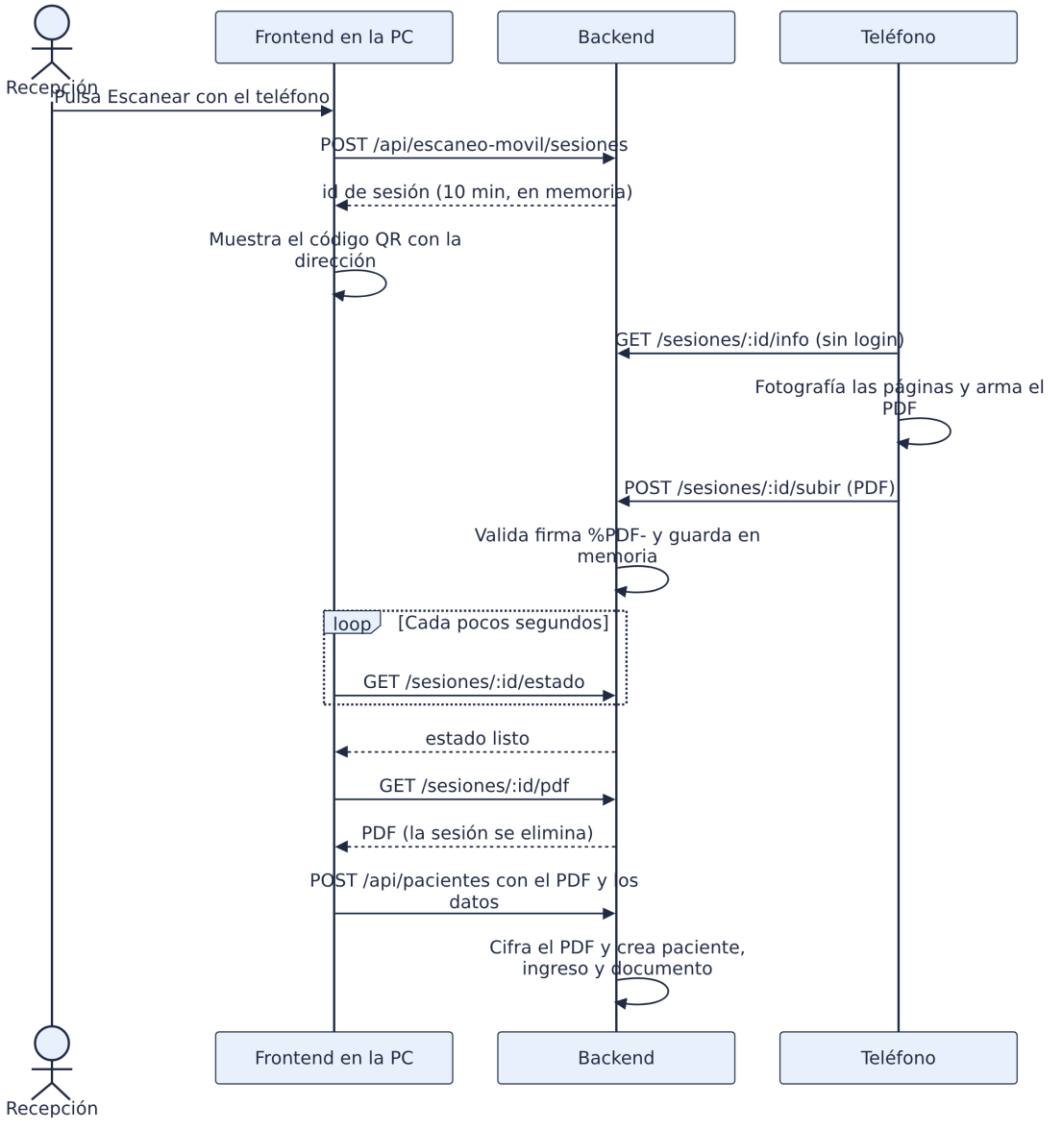
Nota. Elaboración propia.

3.8 Flujo de escaneo por teléfono

Cuando la computadora de recepción no tiene cámara, el sistema usa el teléfono. La computadora crea una sesión efímera y muestra un código QR; el teléfono, sin iniciar sesión, abre la dirección del QR, fotografía las páginas, arma el PDF y lo sube. La computadora recoge el PDF una sola vez y lo entrega al flujo normal de registro de paciente, donde se cifra y se guarda. La sesión vive solo en la memoria del backend y expira a los 10 minutos.

Figura 15

Flujo de escaneo de documentos con el teléfono



Nota. Elaboración propia.

3.9 Control de acceso por rol

Un usuario puede tener varios roles; sus permisos son la unión de los de cada uno. La siguiente matriz muestra qué módulos de la interfaz abre cada rol (`MODULOS_POR_ROL` en `frontend/src/utils/roles.js`).

Rol	Módulos visibles en el menú
Administrador	Todos: Registro, Expediente, Documentos, Tratamiento, Referidos, Financiera, Gastos, Farmacia, Seguridad y Reportes
Recepción	Registro y Expediente
Enfermería	Registro y Expediente
Consulta	Registro y Expediente
Facturación	Registro, Expediente, Financiera, Gastos y Referidos

Rol	Módulos visibles en el menú
Farmacia	Registro y Farmacia

La interfaz oculta lo que el rol no usa, pero **la autorización real se aplica en el backend** con `requireRole` en cada ruta (sección 7). El módulo Bitácora de Visitas no aparece en el menú de ningún rol; la ruta `/bitacora` sigue existiendo y solo el Administrador puede abrirla escribiendo la dirección. El modo restringido de la barra lateral lo alterna únicamente el Administrador.

3.10 Especificaciones técnicas

Versiones de tecnologías

Versiones declaradas en `package.json` (rangos semver).

Paquete	Versión	Uso
express	^4.19.0	Servidor HTTP
express-async-errors	^3.1.1	Propaga errores async al manejador central
@prisma/client, prisma	^5.19.0	ORM y CLI de migraciones
jsonwebtoken	^9.0.0	Firma y verificación de JWT (HS256)
bcrypt	^5.1.0	Hash de contraseñas (cost 10)
multer	^2.2.0	Recepción de multipart/form-data en memoria
zod	^4.6.5	Esquema de la lectura avanzada
@anthropic-ai/sdk	^0.127.0	Cliente de la lectura avanzada (opcional)
dotenv	^16.4.0	Carga de `.env`
cors	^2.8.5	CORS
react, react-dom	^18.3.0	Interfaz
react-router-dom	^6.26.0	Enrutamiento de la SPA
vite	^5.4.0	Compilación y servidor de desarrollo
tailwindcss	^3.4.0	Estilos utilitarios
tesseract.js	^7.0.0	OCR en el navegador
pdfjs-dist	^6.3.289	Lectura de PDF en el navegador
jspdf	^4.2.1	Generación de PDF (fichas, facturas, recetas, escaneos)
recharts	^2.15.4	Gráficas de reportes
qrcode	^1.5.4	Código QR del escaneo móvil
lucide-react	^1.33.0	Iconos
@playwright/test	^1.64.0	Pruebas E2E

Imágenes de contenedor: `node:22-alpine` (backend y compilación del frontend), `nginx:alpine` (frontend) y `postgres:16-alpine` (base de datos).

Puertos

Servicio	Puerto	Configurable en	Alcance
Frontend (Nginx, Docker)	8080 → 80	`FRONTEND_PORT`	Publicado al host
Frontend (Vite, desarrollo)	5173	`vite.config.js`	Equipo del desarrollador
Backend (Express)	4000	`PORT`	Red interna de Compose
PostgreSQL	5432	—	Red interna de Compose

4. Modelo de datos

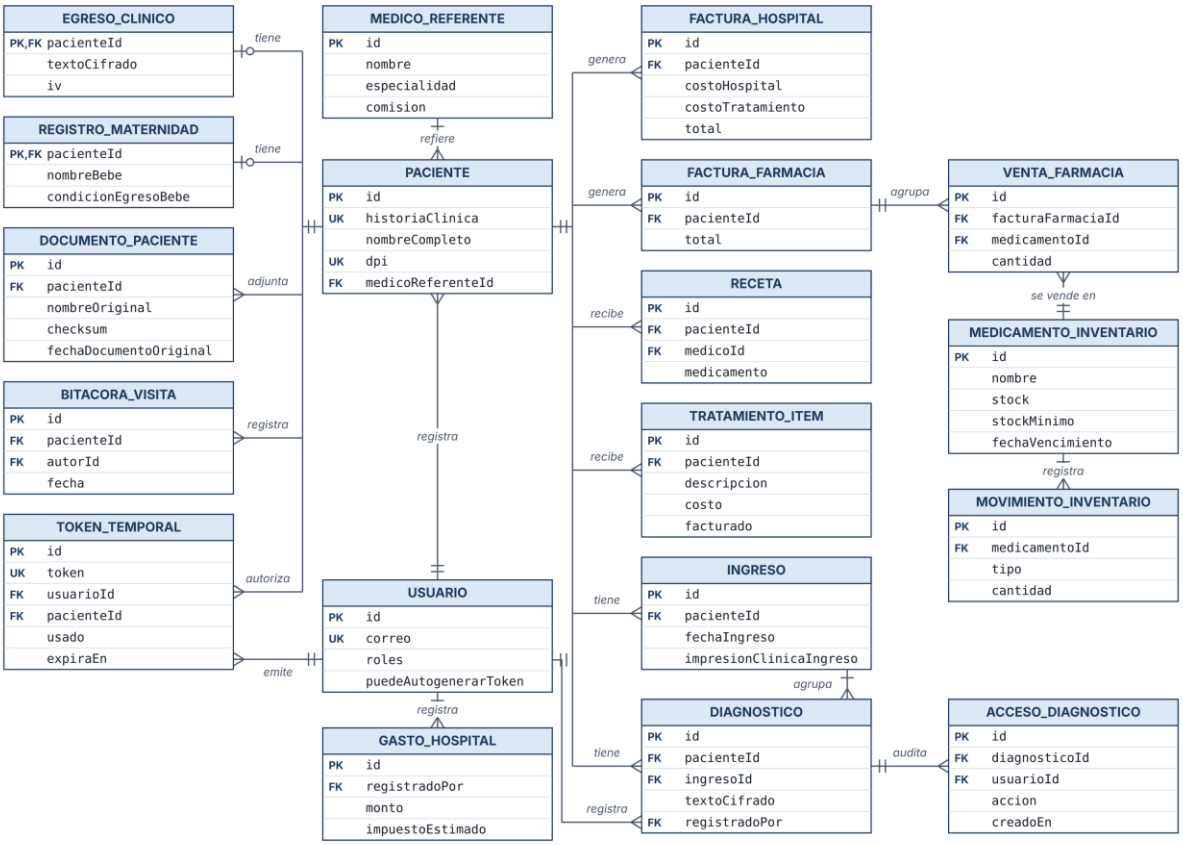
4.1 Motor y convenciones

- PostgreSQL 16. El esquema se gestiona exclusivamente con Prisma Migrate (`backend/prisma/migrations/`).
- Identificadores `Int` autoincrementales, fechas `TIMESTAMP(3)` y montos `DECIMAL(10,2)`.
- Arreglos nativos para los roles (`Usuario.roles TEXT[]`).
- Los campos cifrados guardan `textoCifrado`, `iv` y `authTag` en hexadecimal.

4.2 Diagrama entidad-relación

Figura 16

Diagrama entidad-relación de la base de datos



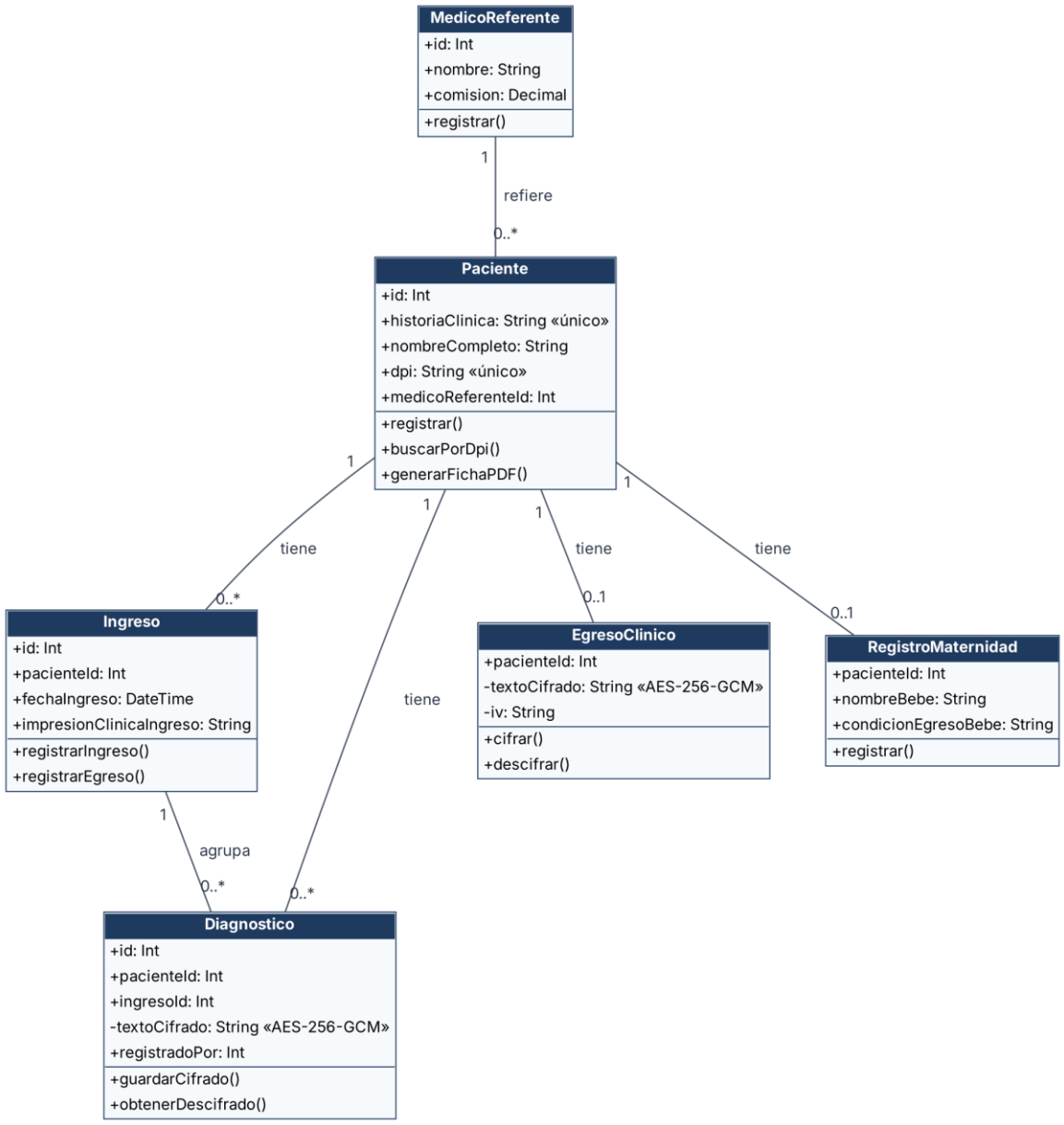
Nota. Elaboración propia.

4.3 Diagramas de clases

Los diagramas de clases agrupan las entidades por área: núcleo clínico y de admisión, seguridad y acceso al diagnóstico, tratamientos y documentos, y finalmente facturación, farmacia y gastos.

Figura 17

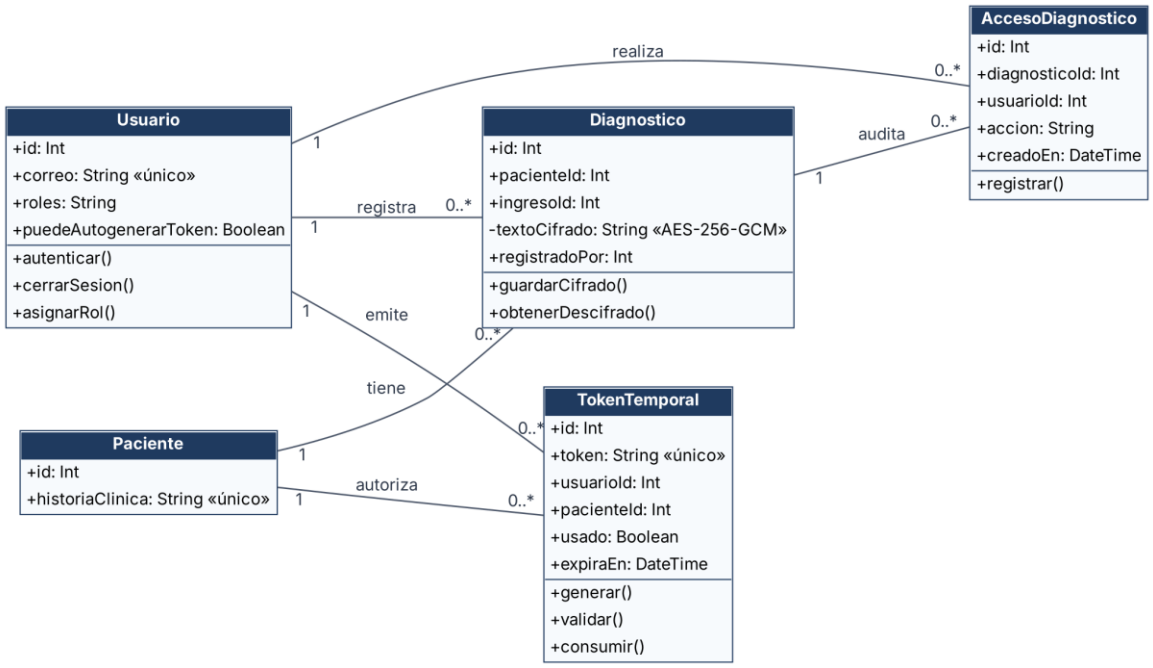
Diagrama de clases, núcleo clínico y de admisión



Nota. Elaboración propia.

Figura 18

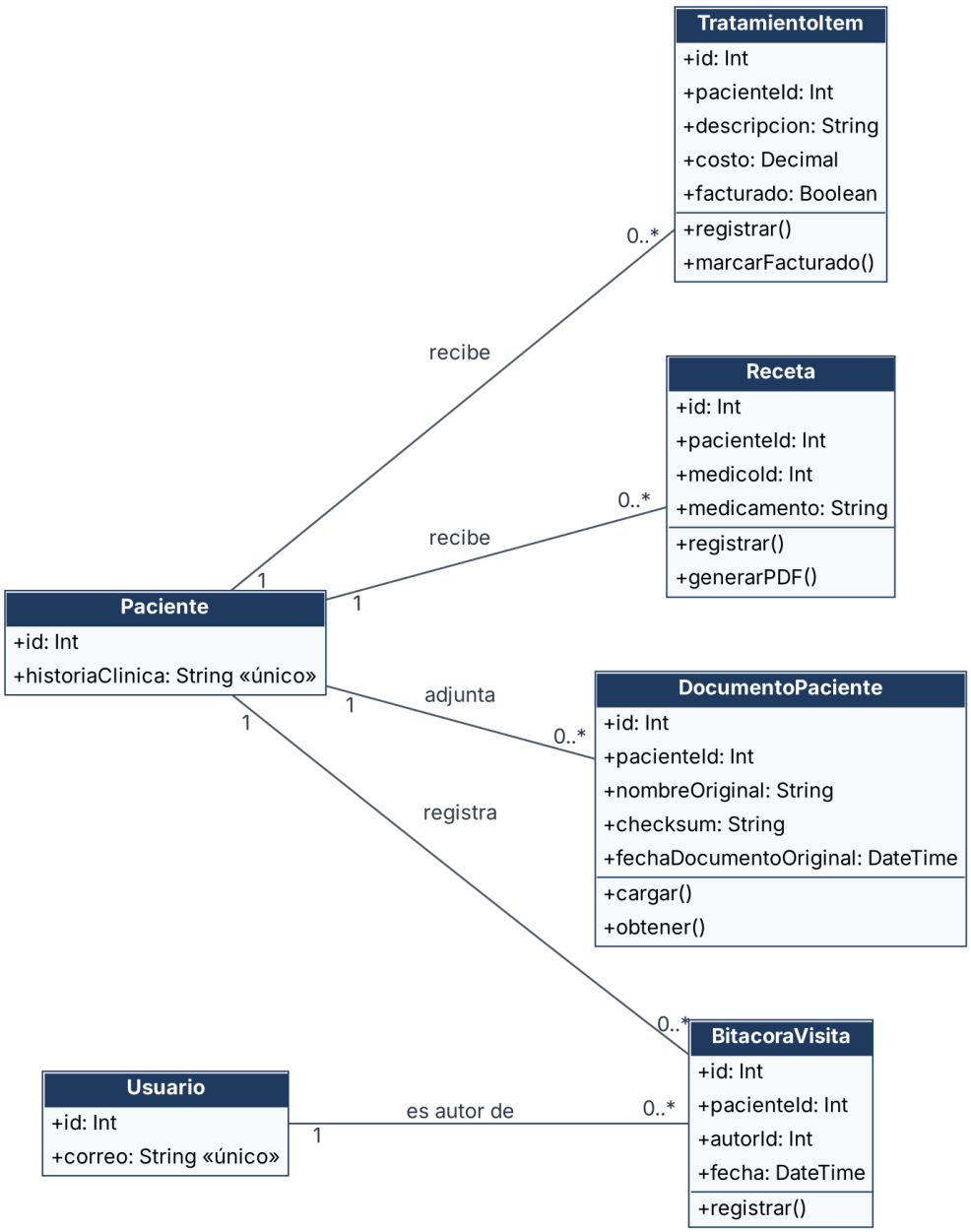
Diagrama de clases, seguridad y acceso al diagnóstico



Nota. Elaboración propia.

Figura 19

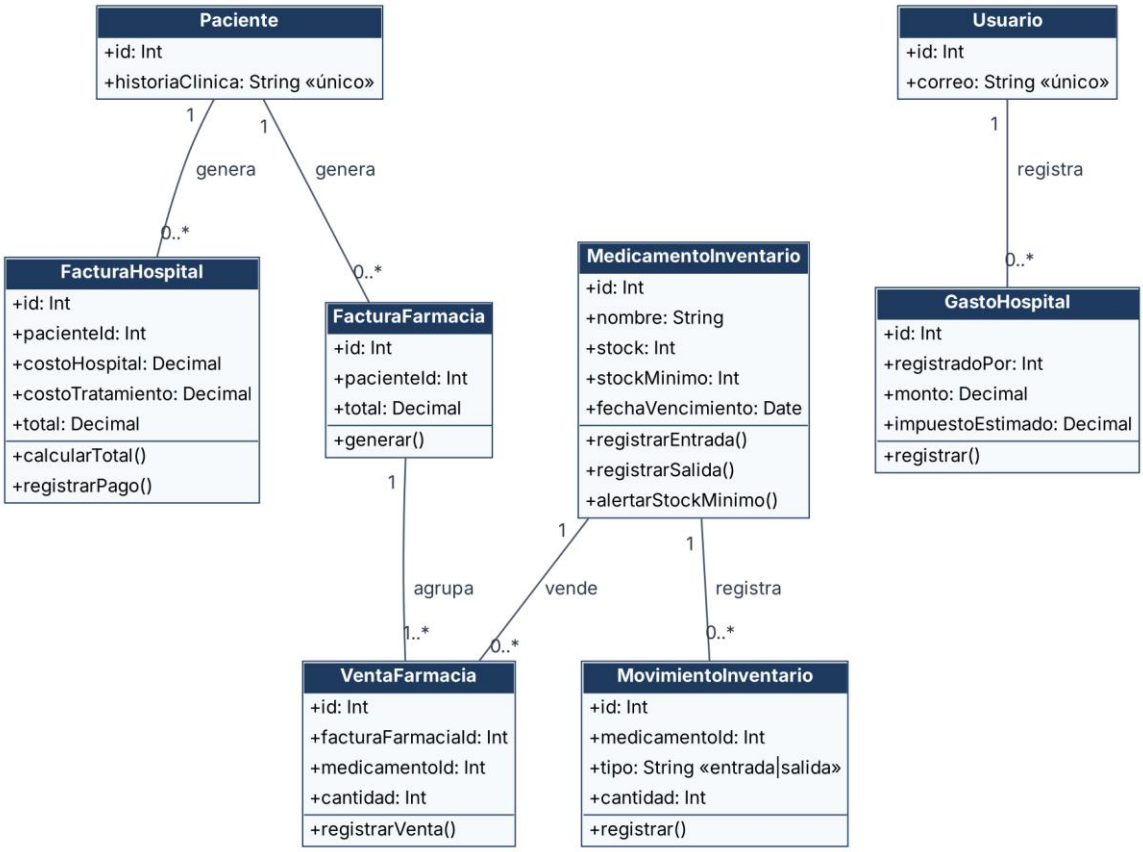
Diagrama de clases, tratamientos, recetas, documentos y visitas



Nota. Elaboración propia.

Figura 20

Diagrama de clases, facturación, farmacia y gastos



Nota. Elaboración propia.

4.4 Entidades

Modelo	Módulo	Propósito	Notas
`Usuario`	Seguridad	Cuenta, `roles[]`, `puedeAutogenerarToken`, colegiado, especialidad, `ultimaActividad`	`correo` único. `passwordHash` bcrypt.
`PasswordResetToken`	Seguridad	Recuperación de contraseña	Guarda el SHA-256 del token; uso único; expira.
`TokenTemporal`	Seguridad	Token de acceso al diagnóstico	`token` único; `usado`; `expiraEn`; `pacienteId` opcional.
`LogActividad`	Auditoría general	Quién hizo qué y cuándo	Índices por usuario, fecha y acción.
`Paciente`	Admisión	Identidad y datos administrativos	`historiaClinica` y `dpi` únicos. Conserva columnas legacy de egreso en texto plano que la API ya no escribe (D13).

Modelo	Módulo	Propósito	Notas
`Ingreso`	Admisión y Expediente	Una admisión del paciente	Uno a muchos con `Paciente`. Incluye `impresionClinicaIngreso` en texto plano (R1).
`EgresoClinico`	Admisión	Datos clínicos de egreso cifrados	Uno a uno con `Paciente`.
`RegistroMaternidad`	Admisión	Datos del nacimiento y del padre	Uno a uno con `Paciente`.
`DocumentoPaciente`	Documentos	Metadatos de documentos y adjuntos	Borrado lógico (`eliminadoEn`). Binario cifrado en disco.
`Diagnostico`	Expediente	Versión cifrada del diagnóstico	Cada guardado crea una fila; no hay UPDATE. `ingresoId` opcional.
`DiagnosticoArchivo`	Expediente	Archivos de "Estudios y resultados"	Binario cifrado en `uploads/diagnosticos/`.
`AccesoDiagnostico`	Auditoría clínica	Cada lectura o registro de un diagnóstico	Usuario, acción, si fue por token y cuál.
`TratamientoItem`	Tratamiento	Costos intrahospitalarios y de farmacia usados	`facturado` evita el doble cobro.
`Receta`	Tratamiento	Receta médica	`medicoId` apunta a un `Usuario` con rol Consulta.
`MedicoReferente`	Referidos	Médico o institución referente	`comisionQ` opcional.
`FacturaHospital`	Financiera	Factura del hospital	`costoTratamiento` y `total` redondeados a 2 decimales.
`CategoriaFiscal`	Financiera	Catálogo editable de categorías	`nombre` único. Tasa informativa.
`GastoHospital`	Financiera	Gastos del hospital	`montoImpuestoEstimado` congelado al registrar.
`FacturaFarmacia`	Farmacia	Cabecera de una venta directa	`pacienteId` opcional.
`VentaFarmacia`	Farmacia	Líneas de la venta	Una por medicamento.
`MedicamentoInventario`	Farmacia	Existencias	`stockMinimo`, `fechaVencimiento`. Sin restricción única en `nombre`.
`MovimientoInventario`	Farmacia	Kardex	`tipo` entrada o salida; `motivo`.
`CatalogoCie10`	Catálogo	Códigos CIE-10 de uso frecuente	`codigo` único.
`BitacoraVisita`	Bitácora de Visitas	Visitas registradas	Sin acceso desde el menú; la API sigue activa.

4.5 Relaciones principales

- `Paciente` 1—N `Ingreso`; `Ingreso` 1—N `Diagnostico` (opcional, `ON DELETE SET NULL`).
- `Paciente` 1—1 `EgresoClinico` y 1—1 `RegistroMaternidad`; 1—N `DocumentoPaciente`, `TratamientoItem`, `Receta`, `FacturaHospital`, `BitacoraVisita`, `TokenTemporal`, `AccesoDiagnostico` y `FacturaFarmacia`.
- `Usuario` 1—N `Diagnostico` (`registradoPor`), `Receta` (`medicoId`), `TokenTemporal` (como titular y como emisor), `DocumentoPaciente`, `GastoHospital` y `LogActividad`.
- `CategoriaFiscal` 1—N `GastoHospital`.
- `MedicamentoInventario` 1—N `MovimientoInventario` y `VentaFarmacia`.

- `FacturaFarmacia` 1—N `VentaFarmacia`.

Las claves foráneas a `Paciente` usan `ON DELETE RESTRICT`. El endpoint `DELETE /api/pacientes/:id` evita ese bloqueo borrando primero, dentro de una transacción, todos los registros dependientes (ver 5.7 y el riesgo R17).

4.6 Restricciones de integridad relevantes

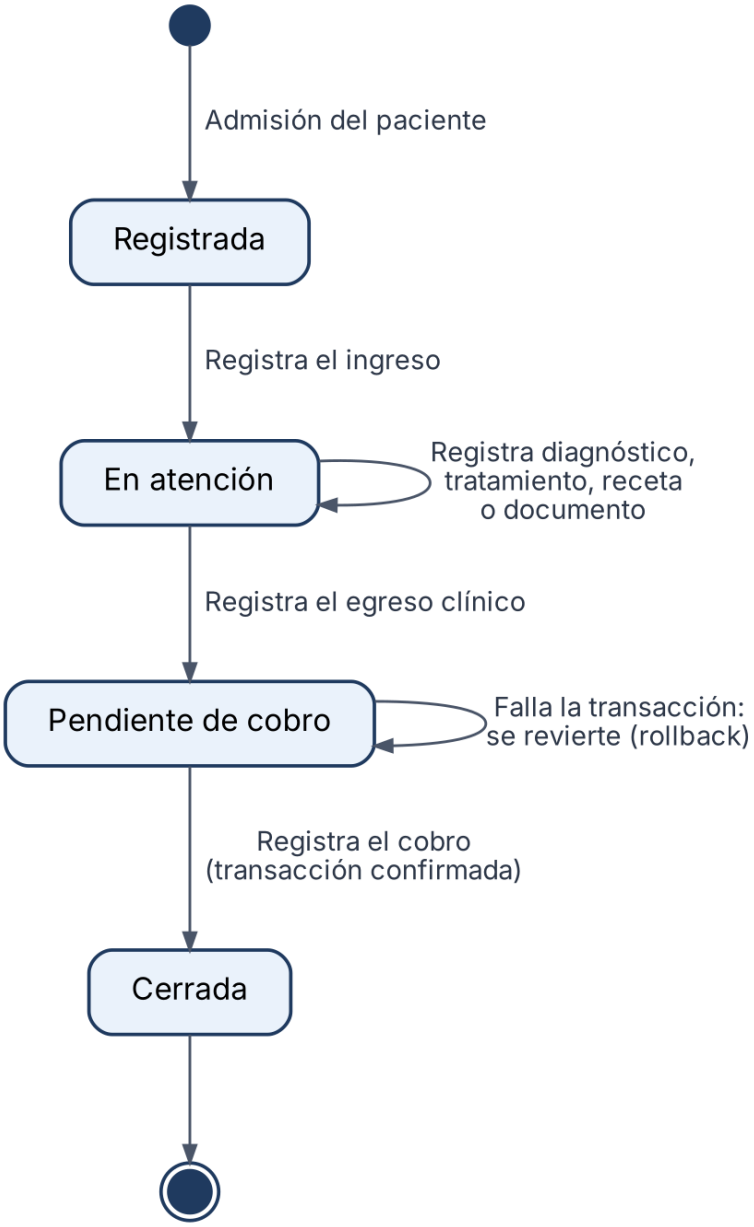
- `Paciente.historiaClinica` y `Paciente.dpi` son únicos. Los controladores responden 409 antes de llegar a la restricción.
- `Usuario.correo` es único.
- `CatalogoCie10.codigo`, `CategoriaFiscal.nombre`, `EgresoClinico.pacienteId` y `RegistroMaternidad.pacienteId` son únicos.
- `MedicamentoInventario` solo tiene un índice (no único) sobre `nombre` (D1).

4.7 Ciclos de vida

Dos entidades tienen estados que conviene tener presentes: la admisión del paciente y el medicamento en inventario.

Figura 21

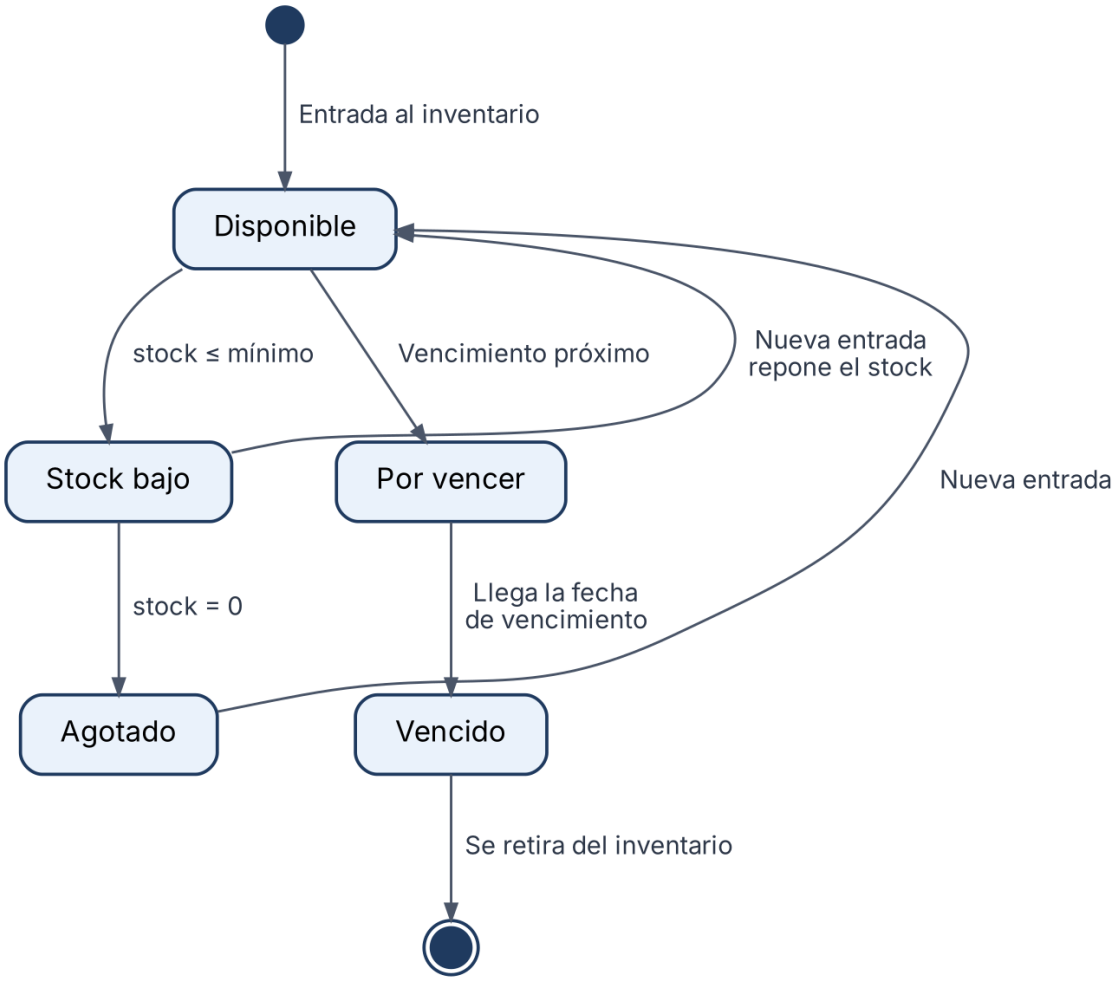
Estados de la admisión del paciente



Nota. Elaboración propia.

Figura 22

Estados de un medicamento en el inventario



Nota. Elaboración propia.

4.8 Borrado lógico y retención

- `DocumentoPaciente.eliminadoEn`: el documento deja de listarse y descargarse, pero el binario se conserva según la política de retención. La depuración física no está automatizada.
- Los diagnósticos no se modifican: cada guardado es una versión nueva. La única operación que los borra es la eliminación completa de un paciente (R17).

4.9 Historial de migraciones

#	Migración	Contenido resumido
1	`20260819022631_init`	Esquema inicial.
2	`20260820195841_agrega_auditoria_diagnostico`	Auditoría de acceso al diagnóstico.

#	Migración	Contenido resumido
3	20260820201918_agrega_accion_a_auditoria	Campo accion.
4	20260820210313_factura_farmacia_multilinea	Facturas de farmacia con líneas.
5	20260820215115_agrega_telefono_emergencia	Teléfono de emergencia.
6	20260820215809_bitacora_general_y_estado_en_linea	LogActividad y ultimaActividad.
7	20260820221553_paso1_agrega_roles_array	roles TEXT[].
8	20260820222525_paso2_elimina_columna_rol	Retira la columna rol única.
9	20260821052433_paso3_agrega_contexto_atencion_diagnostico	Tipo de atención, servicio, sala y cama.
10	20260825193155_paso4_agrega_catalogo_cie10	Catálogo CIE-10.
11	20260825193915_paso5_agrega_personal_medico_procedimiento	Equipo quirúrgico, colegiado, especialidad.
12	20260825195942_paso6_agrega_archivo_diagnostico	DiagnosticoArchivo.
13	20260825215914_paso7_agrega_tipo_sangre	Tipo de sangre.
14	20260908120000_paso8_egreso_cifrado_anexos_reset_colegiado	EgresoClinico cifrado; reinicio de colegiado.
15	20260913150000_paso9_documentos_paciente	DocumentoPaciente.
16	20260915120000_paso10_fecha_documento_original	fechaDocumentoOriginal.
17	20260921120000_paso11_maternidad_padres	Datos del bebé y del padre.
18	20260923120000_paso12_fusion_anexos_documentos	Fusión de Anexos en Documentos.
19	20260923150000_paso13_gastos_hospital	GastoHospital y CategoriaFiscal.
20	20260924120000_paso14_impuesto_estimado_gasto	montoImpuestoEstimado.
21	20261008120000_datos_formulario_admision	Cónyuge, emergencia, referido por, otras hospitalizaciones; autopsia del bebé.
22	20261013120000_paso15_ingresos_paciente	Tabla Ingreso, Diagnostico.ingresoId y relleno de datos existentes.
23	20261022120000_paso16_documento_por_ingreso	Los documentos se asocian al ingreso al que pertenecen.
24	20261025061757_comision_medico_referente_opcional	comisionQ del médico referente pasa a ser opcional.

La migración 22 crea `Ingreso` con clave foránea a `Paciente` e índices por `pacienteId` y `fechaIngreso`, agrega `Diagnostico.ingresoId` (nullable, `ON DELETE SET NULL`), inserta un `Ingreso` por cada paciente con `fechaIngreso` no nula y asigna a cada diagnóstico existente el ingreso de su paciente.

5. Documentación técnica del backend

Esta sección documenta los módulos de la capa de servidor del sistema, desarrollada en Node.js con Express y Prisma. Cada módulo se describe con su archivo, sus dependencias y el detalle de sus funciones: propósito, parámetros, lógica interna y resultado. Los controladores usan `express-async-errors`, así que un error lanzado dentro de una función `async` llega al manejador central de `app.js` sin necesidad de `try/catch` en cada ruta.

5.1 Infraestructura principal

Archivos: `src/server.js`, `src/app.js`, `src/config/env.js`, `src/config/prisma.js`.

Conjunto de archivos que inicializan y configuran la aplicación: punto de entrada, validación de variables de entorno, montaje de las rutas por módulo, comprobación de salud y manejo central de errores.

Archivo	Responsabilidad
`server.js`	Arranque: escucha en `PORT` (4000 por defecto).
`app.js`	Crea la aplicación Express, monta los 13 routers bajo `/api`, define `/api/health` y el manejador de errores.
`config/env.js`	Valida las variables críticas al arrancar y exporta los valores de configuración.
`config/prisma.js`	Exporta el cliente único de Prisma (`PrismaClient`) para toda la aplicación.

Arranque del servidor [server.js]

Campo	Detalle
Propósito	Poner la aplicación a escuchar peticiones HTTP.
Parámetros	Ninguno. Lee `process.env.PORT`.
Lógica	Importa `app.js` (que a su vez ejecuta la validación de `env.js`) y llama a `app.listen(PORT)`. Escribe en consola `Servidor backend Hospital Verapaz escuchando en puerto <PORT>`.
Resultado	Servidor HTTP en el puerto 4000 o el indicado por `PORT`.

Validación de entorno [config/env.js]

Campo	Detalle
Propósito	Fallar rápido y con un mensaje claro si falta una variable crítica, en lugar de fallar después en medio de una petición.
Parámetros	Variables de entorno: `DATABASE_URL`, `JWT_SECRET`, `ENCRYPTION_KEY` y las opcionales de la sección 2.3.
Lógica	Acumula errores: `DATABASE_URL` ausente; `JWT_SECRET` ausente o menor de 16 caracteres; `ENCRYPTION_KEY` ausente o que no cumpla `/^[0-9a-fA-F]{64}$/`. Si hay errores, los imprime y termina con `process.exit(1)`. Si no, exporta `ENCRYPTION_KEY`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `TEMP_TOKEN_EXPIRES_MIN`, `PASSWORD_RESET_EXPIRES_MIN`, `PASSWORD_RESET_EXPOSE_TOKEN` y `UPLOAD_MAX_MB_ESTUDIOS`.
Resultado	El proceso arranca con configuración válida o se detiene con código 1.

Comprobación de salud [app.js]

Campo	Detalle
Propósito	Permitir que Docker, la prueba de humo del pipeline y el monitoreo detecten tanto la caída de la API como la pérdida de conexión con PostgreSQL.
Parámetros	Ninguno. `GET /api/health`, sin autenticación.
Lógica	Ejecuta `SELECT 1` con `prisma.$queryRaw`. Si responde, devuelve 200; si lanza error, devuelve 503.
Resultado	`200 {"status":"ok","db":"ok"}` o `503 {"status":"degraded","db":"error"}`.

Manejador de errores [app.js]

Campo	Detalle
Propósito	Unificar la forma de las respuestas de error en toda la API.
Parámetros	`(err, req, res, next)`.

Campo	Detalle
Lógica	Si `err.code` es `P2002` (violación de unicidad de Prisma) responde 409. Si es `LIMIT_FILE_SIZE` (multer) responde 422. En cualquier otro caso registra el error en consola y responde con `err.status` o 500 y `err.message`.
Resultado	Cuerpo `{ "error": "mensaje" }`. Ver el riesgo R11 sobre el mensaje en los 500.

5.2 Middlewares

Archivos: `src/middlewares/auth.middleware.js`, `src/middlewares/rate-limit.middleware.js`.

Dependencias: `jsonwebtoken` (verificación del JWT), `config/prisma.js` y `utils/roles.util.js`.

requireAuth(req, res, next)

Campo	Detalle
Propósito	Exigir una sesión iniciada en toda ruta protegida (RF-29).
Parámetros	Encabezado `Authorization: Bearer <JWT>`.
Lógica	Sin encabezado responde 401. Verifica la firma con `jwt.verify` y `JWT_SECRET`. Con el `id` del payload vuelve a leer en la base el nombre y los **roles vigentes** del usuario, de modo que un cambio de rol o la baja de un rol se aplica de inmediato aunque el token sea anterior. Si el usuario ya no existe responde 401. Deja `req.user = { ...payload, id, nombre, roles }` y actualiza `ultimaActividad` sin esperar la respuesta y sin interrumpir si falla.
Resultado	Continúa con `next()` o responde 401 (`No autenticado`, `Usuario no encontrado`, `Token invalido o expirado`).

requireRole(...rolesPermitidos)

Campo	Detalle
Propósito	Restringir una ruta a ciertos roles (RF-30 y RF-32).
Parámetros	Uno o más valores de `ROLES` (`Administrador`, `Recepcion`, `Consulta`, `Enfermeria`, `Facturacion`, `Farmacia`).
Lógica	Devuelve un middleware que revisa si al menos uno de los roles de `req.user.roles` está en la lista. Un usuario con varios roles obtiene la unión de sus permisos.
Resultado	`next()` o `403 {"error":"No tiene permiso para esta accion"}`.

requireTempToken(req, res, next)

Campo	Detalle
Propósito	Exigir un token de acceso temporal para ver o registrar el diagnóstico confidencial (RF-11, RF-33, RF-34, RNF-12).
Parámetros	Encabezado `x-temp-token`; `req.params.id` (paciente).
Lógica	Si el usuario es Administrador, continúa sin token. Si no, busca el token en `TokenTemporal` y verifica en este orden: que exista, que no esté usado, que no haya expirado, que pertenezca al usuario autenticado y que, si el token trae `pacienteId`, coincida con el `:id` de la ruta. Si todo es válido lo marca `usado = true` antes de continuar y deja el registro en `req.tempToken`.
Resultado	`next()` o `403` con el motivo (`Requiere token de acceso temporal`, `Token invalido`, `El token ya fue utilizado`, `El token ha expirado`, `El token no pertenece a este usuario`, `El token no autoriza a este paciente`).

createRateLimiter(opciones)

Campo	Detalle
Propósito	Proteger el inicio de sesión y la recuperación de contraseña contra la fuerza bruta.
Parámetros	`{ windowMs = 5 min, max = 10, mensaje, countFailuresOnly = false }`.
Lógica	Crea un mapa propio por limitador, con clave la IP (`req.ip`). Cada ventana lleva un contador. Con `countFailuresOnly`, solo cuentan las respuestas con código mayor o igual a 400 y las peticiones en curso se tratan como pendientes para no abrir una brecha por concurrencia. Siempre fija el encabezado `Retry-After`. Limpia ventanas vencidas cuando el mapa supera 10 000 entradas.
Resultado	Un middleware que responde `429 {"error", "reintentaEnSeg"}` al excederse el máximo. El estado vive en memoria (ver D10 y R14).

_resetVentanas()

Campo	Detalle
Propósito	Reiniciar todos los limitadores. Solo para pruebas.
Parámetros	Ninguno.
Lógica	Recorre los mapas de todos los limitadores creados y los vacía.
Resultado	Sin valor de retorno.

5.3 Utilidades

Archivos: `src/utils/crypto.util.js`, `archivos.util.js`, `periodos.util.js`, `paginacion.util.js`, `roles.util.js`.

Cifrado [crypto.util.js]

Dependencia: módulo `crypto` de Node. Algoritmo AES-256-GCM con IV aleatorio de 12 bytes y etiqueta de autenticación de 16 bytes.

Función	Propósito	Parámetros	Lógica	Resultado
`encrypt(text, keyHex)`	Cifrar texto (diagnósticos y egreso clínico)	Texto UTF-8 y llave de 64 hex	Genera un IV, cifra y obtiene la etiqueta	`{ encrypted, iv, authTag }` en hexadecimal
`decrypt({encrypted, iv, authTag}, keyHex)`	Descifrar texto	Objeto anterior y llave	Fija la etiqueta y descifra; lanza error si no coincide	Texto UTF-8
`encryptBuffer(buffer, keyHex)`	Cifrar un archivo binario	Buffer y llave	Cifra y arma el sobre `[IV 12][authTag 16][ciphertext]`	Buffer con el sobre
`decryptBuffer(sobre, keyHex)`	Descifrar un sobre	Buffer y llave	Separa IV, etiqueta y texto cifrado; verifica la etiqueta	Buffer original
`leerArchivoCifrado(ruta, keyHex)`	Leer un archivo del volumen	Ruta y llave	Si el nombre empieza con `enc_` descifra el sobre; si es un archivo histórico en claro lo devuelve tal cual	Buffer del contenido

Archivos [archivos.util.js]

Función	Propósito	Parámetros	Lógica	Resultado
`tiposPermitidos()`	Lista de tipos MIME aceptados	Ninguno	Lee `UPLOAD_TIPOS_PERMITIDOS` o usa PDF, PNG y JPEG	Arreglo de MIME
`validarArchivo(file)`	Validar un archivo subido	Objeto de multer (`originalname`, `mimetype`, `buffer`)	Exige tres coincidencias: MIME permitido, extensión acorde al MIME y firma de los primeros bytes (`%PDF-`, PNG, `FF D8 FF`). Rechaza nombres con `/`, `\` o `..`	`null` si es válido o el texto del error
`nombreInternoAleatorio()`	Nombre seguro para disco	Ninguno	`enc_` más 24 bytes aleatorios en hexadecimal y `.bin`	Cadena
`guardarArchivoCifrado(buffer, directorio, keyHex)`	Persistir un archivo cifrado	Buffer, carpeta, llave	Crea la carpeta, cifra con `encryptBuffer` y escribe con `fs/promises` (asíncrono, para no bloquear el servidor)	Nombre interno generado
`validarPdf(file)`	Verificar que un buffer sea PDF	Objeto de multer	Compara los primeros 5 bytes con `%PDF-` y el MIME	`null` o texto del error
`tamanoLegible(bytes)`	Mostrar un tamaño	Número de bytes	Convierte a B, KB o MB	Cadena

Periodos y paginación

Función	Propósito	Parámetros	Lógica	Resultado
`rangoFecha(desde, hasta)`	Filtro de fechas para los reportes	Textos `AAAA-MM-DD`	Convierte a instantes en la zona del hospital (UTC-6); "hasta" incluye todo ese día. Ignora valores inválidos	Objeto `{gte, lte}` de Prisma o `undefined`
`diaEnHospital(instante)`	Día calendario de un instante	Fecha	Formatea con `Intl.DateTimeFormat` en `America/Guatemala`	`AAAA-MM-DD`
`clavePeriodo(instante, granularidad)`	Agrupar por periodo	Fecha y `dia`, `semana` o `mes`	Devuelve el día, el lunes de la semana o el mes	Clave del periodo
`leerPaginacion(req, opciones)`	Paginar listados	`req.query.page` y `pageSize`	Limita `pageSize` a 100 y calcula `skip` y `take`	`{ page, pageSize, skip, take }`

Roles [roles.util.js]

Exporta `ROLES` (objeto congelado con los seis roles) y `ROLES_VALIDOS` (arreglo de sus valores), que usan las rutas, los middlewares y la validación de usuarios.

5.4 Servicios

Archivos: `src/services/actividad.service.js`, `notificacion.service.js`, `facturacion.service.js`, `lecturaVision.service.js`.

registrarActividad(usuarioId, accion, detalle) [actividad.service.js]

Campo	Detalle
Propósito	Dejar constancia en la bitácora general de quién hizo qué y cuándo.
Parámetros	Id del usuario (puede ser nulo), texto de la acción y detalle opcional.
Lógica	Crea una fila en `LogActividad`. Si la escritura falla, escribe el error en consola y **no lo propaga**, para no interrumpir la acción principal.
Resultado	Promesa sin valor.

Notificación [notificacion.service.js]

Función	Propósito	Parámetros	Lógica	Resultado
`construirEnlaceReset(token)`	Armar el enlace de recuperación	Token en claro	Une `APP_URL` con `/recuperar/cambiar?token=`	URL
`enviarEnlaceRecuperacion({correo, nombre, enlace})`	Entregar el enlace al usuario	Datos del destinatario	Con `NOTIFICACION_PROVEEDOR=consola` escribe el enlace en el log; cualquier otro valor lanza un error porque no hay otro proveedor implementado	`{ entregado: false, modo: "consola" }`

generarFacturaHospital({ pacienteId, costoHospital, formaPago }) [facturacion.service.js]

Campo	Detalle
Propósito	Generar la factura del hospital sumando el costeo pendiente del paciente (RF-17, RF-18, RF-19, RNF-13).
Parámetros	Id del paciente, costo base del hospital y forma de pago.
Lógica	Dentro de una transacción: busca al paciente (404 si no existe); lee los `TratamientoItem` con `facturado = false`; suma sus costos; calcula `total = costoHospital + costoTratamiento` redondeado a 2 decimales; crea la `FacturaHospital`; marca los tratamientos como `facturado = true` para evitar el doble cobro.
Resultado	La factura creada. Si algún paso falla, Prisma revierte todo.

registrarVentaFarmacia({ pacienteId, items, registradoPor }) [facturacion.service.js]

Campo	Detalle
Propósito	Registrar una venta directa de farmacia con uno o varios medicamentos (RF-20, RF-24, RF-27).
Parámetros	Paciente opcional, arreglo `{ medicamentoId, cantidad }` y usuario que registra.
Lógica	En una transacción valida cada línea (404 si el medicamento no existe, 409 si no hay stock), calcula subtotales con redondeo a 2 decimales, crea una sola `FacturaFarmacia`, y por cada línea crea la `VentaFarmacia`, un `MovimientoInventario` de salida con motivo `venta directa` y descuenta el stock.
Resultado	La factura con sus líneas, el paciente y el usuario que registró. Una línea sin stock revierte toda la venta.

reporteConsolidado({ desde, hasta }) [facturacion.service.js]

Campo	Detalle
Propósito	Calcular el reporte financiero consolidado de hospital y farmacia (RF-21).
Parámetros	Rango de fechas opcional.
Lógica	Consulta en paralelo las facturas del hospital, las de farmacia y los gastos del periodo (los gastos se filtran por su `fecha` real y no por la de captura). Suma ingresos, gastos e impuestos estimados, y agrupa los gastos por categoría fiscal ("Sin clasificar" si no tienen).
Resultado	Objeto con `ingresosHospital`, `ingresosFarmacia`, `totalConsolidado`, `totalGastos`, `totalImpuestosEstimados`, `ingresoNeto`, `gastosPorCategoria` y las cantidades de facturas y gastos.

Lectura avanzada [lecturaVision.service.js]

Dependencias: `@anthropic-ai/sdk` y `zod`. Servicio opcional, apagado por defecto. Al activarlo, la imagen de la primera página del expediente se envía a la API de Anthropic y solo vive en memoria durante la petición.

Función	Propósito	Parámetros	Lógica	Resultado
`lecturaAvanzadaDisponible()`	Saber si la función está activa	Ninguno	Comprueba que exista `ANTHROPIC_API_KEY`	Booleano
`modeloConfigurado()`	Modelo a usar	Ninguno	`ANTHROPIC_MODEL` o `claude-opus-5`	Texto
`normalizarCampos(crudo)`	Limpiar lo que devuelve el modelo	Objeto con los 7 campos	Recorta textos, exige DPI de 13 dígitos y teléfono de 8, valida fechas `AAAA-MM-DD` y descarta el teléfono del membrete del hospital	Objeto normalizado con `null` donde no hay dato confiable
`extraerDatosConVision(buffer, mediaType, { cliente })`	Leer la ficha manuscrita	Imagen y tipo; `cliente` inyectable para pruebas	Valida el tipo (422), exige la clave (501), llama al modelo con un esquema Zod de 7 campos y traduce los errores de la API a `LecturaVisionError` (429, 502)	Campos normalizados

5.5 Autenticación y tokens

Archivo: `src/controllers/auth.controller.js`. **Rutas:** `src/routes/auth.routes.js` (`/api/auth`).

Dependencias: `bcrypt` (hash de contraseñas), `jsonwebtoken` (sesión), `crypto` (tokens aleatorios), `actividad.service.js` y `notificacion.service.js`.

Método	Ruta	Acceso
POST	`/auth/login`	público, `loginLimiter` (10 fallos por 5 min)
POST	`/auth/forgot-password`	público, 5 peticiones por 5 min
POST	`/auth/reset-password`	público, 5 peticiones por 5 min

Método	Ruta	Acceso
POST	`/auth/logout`	autenticado
GET	`/auth/me`	autenticado
POST	`/auth/token`	Administrador
POST	`/auth/token/auto`	Administrador

login(req, res)

Campo	Detalle
Propósito	Iniciar sesión con correo y contraseña (RF-29).
Parámetros	Cuerpo `{ correo, password }`.
Lógica	Exige ambos campos (400). Busca el usuario por correo y compara la contraseña con `bcrypt.compare`; ante un fallo responde 401 `Credenciales invalidas` sin indicar cuál dato falló. Actualiza `ultimaActividad`, registra `login` en la bitácora y firma un JWT con `firmarSesion`.
Resultado	`{ token, usuario: { id, nombre, correo, roles, puedeAutogenerarToken } }`.

firmarSesion(usuario) (función interna)

Campo	Detalle
Propósito	Emitir el JWT de la sesión.
Parámetros	Usuario con `id`, `nombre` y `roles`.
Lógica	`jwt.sign` con el payload `{ id, nombre, roles }`, `JWT_SECRET` y la expiración `JWT_EXPIRES_IN` (8 h por defecto).
Resultado	Cadena JWT.

me(req, res)

Campo	Detalle
Propósito	Devolver la sesión actual con los roles frescos de la base. El frontend lo llama al arrancar para no depender de lo guardado en `localStorage`.
Parámetros	Ninguno (usa `req.user.id`).
Lógica	Lee `id`, `nombre`, `correo`, `roles` y `puedeAutogenerarToken` del usuario.
Resultado	`{ usuario }` o 401 si el usuario ya no existe.

logout(req, res)

Campo	Detalle
Propósito	Dejar constancia del cierre de sesión.
Parámetros	Ninguno.
Lógica	Registra `logout` en la bitácora. No hay lista de revocación: el JWT sigue siendo válido hasta que expire (R13).
Resultado	`{ ok: true }`.

forgotPassword(req, res)

Campo	Detalle
Propósito	Solicitar el enlace de recuperación de contraseña.
Parámetros	Cuerpo `{ correo }`.

Campo	Detalle
Lógica	Responde siempre con el mismo mensaje y código, exista o no el correo, para no revelar qué cuentas existen. Si existe, genera 32 bytes aleatorios, guarda solo su SHA-256 en `PasswordResetToken` con vigencia `PASSWORD_RESET_EXPIRES_MIN`, registra la solicitud y entrega el enlace por el adaptador de notificación. Con `PASSWORD_RESET_EXPOSE_TOKEN=true` (solo laboratorio) devuelve además el token.
Resultado	`{ ok: true, mensaje }`.

resetPassword(req, res)

Campo	Detalle
Propósito	Cambiar la contraseña con el token de recuperación.
Parámetros	Cuerpo `{ token, password }`.
Lógica	Exige ambos y una contraseña de al menos 8 caracteres (400). Busca el token por su hash y verifica que exista, no esté usado y no haya expirado. En una sola transacción actualiza el hash bcrypt (cost 10) y marca el token como usado. Registra `reset_password`.
Resultado	`{ ok: true, mensaje }`.

emitirToken({ usuarioId, pacienteId, emitidoPor }) (función interna)

Campo	Detalle
Propósito	Crear un token temporal de acceso al diagnóstico.
Parámetros	Titular, paciente opcional y quien lo emite.
Lógica	Genera 24 bytes aleatorios (48 hex), fija la expiración en `TEMP_TOKEN_EXPIRES_MIN` minutos (15 por defecto) y crea el `TokenTemporal`.
Resultado	`{ token, expiraEn }`.

solicitarTokenTemporal(req, res)

Campo	Detalle
Propósito	El Administrador emite un token para otro usuario (RF-33).
Parámetros	Cuerpo `{ usuarioId, pacienteId? }`.
Lógica	Exige `usuarioId` (400), verifica que el usuario exista (404) y emite el token con el administrador como emisor.
Resultado	`201 { ok: true, token, expiraEn }`.

autogenerarTokenTemporal(req, res)

Campo	Detalle
Propósito	Emitir un token al propio usuario (RF-34).
Parámetros	Cuerpo `{ pacienteId? }`.
Lógica	La ruta exige rol Administrador, de modo que en la práctica solo el Administrador llega al controlador. El controlador comprueba además `puedeAutogenerarToken` o el rol Administrador. Un usuario no administrador con ese permiso recibe 403 desde la ruta (riesgo R16).
Resultado	`201 { ok: true, token, expiraEn }`.

5.6 Usuarios

Archivo: `src/controllers/usuarios.controller.js`. **Rutas:** `/api/usuarios`.

Solo el Administrador lista, crea o reasigna roles; el catálogo de médicos es la excepción y lo puede consultar cualquier usuario autenticado. Un usuario puede tener varios roles a la vez. Dependencias: `bcrypt`, `utils/roles.util.js`, `actividad.service.js`.

Método	Ruta	Acceso
GET	`/usuarios/medicos`	autenticado
GET	`/usuarios`, `/usuarios/:id`	Administrador
POST	`/usuarios`	Administrador
PUT	`/usuarios/:id`	Administrador

listarMedicos(req, res)

Campo	Detalle
Propósito	Entregar el catálogo de médicos para elegir al médico responsable de una receta.
Parámetros	Ninguno.
Lógica	Filtra los usuarios con el rol Consulta y devuelve solo nombre, colegiado, especialidad y roles, ordenados por nombre.
Resultado	Arreglo de médicos.

listar(req, res) y obtenerUno(req, res)

Campo	Detalle
Propósito	Consultar usuarios.
Parámetros	`:id` para el detalle.
Lógica	Usan `SELECT_PUBLICO` (id, nombre, correo, roles, permiso de token, colegiado, especialidad, última actividad y fecha de alta); nunca devuelven el hash.
Resultado	Arreglo o un usuario; 404 si no existe.

crear(req, res)

Campo	Detalle
Propósito	Dar de alta un usuario.
Parámetros	`{ nombre, correo, password, roles[], puedeAutogenerarToken?, colegiado?, especialidad? }`.
Lógica	Exige nombre, correo y contraseña; valida los roles con `validarRoles` (al menos uno y todos válidos); genera el hash con bcrypt cost 10; guarda colegiado y especialidad solo si el usuario tiene el rol Consulta. No valida una longitud mínima de contraseña (R10). Registra `crear_usuario`.
Resultado	`201` con el usuario público.

actualizar(req, res)

Campo	Detalle
Propósito	Reasignar roles y el permiso de autogenerar token (RF-32 y RF-34).
Parámetros	`{ nombre?, roles?, puedeAutogenerarToken?, colegiado?, especialidad? }`.
Lógica	Valida los roles si vienen. Si el usuario deja de tener el rol Consulta, limpia colegiado y especialidad en la misma escritura. Traduce `P2025` a 404. Registra `actualizar_usuario`.
Resultado	El usuario actualizado.

validarRoles(roles) (función interna)

Campo	Detalle
Propósito	Validar el arreglo de roles de una alta o cambio.
Parámetros	Arreglo de roles.
Lógica	Exige un arreglo no vacío y que cada valor esté en `ROLES_VALIDOS`.
Resultado	`null` o el texto del error.

5.7 Pacientes

Archivo: `src/controllers/pacientes.controller.js`. **Rutas:** `/api/pacientes`.

Módulo de registro y admisión. Los datos clínicos de egreso (diagnósticos, complicaciones, operaciones, autopsia y causa de muerte) se guardan cifrados en `EgresoClinico`, separados de los datos administrativos; los listados nunca los exponen porque usan un `select` explícito (`SELECT_LISTADO`). Dependencias: `crypto.util.js`, `archivos.util.js`, `paginacion.util.js`, `actividad.service.js`.

Método	Ruta	Acceso
GET	`/pacientes`, `/pacientes/:id`	autenticado
POST	`/pacientes`	Recepción, Administrador
PUT	`/pacientes/:id`	Recepción, Administrador
DELETE	`/pacientes/:id`	Recepción, Administrador
GET	`/pacientes/:id/ingresos`	Administrador, Recepción, Consulta, Enfermería
POST	`/pacientes/:id/ingresos`	Recepción, Administrador

validarPaciente(body, { esEdicion })

Campo	Detalle
Propósito	Validar los datos de admisión (RF-01).
Parámetros	Cuerpo de la petición y bandera de edición parcial.
Lógica	En un alta exige fecha y hora de ingreso, historia clínica numérica (máximo 40 dígitos), nombre, dirección, teléfono, lugar y fecha de nacimiento, edad, sexo, estado civil (`S`, `C`, `U`, `D`, `V`), ocupación, religión, nacionalidad y DPI. En toda petición valida: DPI de 13 dígitos, teléfonos de 8 dígitos (paciente, cónyuge, emergencia y encargado), edad entera entre 0 y 130, nombre solo con letras y fecha de nacimiento no futura. En edición, nombre y DPI no son obligatorios, pero si vienen se validan.
Resultado	`null` o el texto del error (la ruta responde 400).

validarMaternidad(m)

Campo	Detalle
Propósito	Validar los datos del nacimiento.
Parámetros	Objeto `maternidad` o nulo.
Lógica	Número de hijo entero mayor que cero, fecha válida, DPI del padre de 13 dígitos y teléfono de 8 (ambos opcionales), hora `HH:MM`, y si hubo autopsia del bebé exige causa y fecha de muerte.
Resultado	`null` o el texto del error.

normalizarObjetosMultipart(body)

Campo	Detalle
Propósito	Cuando el alta llega como multipart, convertir `egresoClinico` y `maternidad` de texto a objeto.
Parámetros	Cuerpo de la petición.
Lógica	Intenta `JSON.parse` de cada campo y exige que sea un objeto.
Resultado	`null` o el texto del error.

tomarEgresoClinico(body) y validarEgresoClinico(payload, condicion)

Campo	Detalle
Propósito	Armar y validar el contenido clínico de egreso que se cifrará.
Parámetros	Cuerpo y la condición de egreso efectiva.
Lógica	`tomarEgresoClinico` acepta la forma nueva (`egresoClinico`) o los campos planos antiguos, con soporte para borrado explícito (`null`). `validarEgresoClinico` exige listas de diagnósticos y complicaciones con `descripcion` y `codigo`, lista de operaciones de texto, y que autopsia y causa de muerte solo se indiquen en condiciones de fallecimiento.
Resultado	Payload o `null`; el validador devuelve `null` o el texto del error (422).

egresoClinicoPara(paciente, autorizado)

Campo	Detalle
Propósito	Descifrar el egreso clínico solo para los roles autorizados.
Parámetros	Paciente con su fila `egresoClinico` y bandera de autorización.
Lógica	Si no está autorizado devuelve `undefined`. Si hay fila cifrada, la descifra y, si falla (llave rotada o dato corrupto), devuelve `null` sin exponer nada. Si no hay fila, usa los campos legacy de texto plano como respaldo (D13).
Resultado	Objeto de egreso, `null` o `undefined`.

listar(req, res)

Campo	Detalle
Propósito	Buscar pacientes por nombre, DPI o historia clínica (RF-02).
Parámetros	`?buscar=`, `?page=`, `?pageSize=`.
Lógica	Con `page` pagina y agrega la fecha escrita en el papel del expediente escaneado; sin `page` devuelve hasta 50 para los selectores. Nunca incluye datos clínicos de egreso.
Resultado	`{ items, total, page, pageSize }` o un arreglo.

obtenerUno(req, res)

Campo	Detalle
Propósito	Devolver la ficha completa de un paciente.
Parámetros	`:id`.
Lógica	Incluye maternidad, médico referente e ingresos (más reciente primero). El egreso clínico se descifra solo para Administrador y Recepción.
Resultado	Paciente o 404. Ver los riesgos R1 y R4 sobre el alcance de esta respuesta.

crear(req, res)

Campo	Detalle
Propósito	Registrar un paciente nuevo, con su historia clínica, sin duplicar el DPI (RF-01, RF-03, RF-04).
Parámetros	JSON o multipart: campos del paciente, `egresoClinico`, `maternidad`, archivo `documento` (PDF del expediente escaneado) y `adjuntos` (hasta 10).
Lógica	Valida en este orden: formato de datos, DPI repetido (409 sin exponer al paciente existente), egreso clínico (422), historia clínica repetida (409), PDF (422) y adjuntos (422). Después, en **una sola transacción**: crea el paciente; si no se capturó historia clínica genera `HC-AAAA-NNNNNN`; cifra y guarda el egreso clínico; cifra y guarda el PDF con su checksum SHA-256 como `DocumentoPaciente` de tipo `expediente_admision`; guarda los adjuntos como `resultado_clinico`; crea la maternidad; y crea el primer `Ingreso`. Registra `crear_paciente`.
Resultado	`201` con el paciente. Todo o nada: no queda un documento sin paciente.

actualizar(req, res)

Campo	Detalle
Propósito	Actualizar datos administrativos, egreso clínico y maternidad (RF-05 a RF-09).
Parámetros	`:id` y cuerpo parcial.
Lógica	Valida como edición; si cambia la historia clínica exige solo dígitos y que no esté asignada a otro paciente (409). En una transacción actualiza el paciente, hace `upsert` del egreso cifrado (o lo elimina si queda vacío) y de la maternidad. Traduce `P2025` a 404 y `P2002` a 409.
Resultado	El paciente actualizado.

validarIngreso(body), listarIngresos(req, res) y crearIngreso(req, res)

Campo	Detalle
Propósito	Gestionar las admisiones de un paciente, que puede ingresar varias veces.
Parámetros	`:id` del paciente; cuerpo `{ fechaIngreso, impresionClinicaIngreso?, serviciosSolicitados?, referidoDe?, referidoPor? }`.
Lógica	`validarIngreso` exige una fecha de ingreso válida. `listarIngresos` devuelve las admisiones de la más reciente a la más antigua. `crearIngreso` verifica que el paciente exista, valida y crea el ingreso, y registra `crear_ingreso`.
Resultado	Arreglo de ingresos o `201` con el ingreso.

eliminar(req, res)

Campo	Detalle
Propósito	Eliminar por completo un paciente y su expediente. Es destructiva e irreversible.
Parámetros	`:id`.
Lógica	Recupera los nombres de los archivos del disco. En una transacción borra accesos al diagnóstico, tratamientos, recetas, diagnósticos y sus archivos, documentos, egreso clínico, maternidad, ingresos, bitácora de visitas, **facturas del hospital** y tokens; desvincula (sin borrar) las facturas de farmacia; y borra al paciente. Después elimina los archivos cifrados del disco (mejor esfuerzo) y registra `eliminar_paciente`.
Resultado	`{ ok: true }`. Existe solo en la API; la interfaz no la ofrece. Ver el riesgo R17.

5.8 Documentos del paciente

Archivo: `src/controllers/documentos.controller.js`. **Rutas:** `/api/pacientes/:id/documentos`.

Comparte la tabla `DocumentoPaciente` para dos usos: el PDF del expediente escaneado al registrar al paciente y los documentos generales de un paciente existente (identificaciones, referencias, constancias). El binario se cifra con AES-256-GCM antes de guardarse en el volumen privado `uploads/documentos`; la tabla guarda solo metadatos, checksum y auditoría. La descarga siempre pasa por la API autorizada: el volumen nunca se sirve de forma estática. Dependencias: `multer`, `archivos.util.js`, `crypto.util.js`.

Método	Ruta	Acceso
GET	`/pacientes/:id/documentos` y `/:documentoId`	Administrador, Recepción, Consulta, Enfermería
POST	`/pacientes/:id/documentos`	Recepción, Administrador
DELETE	`/pacientes/:id/documentos/:documentoId`	Recepción, Administrador

uploadDocumento (multer)

Campo	Detalle
Propósito	Recibir los archivos en memoria con límite de tamaño.
Parámetros	Campos `documento` (1) y `adjuntos` (hasta 10); límite `DOCUMENT_MAX_MB` (20 MB).
Lógica	`multer.memoryStorage()`: el archivo nunca toca el disco en claro.
Resultado	`req.files` con los buffers.

subir(req, res)

Campo	Detalle
Propósito	Agregar un documento a un paciente existente.
Parámetros	Multipart: `documento`, y opcionales `nombreOriginal`, `tipoDocumental`, `paginas`, `fechaDocumentoOriginal`, `ingresoId`.
Lógica	Valida el archivo con `validarArchivo` (422), limpia el nombre original, comprueba que el paciente exista (404), que las páginas estén entre 1 y `DOCUMENT_MAX_PAGES` (422) y que el tamaño no exceda el máximo (413). Si trae `ingresoId`, exige que pertenezca al mismo paciente antes de escribir en disco. Calcula el SHA-256, cifra y guarda el binario, crea el registro y registra `subir_documento`.
Resultado	`201` con los metadatos del documento.

listar(req, res)

Campo	Detalle
Propósito	Listar los documentos vigentes de un paciente.
Parámetros	`:id`, `?page=`, `?pageSize=`.
Lógica	Excluye los eliminados lógicamente y devuelve solo metadatos (nunca la ruta física ni el nombre interno).
Resultado	Lista paginada o hasta 100 documentos.

descargar(req, res)

Campo	Detalle
Propósito	Entregar un documento descifrado.
Parámetros	`:id` y `:documentoId`.
Lógica	Verifica que el documento exista, pertenezca al paciente y no esté eliminado. Lee y descifra el archivo del volumen, registra `descargar_documento` y responde con el `Content-Type` real y `Content-Disposition: inline`.
Resultado	Binario del documento; 404 si falta; 500 si no se puede descifrar.

eliminar(req, res)

Campo	Detalle
Propósito	Retirar un documento de la vista sin destruir el binario (política de retención).
Parámetros	`:id` y `:documentoId`.
Lógica	Marca `eliminadoEn` con la fecha actual y registra `eliminar_documento`. Si ya estaba eliminado responde 409.
Resultado	`{ ok: true }`.

5.9 Lectura avanzada de expedientes

Archivo: `src/controllers/lecturaExpediente.controller.js`. **Rutas:** `/api/pacientes/lectura-avanzada/estado` y `/api/pacientes/leer-expediente`, ambas para Recepción y Administrador.

tipoRealDeImagen(buffer)

Campo	Detalle
Propósito	Detectar el tipo real de una imagen, porque el `Content-Type` del cliente no es confiable.
Parámetros	Buffer de la imagen.
Lógica	Compara los primeros bytes con las firmas de JPEG, PNG y WebP.
Resultado	El tipo MIME o `null`.

estado(req, res)

Campo	Detalle
Propósito	Indicar a la pantalla si debe ofrecer la lectura avanzada o ir directo al OCR local.
Parámetros	Ninguno.
Lógica	Consulta `lecturaAvanzadaDisponible()`.
Resultado	`{ disponible, modelo }`.

leer(req, res)

Campo	Detalle
Propósito	Leer con el modelo de visión los datos básicos de la ficha escaneada.
Parámetros	Multipart con el campo `imagen` (hasta 8 MB, en memoria).
Lógica	Responde 501 si no está configurada, 400 si falta la imagen y 422 si no es JPEG, PNG o WebP. Llama a `extraerDatosConVision` y registra en la bitácora solo que se usó la función, sin contenido clínico. Traduce `LecturaVisionError` a su código HTTP.
Resultado	`{ campos, modelo }`.

5.10 Expediente clínico y diagnóstico

Archivo: `src/controllers/expedientes.controller.js`. **Rutas:** `/api/expedientes`.

Módulo del diagnóstico confidencial. El acceso a `GET` y `POST` de `/expedientes/paciente/:id` pasa por `requireTempToken`: solo el Administrador o un usuario con token vigente puede ver o modificar el contenido. El contenido del diagnóstico es un JSON con la plantilla clínica, que se cifra antes de guardarse. Cada guardado crea una versión nueva; nunca se sobrescribe. Dependencias: `multer`, `crypto.util.js`, `archivos.util.js`.

Método	Ruta	Acceso
GET	`/expedientes`	autenticado (solo metadatos)
GET	`/expedientes/paciente/:id`	Administrador, u otro rol con `x-temp-token`
POST	`/expedientes/paciente/:id`	Administrador, u otro rol con `x-temp-token`

listar(req, res)

Campo	Detalle
Propósito	Saber si un paciente ya tiene diagnóstico sin exponer datos sensibles.
Parámetros	`?pacienteId=`.
Lógica	Devuelve solo `id`, `pacienteId`, `ingresoId`, `codigoCie` y fecha de los últimos 50 diagnósticos.
Resultado	Arreglo de metadatos.

obtenerUno(req, res)

Campo	Detalle
Propósito	Devolver el diagnóstico más reciente descifrado, su historial y las admisiones del paciente (RF-10, RF-11).
Parámetros	`:id` del paciente.
Lógica	Lee todas las versiones de más reciente a más antigua y las descifra con `descifrarDiagnostico`. Agrupa los diagnósticos bajo cada ingreso. Registra en `AccesoDiagnostico` la acción `ver`, si fue por token y su id (RNF-08). Responde 404 si no hay diagnóstico.
Resultado	`{ ...actual, historial, ingresos }`.

descifrarDiagnostico(diagnostico) e inlineArchivosEstudios(datos, pacienteId) (funciones internas)

Campo	Detalle
Propósito	Descifrar una versión y adjuntar sus archivos para mostrarlos sin otro endpoint.
Parámetros	Fila de `Diagnostico`; datos descifrados y paciente.
Lógica	Descifra el JSON; si el contenido es un texto plano anterior a la plantilla lo devuelve como `legacyTexto`. Por cada estudio, archivo del diagnóstico y adjunto adicional, valida que el archivo pertenezca al mismo paciente, lo descifra del volumen y lo incrusta como `data:` URI en base64.
Resultado	Diagnóstico con `datos` o `legacyTexto`, y archivos incrustados.

crear(req, res)

Campo	Detalle
Propósito	Registrar un diagnóstico cifrado (RF-10).
Parámetros	Multipart: `datos` (JSON), `ingresoId?`, `tipoAtencion?`, `servicio?`, `salaCama?`, `codigoCie?` y archivos `estudio_<i>`, `diagnostico_adjunto`, `adjunto_extra_<i>`.
Lógica	Valida el JSON (400). Determina el ingreso: el indicado (404 si no es del paciente) o el más reciente. Valida cada archivo (422), cifra cada uno antes de tocar el disco y deja solo su metadato en `DiagnosticoArchivo`. Cifra `datos` con AES-256-GCM, crea la fila de `Diagnostico` y registra en `AccesoDiagnostico` la acción `registrar`.
Resultado	`201 { ok, id, pacienteId, codigoCie, creadoEn }`.

5.11 Escaneo móvil

Archivo: `src/controllers/escaneoMovil.controller.js`. **Rutas:** `/api/escaneo-movil`.

Vincula la computadora con la cámara del teléfono. La sesión es un intercambio efímero: la computadora (autenticada) crea una sesión de un solo uso; el teléfono, sin inicio de sesión y autorizado solo por conocer el identificador aleatorio, sube el PDF; la computadora lo recoge. No se guarda en base de datos ni en disco: vive en un `Map` en memoria y expira a los 10 minutos. Un barrido cada 5 minutos limpia las sesiones abandonadas.

Método	Ruta	Acceso
POST	`/escaneo-movil/sesiones`	Recepción, Administrador
GET	`/escaneo-movil/sesiones/:id/estado`	propietario de la sesión
GET	`/escaneo-movil/sesiones/:id/pdf`	propietario de la sesión
DELETE	`/escaneo-movil/sesiones/:id`	propietario de la sesión
GET	`/escaneo-movil/sesiones/:id/info`	público (30 peticiones por 5 min)
POST	`/escaneo-movil/sesiones/:id/subir`	público (30 peticiones por 5 min)

crear(req, res)

Campo	Detalle
Propósito	Abrir una sesión de escaneo.
Parámetros	Ninguno.
Lógica	Limpia las vencidas, genera un UUID v4 con `crypto.randomUUID` y guarda `{ creadoPor, expiraEn, estado: "esperando" }`.
Resultado	`201 { id, expiraMinutos }`.

estado(req, res), obtenerPdf(req, res) y cancelar(req, res)

Campo	Detalle
Propósito	Lado de la computadora: consultar el avance, recoger el PDF y cancelar.
Parámetros	`:id` de la sesión.
Lógica	`obtenerPropia` devuelve 404 si la sesión no existe o expiró y 403 si pertenece a otro usuario. `estado` informa estado, páginas y nombre. `obtenerPdf` exige estado `listo` (409 si no) y entrega el PDF **una sola vez**: al servirlo elimina la sesión. `cancelar` elimina la sesión.
Resultado	`{ estado, paginas, nombreOriginal }`, el PDF, o `{ ok: true }`.

info(req, res) y subirPdf(req, res)

Campo	Detalle
Propósito	Lado del teléfono: comprobar que el enlace es válido y subir el PDF.
Parámetros	`:id`; multipart con `documento` (PDF, hasta 20 MB), `paginas` y `nombre`.
Lógica	`info` devuelve si la sesión es válida. `subirPdf` responde 404 si la sesión no existe, 409 si ya recibió un documento, 400 si falta el archivo y 422 si no empieza con `%PDF-`. Guarda el buffer en memoria y cambia el estado a `listo`.
Resultado	`{ valido, estado }` o `{ ok: true }`.

6. Documentación técnica del frontend

Esta sección documenta la aplicación web de una sola página (SPA) desarrollada con React 18, Vite 5 y Tailwind CSS. El código vive en `frontend/src` y se organiza en puntos de entrada, contexto de autenticación, servicios, hooks, utilidades, componentes y páginas. Cada pieza se describe con su propósito, parámetros, lógica y resultado, igual que en el backend.

6.1 Punto de entrada y enrutamiento

Archivos: `src/main.jsx`, `src/App.jsx`.

Dependencias: `react-router-dom`, `AuthContext`, `utils/roles.js`.

main.jsx

Campo	Detalle
Propósito	Montar la aplicación en el elemento `#root`.
Parámetros	Ninguno.
Lógica	Envuelve el árbol en `BrowserRouter` y `AuthProvider`, en ese orden, y renderiza `App`.
Resultado	Aplicación con enrutamiento y sesión disponibles para todos los componentes.

App()

Campo	Detalle
Propósito	Declarar las rutas públicas y las protegidas por rol.
Parámetros	Ninguno. Lee la sesión con `useAuth()`.
Lógica	Las rutas públicas son `/recuperar`, `/recuperar/cambiar`, `/escaneo-movil/:sesionId` y el comodín `*`, que muestra el inicio de sesión. Si no hay sesión, cualquier otra ruta lleva al login. Con sesión, cada ruta se envuelve en `ModuloProtegido`, que consulta `puedeVerModulo(usuario, modulo)`. Si el rol no puede ver el módulo, redirige al primer módulo permitido.
Resultado	El módulo solicitado dentro de `Layout`, o una redirección.

ModuloProtegido, Redirigir y ConPacienteDeUrl (componentes internos de App.jsx)

Componente	Propósito	Lógica
`ModuloProtegido`	Bloquear módulos no permitidos al rol	Recibe el nombre del módulo y sus hijos; si `puedeVerModulo` es falso, devuelve `Redirigir`
`Redirigir`	Enviar al usuario a su módulo de inicio	Calcula el primer módulo de `modulosPermitidos(usuario)` y navega con `replace`

Componente	Propósito	Lógica
`ConPacienteDeUrl`	Cargar el paciente indicado en `?paciente=`	Lee el parámetro, consulta `/pacientes/:id` y pasa el paciente al módulo

Rutas protegidas:

Ruta	Módulo	Página
`/registro`	registro	`Registro.jsx`
`/expediente`	expediente	`Expediente.jsx`
`/documentos` (y `/anexos`, que redirige)	registro	`Documentos.jsx`
`/tratamiento`	expediente	`Tratamiento.jsx`
`/referidos`	referidos	`Referidos.jsx`
`/financiera`	financiera	`Financiera.jsx`
`/gastos`	gastos	`Gastos.jsx`
`/farmacia`	farmacia	`Farmacia.jsx`
`/seguridad`	seguridad	`Seguridad.jsx`
`/reportes`	reportes	`Reportes.jsx`
`/bitacora`	bitacora	`Bitacora.jsx` (sin entrada en el menú; solo el Administrador la abre por URL)

6.2 Contexto de autenticación

Archivo: `src/context/AuthContext.jsx`.

AuthProvider({ children })

Campo	Detalle
Propósito	Mantener la sesión del usuario y ofrecerla a todo el árbol de componentes.
Parámetros	`children`.
Lógica	Al arrancar lee el token de `localStorage` y llama a `GET /auth/me` para refrescar usuario y roles desde el servidor, sin confiar en lo guardado. Expone `login(correo, password)`, `logout()` y `sesionExpirada`. Escucha el evento global `auth:sesion-expirada`, que dispara `api.js` ante un 401, para cerrar la sesión y mostrar el aviso.
Resultado	Valor de contexto `{ usuario, cargando, login, logout, sesionExpirada }`.

useAuth()

Campo	Detalle
Propósito	Acceso al contexto desde cualquier componente.
Parámetros	Ninguno.
Lógica	`useContext(AuthContext)`.
Resultado	El valor del contexto.

6.3 Servicio de API

Archivo: `src/services/api.js`.

Función	Propósito	Parámetros	Lógica	Resultado
`request(method, path, body)`	Llamada JSON a la API	Método, ruta relativa a `/api` y cuerpo	Agrega `Authorization: Bearer` desde `localStorage`; si el cuerpo es `FormData` no fija `Content-Type`; ante 401 emite `auth:sesion-expirada`; ante error lanza `Error` con el mensaje del servidor	Cuerpo JSON de la respuesta
`requestBlob(path)`	Descargar binarios (PDF, imágenes)	Ruta	Igual que la anterior pero devuelve `Blob`	`Blob`
`api.get/post/put/del`	Atajos por verbo	Ruta y cuerpo	Delegan en `request`	JSON
`api.getBlob`	Atajo de descarga	Ruta	Delega en `requestBlob`	`Blob`

La constante `BASE_URL` es `/api`, y Nginx reenvía esas rutas al contenedor del backend (sección 3.2), por lo que el navegador nunca conoce la dirección interna.

6.4 Hooks

Archivos: `src/hooks/useFetch.js`, `src/hooks/usePaginatedFetch.js`.

Hook	Propósito	Parámetros	Lógica	Resultado
`useFetch(path, { enabled })`	Cargar datos al montar o al cambiar la ruta	Ruta y bandera de activación	Llama a `api.get` cuando `enabled` es verdadero y descarta respuestas de peticiones superadas	`{ data, loading, error, reload }`
`usePaginatedFetch(path, { pageSize })`	Listados paginados con búsqueda	Ruta base y tamaño de página	Mantiene `page` y `buscar`, reinicia la página al cambiar la búsqueda y arma `?page=&pageSize=&buscar=`	`{ items, total, page, setPage, buscar, setBuscar, loading, error, reload }`

6.5 Utilidades

Carpeta: `src/utils`.

Roles [roles.js]

Función	Propósito	Lógica	Resultado
`tieneRol(usuario, ...roles)`	Saber si el usuario tiene alguno de los roles	Compara contra `usuario.roles`	Booleano
`etiquetasRoles(usuario)`	Mostrar roles legibles	Mapea con `ROLE_LABELS`	Texto separado por comas
`modulosPermitidos(usuario)`	Módulos que ve el usuario	Une los módulos de `MODULOS_POR_ROL` para todos sus roles	Arreglo de módulos
`puedeVerModulo(usuario, modulo)`	Autorizar una ruta o botón	Consulta `modulosPermitidos`	Booleano

La matriz `MODULOS_POR_ROL` es la que aplica el frontend. La seguridad real vive en el backend (sección 8): el frontend solo decide qué se muestra.

Rol	Módulos visibles
Administrador	Todos
Recepción	Registro, Expediente
Enfermería	Registro, Expediente
Consulta	Registro, Expediente
Facturación	Registro, Expediente, Financiera, Gastos, Referidos
Farmacia	Registro, Farmacia

Validadores y formato

Archivo	Función	Propósito y lógica
`dpi.js`	`limpiarDPI`, `formatearDPI`, `validarDPI`	Deja solo dígitos, agrupa en `1234 12345 1234` y valida los 13 dígitos: dígito verificador con los 8 primeros dígitos por los pesos 2 a 9, módulo 11, y departamento entre 1 y 22
`telefono.js`	Utilidades de teléfono	Normaliza a 8 dígitos y formatea
`fechas.js`	`fechaDia`, `fechaLocal`, `fechaHoraLocal`, `hoyISO`, `calcularEdad`	Muestra fechas de día en UTC para evitar el corrimiento de un día y las de hora en la zona local; calcula la edad
`texto.js`	`normalizarTexto`	Quita acentos y pasa a minúsculas para búsquedas
`guatemala.js`	Catálogos	Departamentos y municipios
`condicionesEgreso.js`	Catálogo	Condiciones de egreso y cuáles implican fallecimiento
`archivos.js`	`abrirArchivoDataUrl`	Abre en una pestaña un archivo incrustado como `data:` URI

Escaneo, OCR y PDF

Todo este procesamiento ocurre en el navegador del usuario; ninguna imagen sale del equipo, salvo en el uso opcional de lectura avanzada (sección 5.9).

Archivo	Funciones	Propósito y lógica
`scanProcessing.js`	`idAleatorio`, `canvasDesdeDataUrl`, `detectarEsquinasMejorada`, `detectarEsquinas`, `rectificarPerspectiva`, `rotar90`, `aplicarFiltro`, `procesarPagina`, `procesarEscaneoAutomatico`	Convierte una foto en una página de documento: detecta las esquinas de la hoja, corrige la perspectiva, rota y aplica un filtro (color, escala de grises o blanco y negro). `procesarEscaneoAutomatico` encadena todo para el modo automático

Archivo	Funciones	Propósito y lógica
`pdfDocumentos.js`	`esPdf`, `TAMANOS_PDF`, `DOCUMENTO_CONFIG`, `generarPdfDePaginas`, `nombreDescarga`, `tamanoLegibleMB`, `validarPdf`, `descargarPdf`, `leerInfoPdf`, `renderizarPrimeraPaginaPdf`	Arma un PDF con `pdf-lib` a partir de las páginas procesadas, lo valida (firma `%PDF-`, tamaño y páginas máximos), lee su información y renderiza la primera página con `pdfjs-dist`
`idbPaginas.js`	`leerPaginasGuardadas`, `guardarPaginas`, `borrarPaginas`	Guarda en IndexedDB las páginas capturadas para no perderlas si se recarga la pantalla
`ocrDatosBasicos.js`	`parsearFecha`, `extraerCampos`, `combinar`, `reconocer`, `extraerDatosBasicos`	OCR local con `tesseract.js` para sugerir los datos básicos de la ficha. `extraerCampos` aplica expresiones regulares al texto; `combinar` fusiona las pasadas de OCR; los valores solo se sugieren y la persona los confirma

6.6 Componentes reutilizables

Carpeta: `src/components`.

Componente	Propósito
`Layout`	Estructura general: barra lateral con los módulos permitidos al rol, encabezado con usuario y cierre de sesión, y el modo restringido
`PageHeader`	Título y acciones de cada página
`Card`	Contenedor con borde y sombra
`Button`	Botón con variantes (primario, secundario, peligro)
`Modal`	Diálogo accesible con cierre por Escape
`Banner`	Mensajes de éxito, error o aviso
`Table`	Tabla con encabezados y estado vacío
`Pagination`	Controles de página
`FormField`, `TextInput`, `Select`, `TextArea`	Campos de formulario con etiqueta y mensaje de error
`Combobox`	Selector con búsqueda
`PacienteBuscador`	Búsqueda de pacientes por nombre, DPI o historia clínica
`Cie10Input`	Búsqueda de diagnósticos en el catálogo CIE-10 (`/cie10`)
`RolesChecklist`	Selección múltiple de roles de un usuario
`FichaHeader`, `FichaSeccion`, `FichaCampo`	Bloques de la ficha del paciente
`FichaPacienteImprimible`, `RecetaImprimible`, `FacturaImprimible`	Vistas con estilos de impresión para ficha, receta y factura
`CameraScannerModal`	Captura de páginas con la cámara del equipo y vista previa
`EscaneoQRModal`	Muestra el código QR de una sesión de escaneo móvil y consulta el estado hasta recibir el PDF

6.7 Páginas

Carpeta: `src/pages`. Cada página es un módulo funcional del sistema; el resto del flujo (formularios, validaciones y vistas) está descrito en el Manual de usuario.

Página	Función	Endpoints principales
`Login`	Inicio de sesión	`POST /auth/login` (vía `AuthContext`)
`RecuperarPassword`	Solicitar el enlace de recuperación	`POST /auth/forgot-password`
`CambiarPassword`	Definir la contraseña nueva	`POST /auth/reset-password`
`Registro`	Alta, edición y búsqueda de pacientes; carga del expediente escaneado	`GET/POST /pacientes`, `PUT /pacientes/:id`, `POST /pacientes/:id/documentos`, `POST /referidos`, `POST /pacientes/leer-expediente`
`Documentos`	Documentos generales del paciente	`GET/POST/DELETE /pacientes/:id/documentos`
`Expediente`	Diagnóstico confidencial, admisiones e historial	`GET/POST /expedientes/paciente/:id`, `POST /auth/token/auto`, `POST /pacientes/:id/ingresos`
`Tratamiento`	Tratamientos y recetas	`POST /tratamientos`, `POST /recetas`
`Referidos`	Registro de pacientes referidos	`POST /referidos`
`Financiera`	Facturas del hospital y costeo	`POST /facturacion`
`Gastos`	Gastos y categorías fiscales	`POST/DELETE /facturacion/gastos`, `POST /facturacion/categorias-fiscales`
`Farmacia`	Inventario, entradas, salidas y ventas	`POST /farmacia`, `/farmacia/ventas`, `/farmacia/:id/salidas`, `PUT /farmacia/:id`
`Reportes`	Reportes financieros, de admisiones, inventario y auditoría	`GET /reportes/...`
`Seguridad`	Usuarios, roles y tokens temporales	`POST /auth/token`, `POST/PUT /usuarios`
`Bitacora`	Registro de actividad (solo por URL, Administrador)	`POST /bitacora`
`EscaneoMovil`	Página pública del teléfono para subir el PDF	`GET /escaneo-movil/sesiones/:id/info`, `POST .../subir`

Modo restringido

El Administrador puede activar el modo restringido haciendo clic en el logotipo del `Layout`. En ese modo solo quedan visibles los elementos de `NAV_ITEMS` con la marca `allowedInRestricted`, lo que reduce la exposición de módulos sensibles durante demostraciones o en equipos compartidos. Es una opción de la interfaz y no sustituye los controles del backend.

6.8 Construcción y servido

Elemento	Detalle
Scripts de `package.json`	`dev` (Vite en modo desarrollo), `build` (empaquetado a `dist/`), `preview`, `test` (`node --test "tests/*.test.mjs"`) y `test:e2e` (Playwright)
Imagen Docker	Construcción en dos etapas: `node` compila con Vite y `nginx` sirve `dist/`
Nginx	Fallback de SPA a `index.html`; proxy de `/api/` hacia `backend:4000`; tipo MIME corregido para `.mjs`; `client_max_body_size 20m`
Proxy en desarrollo	Vite reenvía `/api` al backend local para evitar CORS

7. Referencia de la API REST

Base: `/api`. Todas las rutas, salvo las marcadas como **público**, requieren `Authorization: Bearer <JWT>`. Los roles se abrevian así: **ADM** Administrador, **REC** Recepción, **CON** Consulta, **ENF** Enfermería, **FAC** Facturación, **FAR** Farmacia. El detalle de cada función está en la sección 5; esta sección es la consulta rápida de rutas, roles y cuerpos.

7.1 Autenticación y seguridad (`/api/auth`)

Método	Ruta	Roles	Descripción
POST	`/auth/login`	público (límite de intentos)	Cuerpo `{correo, password}`. Responde `{token, usuario}`.
POST	`/auth/logout`	autenticado	Registra el cierre de sesión.
POST	`/auth/forgot-password`	público (límite de intentos)	Cuerpo `{correo}`. Respuesta genérica.
POST	`/auth/reset-password`	público (límite de intentos)	Cuerpo `{token, password}` (mínimo 8 caracteres).
POST	`/auth/token`	ADM	Cuerpo `{usuarioId, pacienteId?}`. Responde `201 {token, expiraEn}`.
POST	`/auth/token/auto`	ADM	Cuerpo `{pacienteId?}`. Emite el token al propio administrador.
GET	`/auth/me`	autenticado	Devuelve el usuario con sus **roles actuales** leídos de la base de datos.

7.2 Usuarios (`/api/usuarios`)

Método	Ruta	Roles	Descripción
GET	`/usuarios/medicos`	autenticado	Usuarios con rol Consulta, para elegir el médico de una receta.
GET	`/usuarios`, `/usuarios/:id`	ADM	Listado y detalle.
POST	`/usuarios`	ADM	Cuerpo `{nombre, correo, password, roles[], puedeAutogenerarToken?, colegiado?, especialidad?}`.
PUT	`/usuarios/:id`	ADM	Actualización parcial. Limpia colegiado y especialidad si el usuario deja de ser Consulta.

7.3 Pacientes (`/api/pacientes`)

Método	Ruta	Roles	Descripción
GET	`/pacientes`	autenticado	`?buscar=`; con `?page=&pageSize=` responde `{items,total,page,pageSize}`; sin `page`, máximo 50.
GET	`/pacientes/:id`	autenticado	Datos del paciente, `ingresos[]` y `egresoClinico` descifrado solo para ADM y REC. Ver R1 y R4.

Método	Ruta	Roles	Descripción
POST	/pacientes	REC, ADM	JSON o multipart: campos del paciente, documento (PDF del escaneo), adjuntos (hasta 10), paginas, fechaDocumentoOriginal. 201; 409 si el DPI existe.
PUT	/pacientes/:id	REC, ADM	Actualización parcial: datos, egresoClinico (null lo elimina) y maternidad.
DELETE	/pacientes/:id	REC, ADM	**Eliminación física** del paciente y sus registros asociados, incluidas las facturas del hospital. Solo existe en la API. Ver R17.
GET	/pacientes/lectura-avanzada/estado	REC, ADM	Indica si la lectura avanzada está activa.
POST	/pacientes/leer-expediente	REC, ADM	Imagen (máximo 8 MB) → datos básicos leídos por visión.
GET	/pacientes/:id/ingresos	ADM, REC, CON, ENF	Historial de ingresos, el más reciente primero.
POST	/pacientes/:id/ingresos	REC, ADM	Cuerpo {fechaIngreso, impresionClinicaIngreso?, serviciosSolicitados?, referidoDe?, referidoPor?}.
GET	/pacientes/:id/documentos	ADM, REC, CON, ENF	Metadatos paginados o lista de hasta 100.
GET	/pacientes/:id/documentos/:documentoId	ADM, REC, CON, ENF	Descarga del binario descifrado con su Content-Type real.
POST	/pacientes/:id/documentos	REC, ADM	Multipart: documento (PDF, PNG o JPG), nombreOriginal?, tipoDocumental?, paginas?, fechaDocumentoOriginal?, ingresoId?. Máximo 30 páginas.
DELETE	/pacientes/:id/documentos/:documentoId	REC, ADM	Borrado lógico.

7.4 Expediente clínico (/api/expedientes)

Método	Ruta	Roles	Descripción
GET	/expedientes	autenticado	Metadatos de los últimos 50 diagnósticos (sin contenido).
GET	/expedientes/paciente/:id	ADM sin token, u otro rol con x-temp-token	Diagnóstico más reciente descifrado, historial[] e ingresos[]. 404 si no hay diagnóstico. Registra AccesoDiagnostico.
POST	/expedientes/paciente/:id	ADM, u otro rol con x-temp-token	Multipart: datos (JSON de la plantilla), ingresoId?, tipoAtencion?, servicio?, salaCama?, codigoCie? y archivos de estudios (máximo 5 MB cada uno). Crea una versión nueva. 201.

Estructura de datos (resumen): motivoConsulta, historiaEnfermedad, signosSintomas[], signosSintomasOtros, signosVitales{pa,fc,fr,temp,spo2,peso,talla}, exploracionFisica{general,cabezaCuello,cardiovascular,respiratorio,abdomen,neurol

ogico,extremidades,piel,otros}, estudios[{tipo,fecha,resultado,archivoId?}], diagnosticoPrincipal{diagnostico,codigoCie,tipo,prioridad,estado,fecha}, diagnosticosSecundarios[], impresionClinica y planConducta.

7.5 Tratamiento, recetas y referidos

Método	Ruta	Roles	Descripción
GET	/tratamientos, /tratamientos/:id	autenticado	?pacienteId= en el listado.
POST	/tratamientos	ENF, CON, ADM	Cuerpo {pacienteId, descripcion, costo ≥ 0, origen: intrahospitalario|farmacia, dosis?, cirujano?, ayudante?, instrumentista?, anestesiologo?}.
PUT	/tratamientos/:id	ENF, CON, ADM	Actualización.
GET	/recetas, /recetas/:id	autenticado	Consulta.
POST	/recetas	CON, ADM	Cuerpo {pacienteId, medicoId, medicamento, dosis, indicaciones, duracion}. medicoId debe ser un usuario con rol Consulta (422 si no).
GET	/referidos, /referidos/:id	autenticado	Consulta.
POST, PUT	/referidos, /referidos/:id	REC, ADM	Cuerpo {nombre, comisionQ, especialidad?}.

7.6 Facturación y gastos (/api/facturacion)

Método	Ruta	Roles	Descripción
GET	/facturacion/reporte	ADM, FAC	Reporte por rango (?desde=&hasta=).
GET	/facturacion/costeo/:pacienteId	ADM, FAC	Costo pendiente de tratamientos y total estimado.
POST	/facturacion	FAC, ADM	Cuerpo {pacienteId, costoHospital ≥ 0, formaPago: transferencia|efectivo}. Transacción.
GET	/facturacion, /facturacion/:id	ADM, FAC	Listado y detalle.
GET	/facturacion/gastos	ADM, FAC	desde, hasta, categoriaFiscalId y page.
POST	/facturacion/gastos	ADM, FAC	Cuerpo {fecha, proveedor, monto > 0, categoriaFiscalId?, numeroFactura?, descripcion?}. Calcula montoImpuestoEstimado.
DELETE	/facturacion/gastos/:id	ADM, FAC	Elimina el gasto.
GET	/facturacion/categorias-fiscales	ADM, FAC	Listado.
POST, PUT	/facturacion/categorias-fiscales[/:id]	ADM	Alta y edición.

7.7 Farmacia (`/api/farmacia`)

Método	Ruta	Roles	Descripción
GET	`/farmacia`, `/farmacia/:id`	FAR, ADM	Inventario; el campo `estado` es una lista que puede contener `ok`, `stock_bajo` y `por_vencer` (60 días o menos).
POST	`/farmacia`	FAR, ADM	Alta de medicamento.
PUT	`/farmacia/:id`	FAR, ADM	Actualización.
POST	`/farmacia/:id/entradas`	FAR, ADM	Cuerpo `{cantidad > 0}`. Crea un movimiento de entrada.
POST	`/farmacia/:id/salidas`	FAR, ADM	Cuerpo `{cantidad > 0, pacienteId}`. 409 si no hay stock.
POST	`/farmacia/ventas`	FAR, ADM	Cuerpo `{items:[{medicamentoId, cantidad}], pacienteId?}`. Transacción.
GET	`/farmacia/ventas`	FAR, ADM	Facturas de farmacia.
GET	`/farmacia/ventas/:id`	FAR, ADM, FAC	Detalle para impresión.

7.8 Bitácora de visitas (`/api/bitacora`)

El módulo conserva su API y su ruta `/bitacora` en la interfaz, pero no aparece en el menú: solo el Administrador accede escribiendo la dirección.

Método	Ruta	Roles	Descripción
GET	`/bitacora`, `/bitacora/:id`	autenticado	`?pacienteId=`.
POST, PUT	`/bitacora`, `/bitacora/:id`	ENF, REC, ADM	Cuerpo `{pacienteId, descripcion}`.

7.9 Reportes (`/api/reportes`)

Todas las rutas requieren ADM. Parámetros: `desde` y `hasta` en la mayoría; `granularidad` (`dia`, `semana` o `mes`, por defecto `mes`) en `ingresos-por-mes`; `medicamentoId` en el kardex; `usuarioId`, `accion`, `buscar` y `page` en actividad; `pacienteId` y `buscar` en la auditoría.

Ruta	Descripción
`/reportes/financiero`	Ingresos de hospital y farmacia, gastos, impuesto estimado e ingreso neto (RF-21).
`/reportes/ingresos-por-mes`	Ingresos agrupados por periodo.
`/reportes/admisiones`	Admisiones por condición de egreso.
`/reportes/facturacion-por-forma-pago`	Totales por efectivo y transferencia.
`/reportes/inventario-kardex`	Kardex de inventario (RNF-10).
`/reportes/auditoria-diagnostico`	Accesos al diagnóstico (RNF-08).
`/reportes/actividad`	Bitácora de actividad filtrable por acción.

7.10 Escaneo móvil (`/api/escaneo-movil`)

Método	Ruta	Acceso	Descripción
POST	`/escaneo-movil/sesiones`	REC, ADM	Crea la sesión y devuelve `{id, expiraMinutos}`.
GET	`/escaneo-movil/sesiones/:id/estado`	propietario	`{estado, paginas, nombreOriginal}`.
GET	`/escaneo-movil/sesiones/:id/pdf`	propietario	Entrega el PDF una sola vez y elimina la sesión.
DELETE	`/escaneo-movil/sesiones/:id`	propietario	Cancela la sesión.
GET	`/escaneo-movil/sesiones/:id/info`	**público**	`{valido, estado}`.
POST	`/escaneo-movil/sesiones/:id/subir`	**público**	Multipart `documento` (PDF, 20 MB). Una sola vez por sesión.

7.11 Catálogo CIE-10 y sistema

Método	Ruta	Acceso	Descripción
GET	`/cie10?q=`	autenticado	Búsqueda por código o descripción.
GET	`/health`	público	Estado del servicio y de la base de datos (ruta completa `/api/health`).

7.12 Contrato de errores

Toda respuesta de error tiene la forma `{ "error": "mensaje" }` con un código HTTP:

Código	Uso
400	Validación de entrada o campos obligatorios ausentes.
401	Sin token, token inválido o expirado.
403	Rol no permitido; token temporal requerido, inválido, usado o expirado.
404	Recurso inexistente.
409	Duplicado (DPI, historia clínica, correo), stock insuficiente, sesión ya usada.
422	Archivo o contenido inválido; reglas de negocio.
429	Límite de intentos excedido (con `Retry-After`).
500	Error no controlado.
503	`/api/health` cuando la base de datos no responde.

7.13 Transacciones

Operación	Función	Qué garantiza
Alta de paciente con documento, adjuntos, maternidad, egreso e ingreso inicial	`pacientes.crear`	Todo o nada
Actualización de paciente	`pacientes.actualizar`	Datos, egreso cifrado y maternidad juntos
Factura del hospital	`generarFacturaHospital`	Costeo, factura y marcado `facturado` atómicos (RNF-13)
Venta de farmacia	`registrarVentaFarmacia`	Stock, factura, líneas y kardex juntos
Restablecer contraseña	`auth.resetPassword`	Cambio de hash y consumo del token juntos

Operación	Función	Qué garantiza
Eliminar paciente	`pacientes.eliminar`	Borrado completo o nada

8. Seguridad

8.1 Autenticación

- `POST /api/auth/login` valida `correo` y `password` con `bcrypt.compare` y emite un JWT firmado con HS256 (`jsonwebtoken`, secreto `JWT_SECRET`).
- El payload del JWT es `{ id, nombre, roles, iat, exp }` y expira según `JWT_EXPIRES_IN` (8 horas por defecto).
- **Los roles se leen de la base de datos en cada petición.** `requireAuth` vuelve a consultar al usuario con el `id` del token, por lo que un cambio de roles o la baja de un rol se aplica de inmediato, sin esperar a que la persona inicie sesión otra vez. El campo `roles` del token solo sirve de respaldo informativo.
- **No hay lista de revocación.** `POST /api/auth/logout` solo registra el evento. Un token robado es válido hasta su expiración, aunque deja de servir si se elimina al usuario (R13).
- El frontend guarda el token en `localStorage` y lo envía como Bearer en cada petición. Al iniciar consulta `GET /auth/me` para refrescar usuario y roles.

8.2 Contraseñas

- Hash bcrypt con cost 10.
- Recuperación: `forgot-password` genera 32 bytes aleatorios, guarda solo su SHA-256 y entrega el enlace por el adaptador de notificación. Vence en `PASSWORD_RESET_EXPIRES_MIN`. La respuesta es idéntica exista o no el correo.
- `reset-password` exige token vigente y sin usar, y una contraseña de al menos 8 caracteres; el cambio y el consumo del token ocurren en una misma transacción.
- La creación de usuarios solo exige que la contraseña no esté vacía; no valida longitud mínima (R10).

8.3 Autorización por rol

`requireRole(...)` compara los roles vigentes del usuario con los de la ruta; con varios roles se obtiene la unión de sus permisos. **La autorización se aplica en el backend**: la interfaz oculta módulos por comodidad, pero no es una barrera de seguridad. La matriz de rutas y roles está en la sección 7.

8.4 Tokens de acceso temporal

1. **Emisión.** El Administrador la hace con `POST /api/auth/token` (usuario y paciente opcional) y la autogeneración con `POST /api/auth/token/auto`. Ambas rutas exigen rol Administrador. El token tiene 24 bytes aleatorios (48 hex) y vigencia de `TEMP_TOKEN_EXPIRES_MIN` (15 minutos). Un usuario no administrador con `puedeAutogenerarToken` recibe 403 (R16).
2. **Verificación** en `GET` y `POST /api/expedientes/paciente/:id`. El Administrador no necesita token. Los demás roles envían `x-temp-token` y se comprueba que exista, no esté usado, no haya expirado, pertenezca al usuario y, si trae paciente, que coincida con el de la ruta.
3. **Consumo.** Si todo es válido se marca `usado = true` antes de ejecutar el controlador (uso único, RNF-12).
4. **Auditoría.** Cada lectura y cada registro crea una fila en `AccesoDiagnostico` con el uso o no de token.

La secuencia completa y el ciclo de vida del token están en la sección 3.7.

8.5 Cifrado

- Algoritmo **AES-256-GCM** con IV aleatorio de 12 bytes por operación y etiqueta de autenticación de 16 bytes.
- **Campos cifrados:** `Diagnostico.textoCifrado` (JSON de la plantilla clínica) y `EgresoClinico.textoCifrado`. Cada fila guarda `iv` y `authTag`.
- **Archivos:** sobre `[IV 12][authTag 16][ciphertext]` con nombre interno `enc_<48 hex>.bin`. Los archivos sin el prefijo `enc_` se leen en claro (históricos) y se pueden cifrar con `scripts/recifrar-archivos.js`.
- Una etiqueta que no coincide provoca un error de descifrado: delata una llave incorrecta o un dato alterado.
- `ENCRYPTION_KEY` debe tener 64 caracteres hexadecimales y se valida al arrancar.

8.6 Manejo de archivos

La validación de `archivos.util.js` exige tres condiciones a la vez: el MIME declarado está permitido (PDF, PNG o JPEG por defecto), la extensión corresponde al MIME y los primeros bytes son la firma del formato. El nombre original nunca se usa para escribir en disco y se rechazan nombres con `/`, `\` o `..`. Los binarios se procesan en memoria y se cifran antes de llegar al disco.

Flujo	Límite	Origen
Archivos de estudios del diagnóstico	5 MB por archivo	`UPLOAD_MAX_MB_ESTUDIOS`

Flujo	Límite	Origen
Documentos del paciente y adjuntos de admisión	20 MB por archivo; 10 adjuntos por envío	`DOCUMENT_MAX_MB`
Escaneo móvil (PDF del teléfono)	20 MB	`escaneoMovil.routes.js`
Lectura avanzada (imagen)	8 MB	`lecturaExpediente.controller.js`
Cuerpo JSON	100 KB	valor por defecto de `express.json()`
Cuerpo en Nginx	20 MB	`client_max_body_size 20m`

8.7 Auditoría

Registro	Qué guarda	Consulta
`LogActividad`	inicio y cierre de sesión, recuperación de contraseña, altas y cambios de pacientes, ingresos, documentos, facturas, ventas, gastos y más	Reportes → Actividad (Administrador)
`AccesoDiagnostico`	cada lectura y registro del diagnóstico: usuario, paciente, fecha y si fue con token	Reportes → Auditoría del diagnóstico
`ultimaActividad` del usuario	última petición autenticada	Seguridad y roles (estado en línea)

`registrarActividad` no bloquea la operación principal: si falla, escribe el error en consola.

8.8 Límites de intentos

Implementación **en memoria** por IP (`req.ip`):

Endpoint	Ventana	Máximo	Modo
`POST /api/auth/login`	5 min	10 fallos	cuenta solo respuestas con código 400 o mayor
`POST /api/auth/forgot-password` y `reset-password`	5 min	5 peticiones	cuenta todas
Escaneo móvil público (`info`, `subir`)	5 min	30 peticiones	cuenta todas

Al excederse responde `429` con `Retry-After`. Los contadores se pierden al reiniciar y no se comparten entre instancias. Detrás de Nginx, `req.ip` es la IP del proxy porque no se configuró `trust proxy`, de modo que todos los clientes comparten el mismo contador de login (R14).

8.9 Red, CORS y TLS

- `app.use(cors())` sin opciones: acepta cualquier origen (R12).
- El backend no implementa TLS; el HTTPS lo proporciona el servidor de borde descrito en la sección 2.9.
- Nginx no añade cabeceras de seguridad (`X-Content-Type-Options`, `Content-Security-Policy`, `Strict-Transport-Security`).
- Solo el frontend publica un puerto en Docker Compose; PostgreSQL solo es alcanzable desde la red interna.

9. Pruebas

Resumen verificado el 26/10/2026 sobre el commit `3f0bf85`: **51** pruebas unitarias del backend, **17** del frontend, **18** pruebas E2E con Playwright y **1** prueba de integración (script de humo) con **115** verificaciones, todas en verde en una ejecución local.

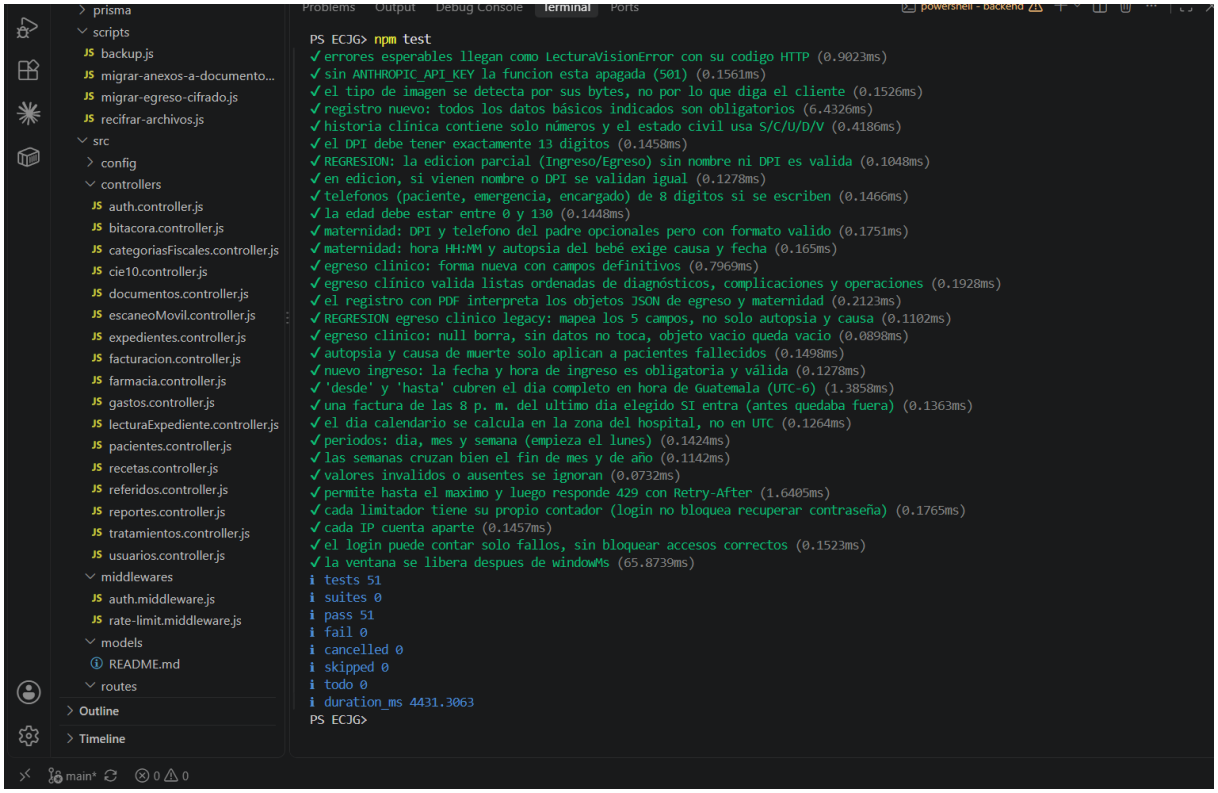
9.1 Matriz de pruebas

Suite	Ubicación	Comando	Requiere	Cubre
Unitarias backend	`backend/tests/unit/*.test.mjs`	`npm test` (en `backend/`)	nada	validación de pacientes, cifrado, archivos, límite de intentos, periodos, escaneo móvil, lectura de visión
Integración backend	`backend/tests/integracion.smoke.mjs`	`BASE=http://localhost:8080/api npm run test:integracion`	stack levantado y contraseña del administrador	flujo completo: acceso, pacientes, ingresos, expediente con tokens, documentos, tratamientos, recetas, facturación, farmacia, reportes y escaneo
Unitarias frontend	`frontend/tests/*.test.mjs`	`npm test` (en `frontend/`)	nada	fechas, validadores y OCR de datos básicos
E2E	`frontend/tests/*.spec.ts`	`npm run test:e2e`	stack levantado y contraseña del administrador	autenticación, pacientes, expediente, escáner y modo restringido

Las siguientes capturas muestran la salida de cada suite al ejecutarla en la terminal de Visual Studio Code.

Figura 23

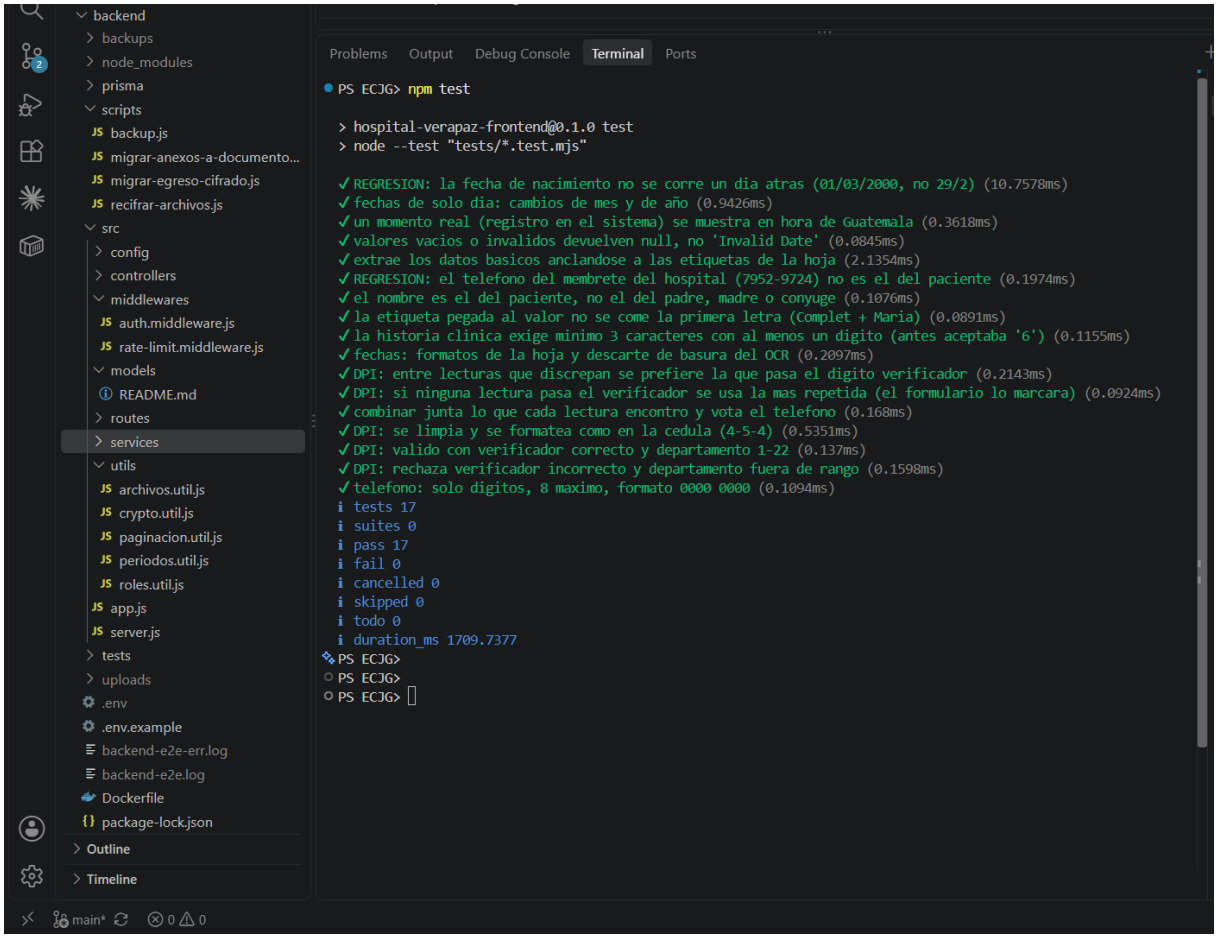
Pruebas unitarias del backend: 51 de 51 aprobadas



Nota. Captura de la terminal de Visual Studio Code, ejecución local.

Figura 24

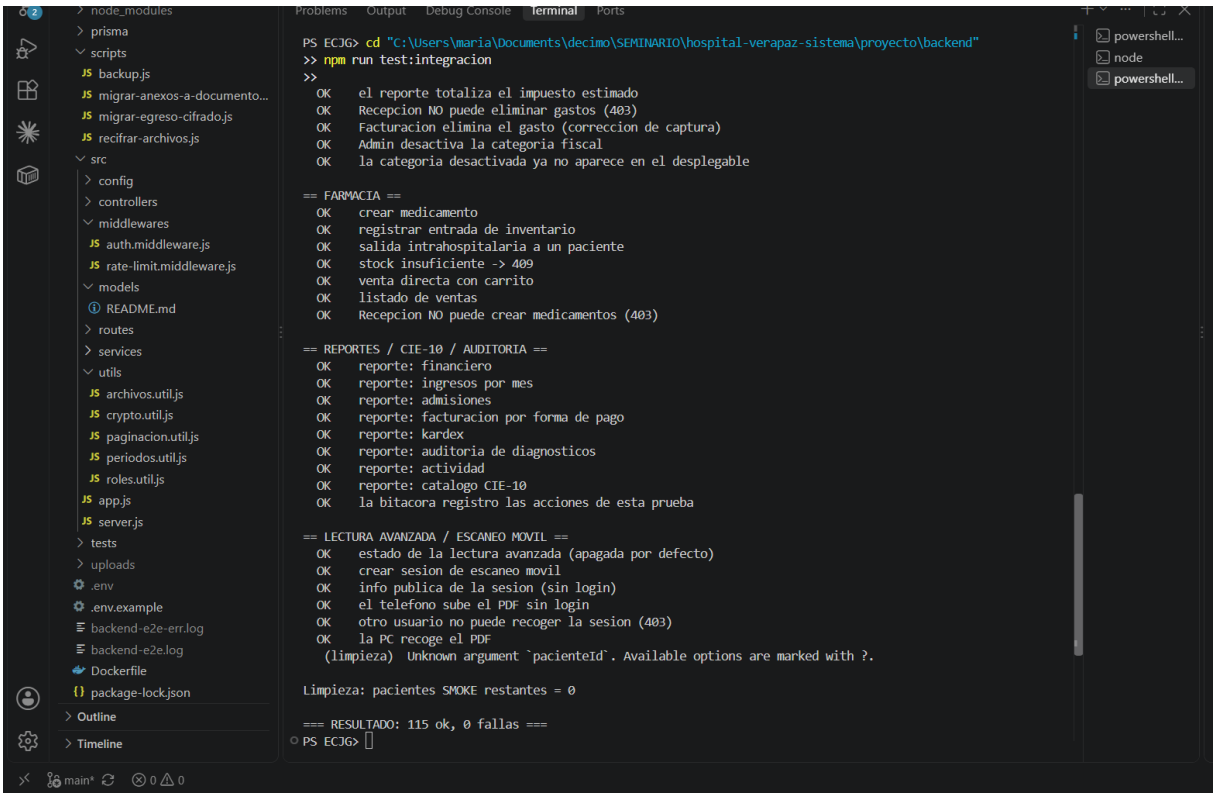
Pruebas unitarias del frontend: 17 de 17 aprobadas



Nota. Captura de la terminal de Visual Studio Code, ejecución local.

Figura 25

Prueba de integración: 115 verificaciones aprobadas, 0 fallas



Nota. Captura de la terminal de Visual Studio Code, ejecución local.

9.2 Prueba de integración

- `BASE` vale por defecto `http://localhost:4000/api`; para el stack Docker use `http://localhost:8080/api`.
- `SEED_ADMIN_PASSWORD` debe coincidir con la contraseña del administrador en la base.
- La prueba crea datos con el prefijo `SMOKE` y los elimina al final con `limpiar()`, en un orden que respeta las claves foráneas.
- Los endpoints de recuperación permiten 5 peticiones cada 5 minutos por IP, de modo que repetir la prueba antes de esa ventana produce 429. Si quedan pacientes `SMOKE` de una ejecución interrumpida, el alta siguiente falla por DPI duplicado y las demás fallan en cascada; hay que limpiarlos primero.
- La limpieza imprime `Unknown argument pacienteId` para `movimientoInventario`. Es inofensivo, porque lo captura `intentar()`, y queda pendiente de corregir.

9.3 Pruebas E2E (Playwright)

- `playwright.config.ts` usa `PLAYWRIGHT_BASE_URL` o `http://localhost:8080`.

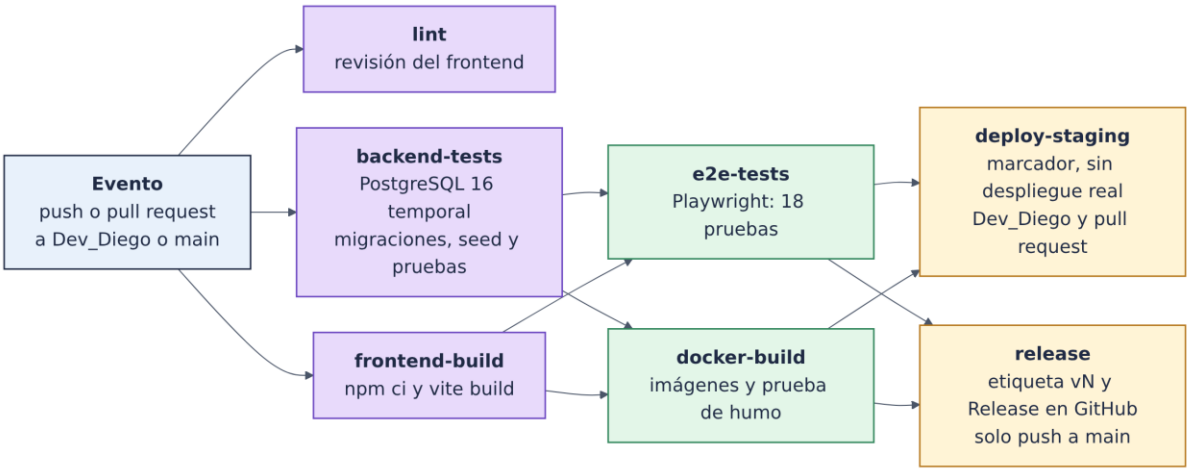
- Las pruebas de inicio de sesión necesitan `SEED_ADMIN_PASSWORD`; sin ella `restricted-mode.spec.ts` falla.
- Archivos: `auth.spec.ts`, `expediente.spec.ts`, `pacientes.spec.ts`, `restricted-mode.spec.ts` y `scanner.spec.ts`.
- Se observó un fallo intermitente en "Logo del hospital es un botón clicable", que no se reprodujo al ejecutarlo aislado. Se trata como prueba inestable.

10. Integración continua

El archivo `.github/workflows/ci.yml` se dispara con `push` y `pull_request` sobre las ramas `Dev_Diego` y `main`.

Figura 26

Flujo del pipeline de integración continua en GitHub Actions



Nota. Elaboración propia.

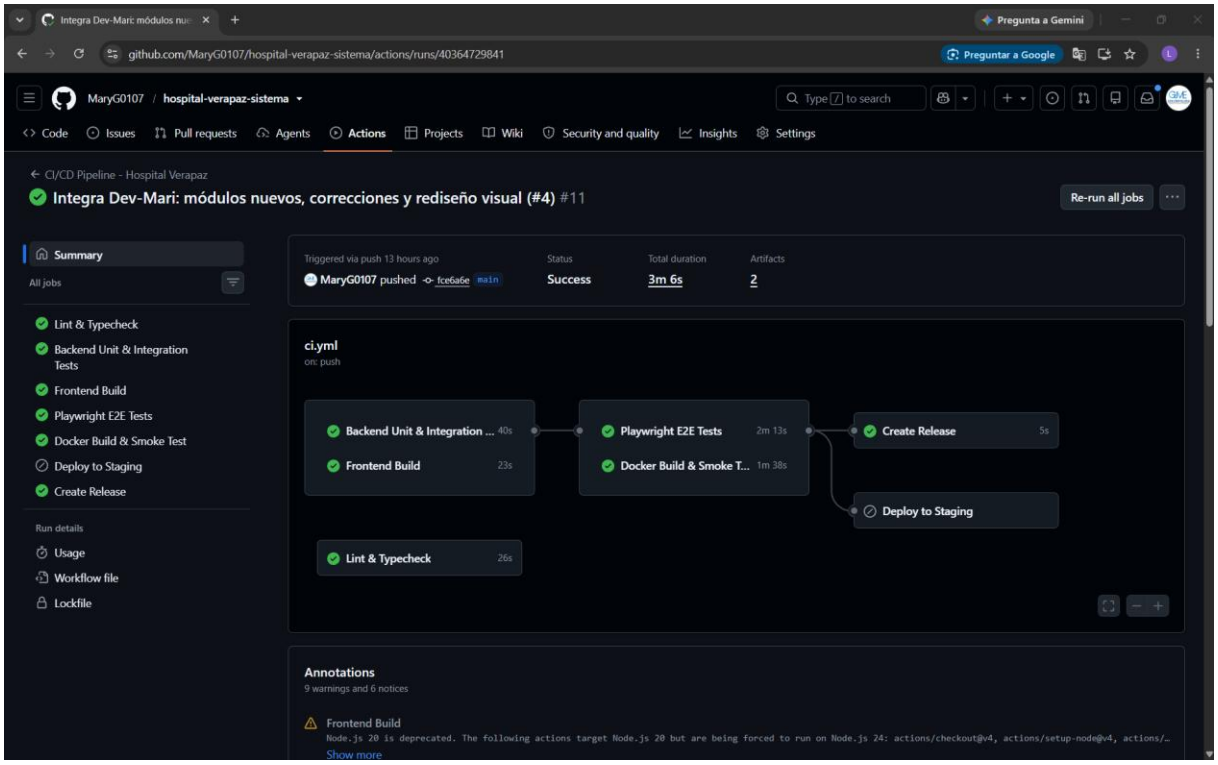
Job	Qué hace	Estado
`lint`	`npm run lint` y `tsc --noEmit` con respaldo mediante `echo`	**Marcador:** el frontend no define `lint`, así que el job nunca falla
`backend-tests`	PostgreSQL 16 como servicio; `prisma generate`, `migrate deploy`, seed y `npm test`	Activo
`frontend-build`	`npm ci` y `npm run build`; sube `dist` como artefacto	Activo
`e2e-tests`	Levanta el backend y sirve el frontend en 5173; ejecuta Playwright. Necesita `frontend-build` y `backend-tests`	Activo
`docker-build`	`docker compose build` y `up -d`; verifica `/api/health`, `/` y el MIME de `pdf.worker`; `down -v`. Necesita `frontend-build` y `backend-tests`	Activo
`deploy-staging`	Solo imprime un mensaje. Necesita `e2e-tests` y `docker-build`; corre en `Dev_Diego` o en un pull request	**Marcador:** requiere configurar el destino
`release`	Crea la release con `actions/create-release@v1`. Necesita `e2e-tests` y `docker-build`; corre solo en un push a `main`	Acción archivada; conviene sustituirla por `gh release create`

Secretos requeridos en GitHub: `POSTGRES_PASSWORD`, `JWT_SECRET`, `ENCRYPTION_KEY` (64 hex) y `SEED_ADMIN_PASSWORD`.

10.1 Evidencia de la integración continua

Figura 27

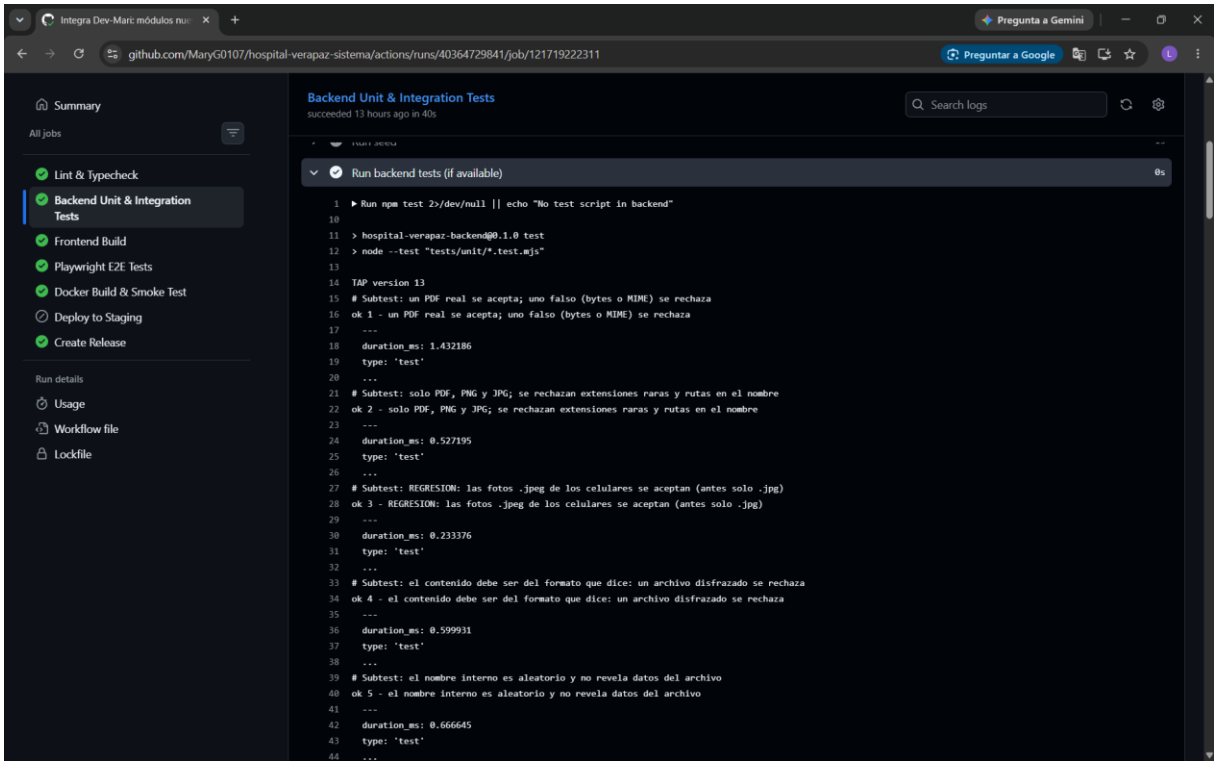
Ejecución del pipeline sobre main (commit fce6a6e): todos los jobs en verde, duración total de 3 min 6 s



Nota. Captura de GitHub del repositorio del sistema.

Figura 28

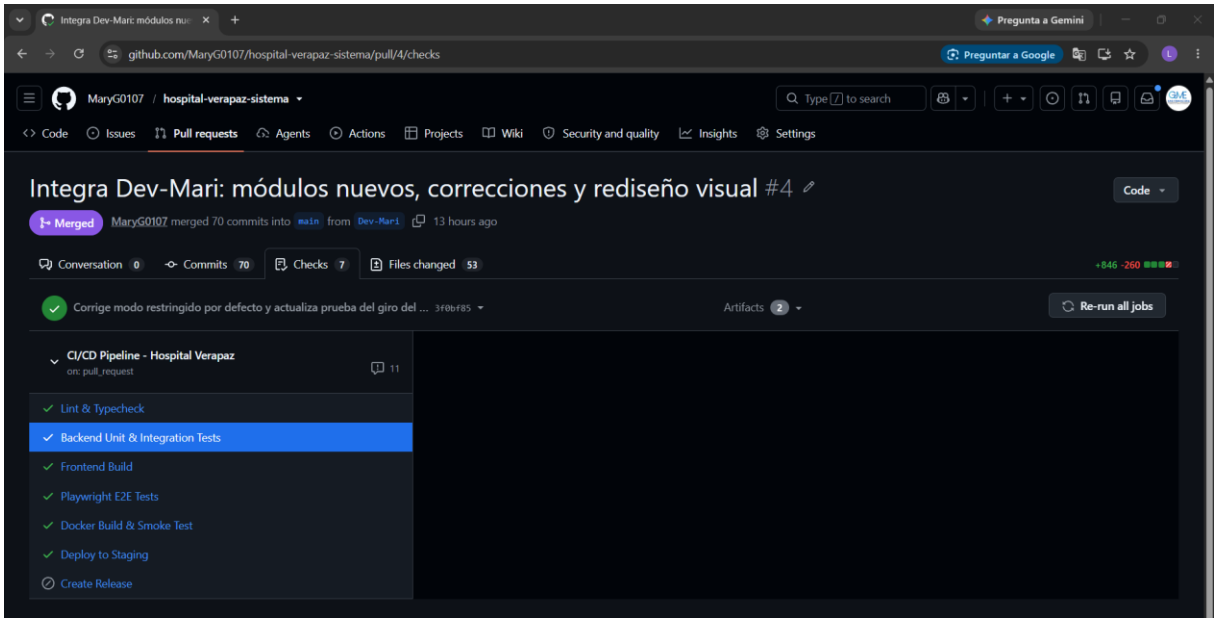
Detalle del job Backend Unit & Integration Tests, con el registro de las pruebas aprobadas



Nota. Captura de GitHub del repositorio del sistema.

Figura 29

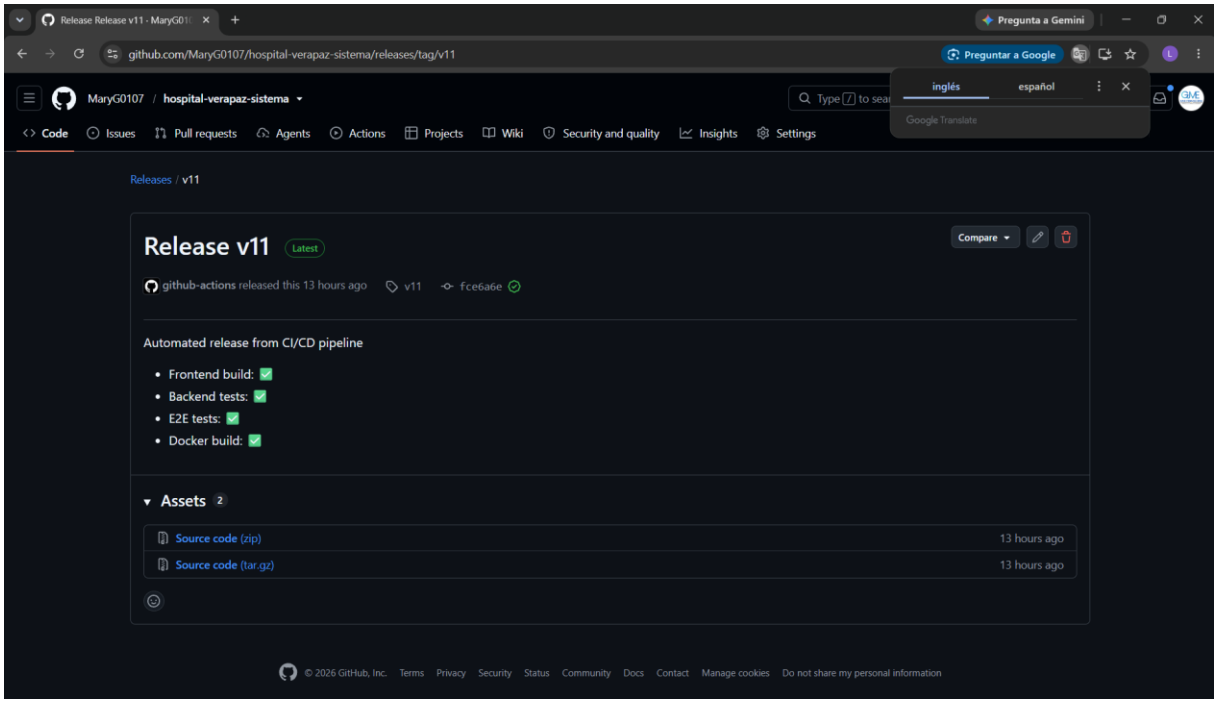
Pull request #4 fusionado en main, con los 7 checks del pipeline aprobados



Nota. Captura de GitHub del repositorio del sistema.

Figura 30

Release v11 publicada automáticamente por el pipeline sobre el commit fce6a6e



Nota. Captura de GitHub del repositorio del sistema.

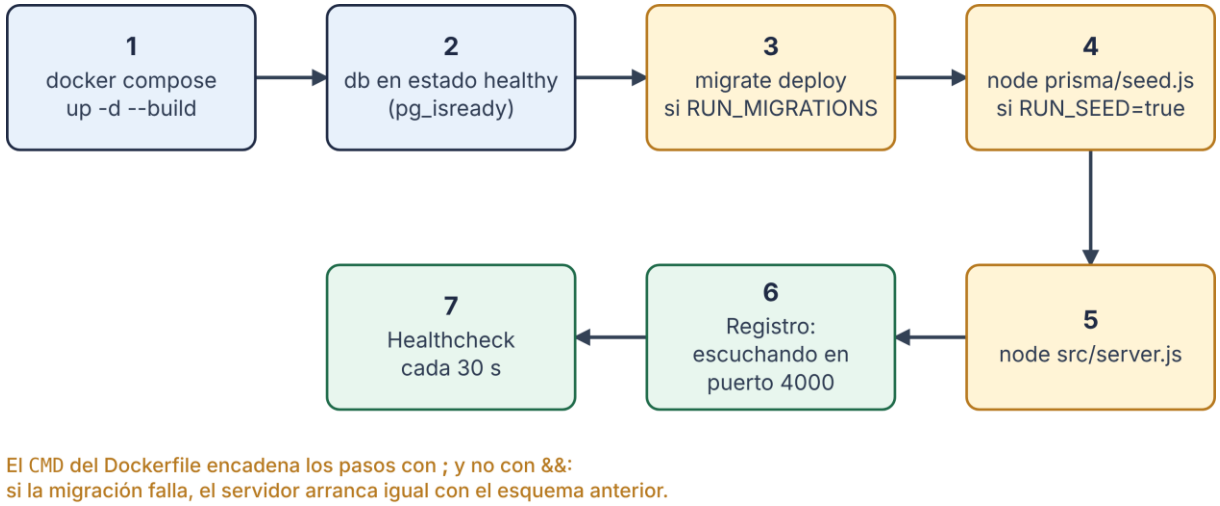
11. Operación y mantenimiento

11.1 Monitoreo

- **Salud:** `curl -fsS http://<host>:8080/api/health` devuelve 200 si la API y la base responden, y 503 si no.
- **Healthcheck del contenedor backend:** `wget` contra `http://127.0.0.1:4000/api/health` cada 30 segundos (definido en el `Dockerfile`).
- **Registros:** `docker compose logs -f backend` incluye los errores 500 y los enlaces de recuperación en modo `consola`.
- **Estado:** `docker compose ps`.

Figura 31

Secuencia de arranque del backend dentro del contenedor.



Nota. Elaboración propia.

11.2 Respaldos

Base de datos. `backend/scripts/backup.js` ejecuta `pg_dump` con `DATABASE_URL` y escribe `backend/backups/verapaz_<fecha>.sql`:

```
cd backend
npm run backup
```

El script requiere `pg_dump` en el equipo que lo ejecuta (busca también en las rutas de PostgreSQL 16 a 18 de Windows). La imagen `node:22-alpine` del backend no lo incluye, así que `npm run backup` sirve en desarrollo local. En el despliegue con Docker la base no publica puertos y el respaldo se hace dentro del contenedor de la base (procedimiento estándar de PostgreSQL, no verificado en el proyecto):

```
docker compose exec -T db pg_dump -U verapaz hospital_verapaz > verapaz_$(date +%Y%m%d_%H%M).sql
```

El proyecto no incluye respaldos programados; se debe configurar una tarea periódica (cron).

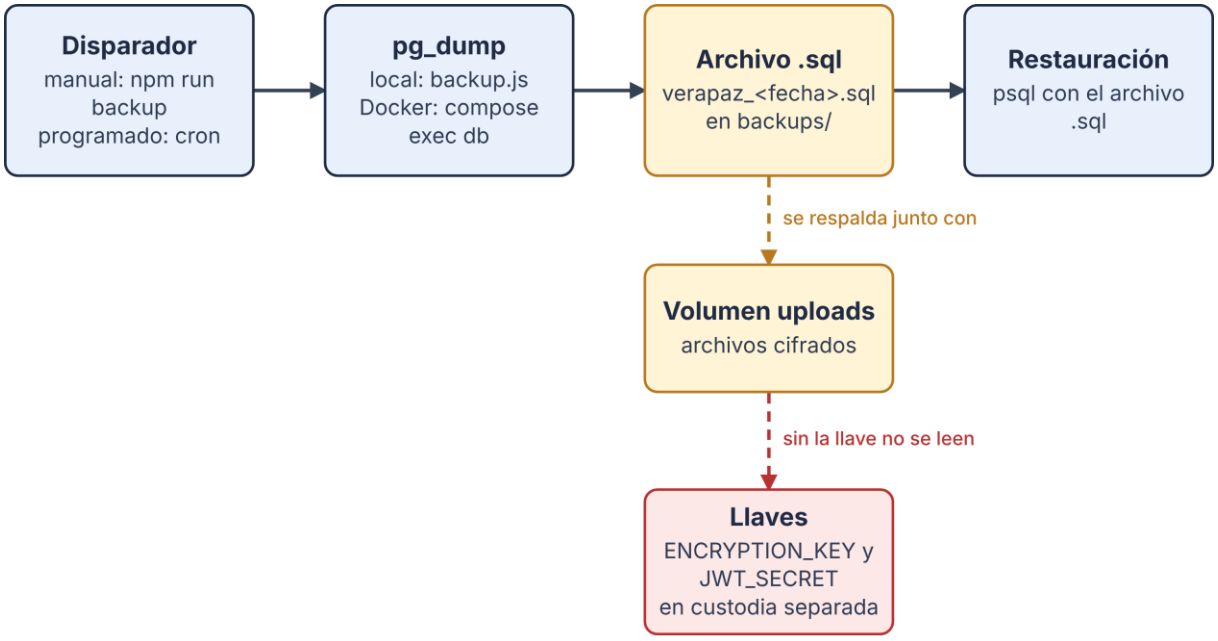
Restauración (procedimiento estándar de PostgreSQL, no verificado en el proyecto):

```
docker compose exec -T db psql -U verapaz -d hospital_verapaz_restore < verapaz_<fecha>.sql
```

Archivos. Los volúmenes `uploads` (`/app/uploads`) y `backups` (`/app/backups`) se respaldan junto con la base. Los archivos de `uploads/` están cifrados: **sin `ENCRYPTION_KEY` no sirven**. Guarde `ENCRYPTION_KEY` y `JWT_SECRET` en una custodia separada del respaldo.

Figura 32

Flujo de respaldo y restauración de la base de datos y de los archivos cifrados.



Nota. Elaboración propia.

11.3 Scripts de mantenimiento

Se ejecutan una sola vez y con respaldo previo, desde `backend/` con `DATABASE_URL` y `ENCRYPTION_KEY` definidos.

Script	Comando	Función	Estado
`scripts/migrar-egreso-cifrado.js`	`npm run migrar-egreso`	Pasa los campos de egreso de `Paciente` al `EgresoClinico` cifrado; no borra las columnas	Histórico; confirmar antes de repetir
`scripts/migrar-anexos-a-documentos.js`	`npm run migrar-anexos`	Migra `AnexoPaciente` a `DocumentoPaciente`	Histórico; la tabla ya no existe
`scripts/recifrar-archivos.js`	`npm run recifrar-archivos`	Cifra los archivos de `uploads/diagnosticos` que quedaron en claro, validando el checksum antes de borrar el original; idempotente	Verificar si quedan archivos en claro

11.4 Rotación de llaves

- **`JWT_SECRET`:** cambiarlo invalida todas las sesiones activas. Es seguro hacerlo en una ventana de mantenimiento.
- **`ENCRYPTION_KEY`:** cambiarla sin re-cifrar los datos deja ilegibles los diagnósticos, el egreso clínico y los archivos. El proyecto no incluye un procedimiento de rotación; debe diseñarse y probarse antes de intentarlo.
- **Cuentas de prueba:** cambie o elimine las cuentas del seed antes de operar con datos reales.

11.5 Notificaciones, lectura avanzada y escaneo móvil

- **Notificaciones.** Con `NOTIFICACION_PROVEEDOR=consola`, único proveedor implementado, los enlaces de recuperación se escriben en el registro del backend. Para enviar correo hay que agregar un caso en `enviarEnlaceRecuperacion`.
- **Lectura avanzada.** Se activa solo con `ANTHROPIC_API_KEY`. Con la clave definida, la primera página del expediente escaneado se envía a Anthropic; es una decisión de privacidad del hospital (R15).
- **Escaneo móvil.** Las sesiones viven en la memoria del proceso. Un reinicio del contenedor las cancela y hay que generar una nueva.

12. Guía para desarrolladores

12.1 Agregar un endpoint

1. Declare la ruta en `routes/<modulo>.routes.js` con `requireAuth` y `requireRole(...)`. Si da acceso al diagnóstico, use `requireTempToken`.
2. Implemente el controlador en `controllers/<modulo>.controller.js`: valide la entrada, responda con `{error}` en los fallos y use `registrarActividad` en altas y cambios.
3. Si la operación toca varias tablas, use `prisma.$transaction`.
4. Agregue pruebas unitarias para la validación pura (patrón de `pacientes.validacion.test.mjs`) y un caso en `integracion.smoke.mjs`.

12.2 Agregar un modelo o cambiar el esquema

1. Edite `prisma/schema.prisma`.
2. Ejecute `npx prisma migrate dev --name <nombre>` en desarrollo.
3. Revise el SQL generado; si hay datos existentes, agregue el relleno a mano (ejemplo: `20261013120000_paso15_ingresos_paciente`).
4. Versione la carpeta de la migración junto con el código.
5. Verifique con `npx prisma migrate deploy` en una base limpia y en una con datos.

12.3 Agregar un rol

1. Agréguelo en `backend/src/utils/roles.util.js` y en `frontend/src/utils/roles.js` (`ROLES`, `ROLE_LABELS` y `MODULOS_POR_ROL`).
2. Ajuste las rutas del backend que deban aceptarlo.
3. Ajuste las páginas del frontend que muestran acciones según el rol.
4. Actualice la sección de roles del Manual de usuario.

12.4 Convenciones observadas

- Comentarios y mensajes de error en español, con referencias a requerimientos (RF-xx, RNF-xx).
- Errores de negocio con código HTTP explícito; no dejar excepciones sin mapear en controladores públicos.
- Montos con `Decimal` en la base y `round2` antes de persistir totales.

13. Limitaciones y riesgos

13.1 Riesgos de seguridad

Evaluación contra el código al commit `3f0bf85`. El hallazgo R5 quedó **corregido** (`GET /api/facturacion` exige Administrador o Facturación) y se conserva por trazabilidad. Severidad: A alta, M media, B baja.

ID	Sev.	Hallazgo	Evidencia	Recomendación
R1	A	`Ingreso.impresionClinicaIngreso` y `Paciente.impresionClinicaIngreso` se guardan en texto plano. `GET /api/pacientes/:id` incluye los ingresos completos y el listado incluye la impresión; ambos son accesibles para cualquier rol autenticado	`pacientes.controller.js` (`obtenerUno`), `schema.prisma`	Cifrar la impresión clínica como el diagnóstico, o retirarla de la respuesta para roles sin acceso clínico
R2	A	Los documentos del paciente (resultados clínicos adjuntos y expediente escaneado) se listan y descargan sin token temporal por Consulta y Enfermería	`pacientes.routes.js`, `documentos.controller.js`	Definir la clasificación; si son clínicos, exigir token o moverlos a los estudios del diagnóstico
R3	M	Los resultados clínicos adjuntos se guardan como `DocumentoPaciente` de tipo `resultado_clinico`, aunque el modelo indica que los resultados de exámenes no deben ir en esa tabla	`pacientes.controller.js`, `schema.prisma`	Decisión pendiente (D2)
R4	M	`GET /api/pacientes` y `/:id` solo exigen autenticación: cualquier rol, incluido Farmacia, ve datos administrativos completos	`pacientes.routes.js`	Restringir por rol o devolver un DTO mínimo
R5	M	`GET /api/facturacion` solo exigía autenticación	`facturacion.routes.js`	**Corregido:** ahora exige ADM o FAC
R6	M	Un token emitido desde la interfaz no lleva `pacienteId`; con él se puede consultar cualquier paciente durante 15 minutos	`Seguridad.jsx`, `auth.controller.js`	Hacer obligatorio `pacienteId` al emitir desde la interfaz

ID	Sev.	Hallazgo	Evidencia	Recomendación
R7	M	El seed imprime en consola la contraseña del administrador y crea cuentas con contraseñas fijas; `RUN_SEED` vale `true` por defecto	`seed.js`, `docker-compose.yml`	`RUN_SEED=false` tras la instalación; cambiar contraseñas; no usar cuentas de prueba en producción
R8	B	`requireTempToken` marca el token como usado antes del controlador; si la operación falla después (por ejemplo, un 400), el token se pierde	`auth.middleware.js`	Consumirlo al final, dentro de la misma transacción
R9	B	En `POST /expedientes/paciente/:id`, multer procesa el cuerpo antes de `requireTempToken`	`expedientes.routes.js`	Invertir el orden: primero el token
R10	B	La creación de usuarios no valida longitud mínima de contraseña	`usuarios.controller.js`	Aplicar la regla de 8 caracteres del reset
R11	B	El manejador de errores devuelve `err.message` en los 500	`app.js`	Mensaje genérico y detalle solo en el registro del servidor
R12	B	CORS abierto y sin cabeceras de seguridad en Nginx	`app.js`, `nginx.conf`	Restringir `origin` a `APP_URL` y añadir cabeceras básicas
R13	B	JWT sin revocación; el cierre de sesión no invalida el token	`auth.controller.js`	Lista de revocación o sesiones en base de datos
R14	B	Límite de intentos en memoria y sin `trust proxy`: el contador de login se comparte entre todos los clientes	`rate-limit.middleware.js`	`app.set("trust proxy", 1)` y un almacén compartido
R15	Info	La lectura avanzada envía la primera página del expediente a un servicio externo cuando `ANTHROPIC_API_KEY` está definida	`lecturaVision.service.js`	Documentar la decisión formal antes de activarla
R16	M	La interfaz muestra *Autogenerar token*, pero `POST /auth/token/auto` exige Administrador; los demás roles reciben 403	`auth.routes.js`	Quitar el botón para otros roles o devolver el permiso a la ruta
R17	A	`DELETE /api/pacientes/:id` borra físicamente al paciente con diagnósticos, ingresos y **facturas del hospital**, lo que contradice la retención de la sección 4.8. La ejecuta Recepción y no deja copia	`pacientes.controller.js` (`eliminar`)	Restringirla al Administrador, exigir confirmación y respaldo previo, o sustituirla por baja lógica que conserve las facturas

13.2 Deuda técnica

ID	Tema	Descripción	Acción sugerida
D1	Datos duplicados	Con `RUN_SEED=true`, cada arranque vuelve a insertar el inventario inicial porque `MedicamentoInventario.nombre` no es único y `skipDuplicates` no lo detecta. En la base de desarrollo cada medicamento inicial tiene 12 copias	`RUN_SEED=false` tras la instalación; corregir el seed con una restricción única; depurar duplicados con cuidado, pues pueden tener movimientos y ventas

ID	Tema	Descripción	Acción sugerida
D2	Decisión pendiente	Los resultados clínicos adjuntos en `DocumentoPaciente` (R3) contradicen el modelo	Decidir si pasan a `DiagnosticoArchivo` o se reclasifica la tabla
D3	Documentación	El `README.md` está desactualizado: habla de 9 módulos (ahora son 11), indica `npm run test` para E2E (el script es `test:e2e`) y que la navegación no usa rutas. Los comentarios de `app.js` citan un nombre de documento que no coincide con el archivo real	Actualizar el README y los comentarios tomando este manual como referencia
D4	Límite de páginas	`DOCUMENT_MAX_PAGES` se valida en `POST /pacientes/:id/documentos`, pero no en el alta de paciente con escaneo	Aplicar la misma validación en `crear`
D5	Variables sin uso	`UPLOAD_MAX_MB_ANEXOS` de `.env.example` no se usa	Eliminarla
D6	Bitácora	El módulo no figura en ningún menú, aunque su ruta, su API y su tabla siguen activas y solo el Administrador la abre por URL	Decidir si se retira en una versión posterior (requiere migrar datos) o se agrega al menú del Administrador
D7	Pruebas	La prueba de integración depende del orden y de los límites de tasa (9.2); la limpieza falla de forma inofensiva sobre `movimientoInventario`	Aislar los datos por ejecución y reducir el límite en el entorno de pruebas
D8	CI	`lint` y `deploy-staging` son marcadores y `release` usa una acción archivada	Ver la sección 10
D9	Errores	`app.js` expone `err.message` en 500 (R11)	Mensaje genérico y registro interno
D10	Escalabilidad	Límites de intentos, sesiones de escaneo y contadores viven en memoria y no escalan a varias instancias	Mover a un almacén compartido (Redis) si se requiere más de una instancia
D11	Auditoría de lectura	Los accesos a documentos del paciente no quedan en `AccesoDiagnostico`; solo se registran las descargas en la bitácora general	Evaluar una auditoría de lectura para documentos clínicos (R2)
D12	Impresión clínica	La impresión clínica de ingreso existe en `Paciente` y en `Ingreso`, en texto plano	Unificar en `Ingreso` y cifrar (R1)
D13	Columnas legacy	`Paciente` conserva `diagnosticoEgresoCodigo`, `complicacionesCodigo`, `operacionesCodigo`, `autopsia` y `causaMuerte` en texto plano; `egresoClinicoPara` las usa de respaldo	Confirmar con conteos que la migración cubrió a todos y retirar las columnas en una migración con respaldo
D14	Arranque	El `CMD` del backend encadena `prisma migrate deploy`, el seed y el servidor con `;`. Si la migración falla, el servidor arranca con un esquema incompatible con el código	Cambiar a `set -e` o a `&&`; mientras tanto, revisar los registros tras cada actualización

14. Glosario

Término	Definición en este sistema
Admisión / Ingreso	Una estadía del paciente en el hospital. Un paciente puede tener varias
AES-256-GCM	Algoritmo de cifrado autenticado con el que se protegen diagnósticos, egreso clínico y archivos
Costeo	Suma de los tratamientos intrahospitalarios y de farmacia pendientes de facturar
Egreso clínico	Diagnósticos, complicaciones, operaciones, autopsia y causa de muerte al salir del hospital; se guarda cifrado
Expediente	Conjunto de diagnósticos (versiones) y estudios de un paciente

Término	Definición en este sistema
Historia clínica (HC)	Identificador numérico único del paciente en el hospital; si no se captura, el sistema genera `HC-AAAA-NNNNNN`
JWT	Token firmado que identifica la sesión; dura 8 horas
Kardex	Historial de movimientos de inventario de un medicamento
Sobre cifrado	Formato de archivo `[IV][etiqueta][texto cifrado]` en disco
Token temporal	Credencial de 15 minutos y uso único para leer o registrar un diagnóstico
SPA	Aplicación de una sola página; el frontend de Madián
RF / RNF	Requerimiento funcional / no funcional

Requerimientos citados en el código: RF-01 a RF-04 (pacientes y DPI), RF-10 y RF-11 (diagnóstico confidencial), RF-15 (cargos intrahospitalarios), RF-16 (referidos), RF-17 a RF-19 (facturación), RF-20, RF-24 y RF-27 (farmacia), RF-21 (reporte financiero), RF-22 y RF-23 (inventario), RF-25 y RF-26 (alertas), RF-29 (inicio de sesión), RF-30 y RF-32 (roles y usuarios), RF-31 (reportes), RF-33 y RF-34 (tokens), RNF-03 (respaldos), RNF-08 (auditoría), RNF-10 (kardex), RNF-12 (uso único) y RNF-13 (transacciones).